NEXORA PLATFORM

NEXORA

The Next Way to Work Together

Complete System Documentation

VERSION

Unversioned — see Document Control

DOCUMENTATION DATE

17 September 2026

SYSTEM STATUS

Active development · Pre-1.0

SOURCE COMMIT

30bfc83

Laravel 12 · Livewire 4 · Tailwind CSS 4 · MySQL · Reverb · Anthropic & OpenAI · GitHub · Slack · Docker on Railway

Contents

Nexora — Complete System Documentation · 34 sections

1 Document Control 10

1.1 Revision identity 10
1.2 Technology stack 10
1.3 Scale of the implementation 11

2 Executive Overview 13

2.1 What Nexora is 13
2.2 The problem it solves 13
2.3 Who uses it 13
2.4 Core workflows 14
2.5 Major capabilities, at a glance 14

3 System Architecture 16

3.1 The shape of the application 16
3.2 Layers, and where the rules live 16
3.3 Why each technology is here 17
3.4 Request lifecycle 18
3.5 Domain model map 19

4 User Roles and Permissions 20

4.1 The three roles 20
4.2 How access is decided 20
4.3 Permission matrix — workspace and boards 22
4.4 Permission matrix — tickets 22
4.5 Permission matrix — conversations, documentation, files 24
4.6 Permission matrix — AI 25
4.7 Permission matrix — reporting and administration screens 26
4.8 Coarse gates 27

5 Authentication and Security 28

5.1 Authentication 28
5.2 Authorization 30
5.3 CSRF protection 32
5.4 Webhook signature verification 32
5.5 Rate limiting 32
5.6 Input validation and output sanitisation 33
5.7 Secret handling 35
5.8 The GitHub token is deliberately weak 35

5.9 File upload security 36
5.10 Storage durability is enforced at boot 36
5.11 HTTP security headers 37
5.12 Transport security 37
5.13 Realtime authorization 37

6 Dashboard and Workspace Shell 38

6.1 There is no "workspace" entity 38
6.2 Navigation 38
6.3 Dashboard 38
6.4 Command palette and global search 38
6.5 Notifications 39
6.6 Activity feed 40
6.7 Settings 40
6.8 Profile 41
6.9 Responsive behaviour 41

7 Board Management 42

7.1 Purpose 42
7.2 Who can do what 42
7.3 Board lifecycle 43
7.4 Tabs 44
7.5 Columns 46
7.6 Labels 47
7.7 Members 47
7.8 Board visibility and customers 47
7.9 The Kanban board 48
7.10 Repository attachment 49

8 Ticket Management 50

8.1 Anatomy of a ticket 50
8.2 Ticket numbering 51
8.3 Ticket lifecycle 51
8.4 Creating a ticket 51
8.5 Editing a ticket 52
8.6 Ticket addressing 52
8.7 Labels, checklists, links and attachments 52
8.8 The two conversation streams 53
8.9 Ticket activity timeline 53
8.10 Filtering and search 54
8.11 What happens when a ticket changes 54
8.12 In-app notifications 55

9 Documentation System 56

9.1 Purpose 56
9.2 Layout 56
9.3 Page hierarchy 56
9.4 The ancestor visibility rule 57
9.5 The editor is not a mode 57
9.6 Autosave, drafts and publishing 58
9.7 Contents list ("On this page") 59
9.8 Permissions 59
9.9 Importing files as pages 59
9.10 Attachments 61
9.11 Search 61
9.12 What is not implemented 61

10 Workspace AI 62

10.1 Three settings that are not the same question 62
10.2 AI architecture 65
10.3 The two surfaces 67
10.4 Context scope 67
10.5 Sessions and conversation history 68
10.6 The retrieval tool loop 70
10.7 Write tools: propose, then confirm 77
10.8 Configuration inheritance 80
10.9 Providers and models 81
10.10 Usage tracking and audit 82
10.11 Streaming 82
10.12 Error handling 82

11 AI Ticket Runs 84

11.1 The two modes 84
11.2 Triggers 84
11.3 Run lifecycle 85
11.4 Apply mode invariants 85
11.5 Isolation and cleanup 86
11.6 Duplicate execution 86
11.7 Retries 86
11.8 Stalled runs 87
11.9 Visibility 87
11.10 Results 87

12 AI Visualisation, Voice and Attachments 89

12.1 AI-generated charts, tables and KPI rows 89

12.2 Voice 91
12.3 AI conversation attachments 92

13 GitHub Integration 95

13.1 Inbound: how a reference becomes a link 96
13.2 Outbound: opening a pull request 98
13.3 Security model 99

14 Slack Integration 101

14.1 Design 101
14.2 Flow 101
14.3 Events 101
14.4 What a message contains, and why it is thin 102
14.5 Configuration and security 102

15 SMS Alerts 104

15.1 Design 104
15.2 Providers 104
15.3 Recipients 104
15.4 Duplicate suppression 104
15.5 Data protection 105
15.6 Other controls 105

16 Statistics and Reporting 106

16.1 Two screens, not one screen with a role switch 106
16.2 Metrics 106
16.3 Filters 108
16.4 Export 108

17 Realtime 110

17.1 Transport 110
17.2 Channels 110
17.3 The audience rule 111
17.4 What updates live 111

18 Theme System 112

18.1 Where the preference lives 112
18.2 How a change is saved 112
18.3 Security note 113
18.4 Coverage 113

19 Notifications, Queues and Scheduled Work 114

19.1 Why background workers are required 114
19.2 The three queues 114

19.3 Jobs 115
19.4 Failed jobs 116
19.5 Scheduled tasks 116
19.6 Console commands 116
19.7 In-app notification architecture 117

20 Files and Attachments 118

20.1 Two pipelines, one storage service 118
20.2 Storage rules 118
20.3 Disks 118
20.4 Download security 119
20.5 Allowed types 119
20.6 Upload buffering 120

21 Database Architecture 121

21.1 Entity relationship overview 121
21.2 Core tables 121
21.3 AI tables 125
21.4 OAuth tables (remote MCP server) 126
21.5 Integration tables 127
21.6 Framework tables 127
21.7 Referential integrity strategy 127
21.8 Notable migration decisions 127

22 Deployment Architecture 129

22.1 Production topology 129
22.2 The Docker build 129
22.3 Entrypoint sequence 131
22.4 Web container processes 131
22.5 PHP and nginx tuning 132
22.6 Health checks 132
22.7 Restart policy 133
22.8 Deployment process 133
22.9 The board tabs release 134
22.10 Rollback considerations 134
22.11 Scaling notes 135

23 Environment Variables 136

23.1 Required 136
23.2 Workspace identity 136
23.3 Session and security 137
23.4 Storage 137
23.5 Queue, cache and realtime 138

23.6 AI — core 138
23.7 AI — limits and caps 139
23.8 AI — chat, tools and knowledge 139
23.9 AI — runs, code generation and repositories 140
23.10 AI — attachments and voice 141
23.11 Remote MCP server 142
23.12 GitHub 142
23.13 Slack and SMS 143
23.14 Mail, logging and seeding 143
23.15 Deployment-only 144

24 Installation and Local Development 145

24.1 Requirements 145
24.2 Quick start 145
24.3 Step by step 145
24.4 Code style 148
24.5 Useful environment settings for development 148

25 Testing 149

25.1 Suites 149
25.2 Running the tests 149
25.3 Current status 149
25.4 What is covered 150
25.5 Test support 152
25.6 Gaps 153

26 Troubleshooting 154

26.1 APP_KEY is missing 154
26.2 Database connection failure 154
26.3 Migration fails with "Base table or view not found: cache_locks" 154
26.4 The queue is not processing 155
26.5 AI attachments stay "pending" for ever 155
26.6 AI runs are stuck at "Running" 156
26.7 AI runs are refused 156
26.8 GitHub webhook returns 401 156
26.9 GitHub webhook returns 202 but nothing links 157
26.10 Slack notifications never arrive 157
26.11 Storage configuration failure 158
26.12 File uploads return 500, or silently vanish 158
26.13 Realtime never updates 158
26.14 Railway deployment fails the health check 159
26.15 Missing environment variables 159

26.16 Configuration changes have no effect 159

27 HTTP Endpoints and Webhooks 161

27.1 Remote MCP server — POST /mcp 161
27.2 Inbound webhook — GitHub 162
27.3 Health endpoints 163
27.4 Application endpoints 163
27.5 Browser routes 164
27.6 Outbound HTTP 166

28 Administrator Guide 167

28.1 Your responsibilities 167
28.2 Managing users 167
28.3 Managing boards 167
28.4 Global AI settings 168
28.5 Monitoring 169
28.6 Routine maintenance 169

29 Team Member Guide 170

29.1 What you can do 170
29.2 The one thing to get right 170
29.3 Splitting a board into tabs 171
29.4 Working a ticket 171
29.5 Using the assistant 171
29.6 Running the AI on a ticket 172
29.7 Documentation 172

30 Customer Guide 173

30.1 What Nexora is, for you 173
30.2 Signing in 173
30.3 Your dashboard 173
30.4 Raising a request 173
30.5 Following progress 173
30.6 Talking to the team 174
30.7 Documentation 174
30.8 Searching 174
30.9 Your progress summary 174
30.10 Asking the assistant 174
30.11 What you cannot do 175

31 User Workflows 176

31.1 Workflow 1 — Standing up a new customer project 176
31.2 Workflow 2 — A customer request, from filing to delivery 176

31.3 Workflow 3 — GitHub activity reaching a ticket 177
31.4 Workflow 4 — Asking the assistant, and confirming a change 178
31.5 Workflow 5 — An AI coding session 179
31.6 Workflow 6 — Publishing documentation to a customer 180

32 Implementation Status 182

32.1 Fully implemented 182
32.2 Partially implemented 183
32.3 Planned / Not Yet Implemented 184
32.4 Documentation gaps and inconsistencies found 185

33 Glossary 187

34 Appendix — Document Provenance 190

SECTION 1

Document Control

This document describes the Nexora platform **as it is actually implemented** in the repository at the commit named below. It was produced by reading the source: routes, configuration, models, migrations, services, policies, middleware, Livewire components, Blade views, JavaScript, container configuration and the test suite.

Nothing here describes a feature that does not exist in the code. Where a capability is present but incomplete it is marked **Partially Implemented**. Where a capability is designed for but absent it is marked **Planned / Not Yet Implemented**. Section 38 collects every such marking in one table.

1.1 Revision identity

FIELD	VALUE
Product name (documentation)	Nexora — The Next Way to Work Together
Package name (`composer.json`)	`aqueduct/workspace`
Runtime workspace name (`WORKSPACE_NAME`)	`Aqueduct Workspace` (deployment-configurable)
Version	**None declared.** No git tag, no `version` field in `composer.json`, no application version constant
Documentation basis	Commit `30bfc83` — *feat: migrate from wire:sort to x-sort for drag-and-drop functionality*
Commit date	11 September 2026
Repository age at time of writing	17 commits, first commit 1 September 2026
Licence	Proprietary
Documentation date	17 September 2026
System status	Active development, pre-1.0. No release tagging in place

IMPORTANT

The product is white-labelled, and the names do not currently agree. The shipped default for `WORKSPACE_NAME` is `Aqueduct Workspace`, and the Composer package is `aqueduct/workspace`. The name *Nexora* appears in the AI system prompt, in source comments and in one accessibility label, but it is not what the application renders by default. A deployment that should present as Nexora must set `WORKSPACE_NAME="Nexora"` and `WORKSPACE_SHORT_NAME="Nexora"`. See section 32.

1.2 Technology stack

LAYER	TECHNOLOGY	VERSION CONSTRAINT	SOURCE

Language	PHP	`^8.2` (container runs 8.3)	`composer.json`, `Dockerfile`
Framework	Laravel	`^12.0`	`composer.json`
Interactive UI	Livewire	`^4.4`	`composer.json`
Styling	Tailwind CSS	`^4.0`	`package.json`
Build tool	Vite	`^7.0.7`	`package.json`
Rich text editor	TipTap (ProseMirror)	`^3.31.3`	`package.json`
Realtime server	Laravel Reverb	`^1.11`	`composer.json`
Realtime client	Laravel Echo + pusher-js	`^2.4.0` / `^8.6.0`	`package.json`
Database	MySQL (`pdo_mysql`)	—	`Dockerfile`, `config/database.php`
Test database	SQLite in-memory	—	`phpunit.xml`
Cache / queue (optional)	Redis via Predis	`^3.6`	`composer.json`
AI provider SDK	`anthropic-ai/sdk`	`^0.45`	`composer.json`
Second AI provider	OpenAI, over plain HTTP	—	`app/Services/AI/OpenAiService.php`
Object storage	`league/flysystem-aws-s3-v3`	`^3.35`	`composer.json`
HTML to Markdown	`league/html-to-markdown`	`^5.1`	`composer.json`
PDF text extraction	`smalot/pdfparser`	`^2.12`	`composer.json`
Code highlighting	`spatie/commonmark-highlighter`	`^3.0`	`composer.json`
Activity log	`spatie/laravel-activitylog`	`^4.12`	`composer.json`
Web server	nginx + PHP-FPM	Alpine packages	`docker/nginx.conf`
Process supervisor	Supervisor	Alpine package	`docker/supervisord.conf`
Container base	`php:8.3-fpm-alpine`	—	`Dockerfile`
Hosting platform	Railway	—	`railway*.json`
Code style	Laravel Pint	`^1.24`	`composer.json`
Testing	PHPUnit	`^11.5.50`	`composer.json`

1.3 Scale of the implementation

MEASURE	COUNT

PHP classes under `app/`	355
Database migrations	42
Eloquent models	25
Livewire components	36
Authorization policies	8
Domain actions (`app/Actions`)	44
Enums	27
Blade views and components	107
Test files	139, across Unit, Feature and a dedicated Security suite
Environment variables in `.env.example`	183

SECTION 2

Executive Overview

2.1 What Nexora is

Nexora is a **shared delivery workspace for a software agency and the customers it builds for**. It is one application in which the delivery team plans and executes work and the customer watches that work happen — on the same tickets, at the same time, but seeing a deliberately different slice of it.

It combines four things that are normally four separate products:

1. **Kanban boards and tickets** — the work itself.
2. **A documentation workspace** — a nested page tree, per board, with a live editor.
3. **An AI assistant with real access to the workspace** — one that reads tickets, documentation, statistics and code activity through the asking person's own permissions, and can draft changes for a human to confirm.
4. **Integrations with the tools engineering already lives in** — GitHub, Slack and SMS.

2.2 The problem it solves

An agency running several customer projects usually ends up with the work in one tool, the documentation in a second, the customer conversation in a third, and the engineering trail in a fourth. Three failures follow from that split, and Nexora is built around all three.

Customers cannot see progress without a person exporting it. Status meetings exist because nobody outside the team can read the board. Nexora gives the customer a real login to the real board — and answers the obvious objection by making *internal* the default for everything, with publication a deliberate, audited, staff-only act.

Internal candour and customer communication collide. A team that shares its board stops writing frankly on it. Nexora gives every ticket **two conversation streams** — one the customer takes part in, one the customer cannot see, cannot count and cannot detect — so the team keeps its working notes without keeping a second system.

Knowing what is happening costs time. Somebody has to read forty tickets to answer "what is overdue?", or open GitHub to answer "did that ship?". Nexora's assistant answers those from the live workspace, using retrieval tools scoped to the asker, and will draft the ticket that should follow.

2.3 Who uses it

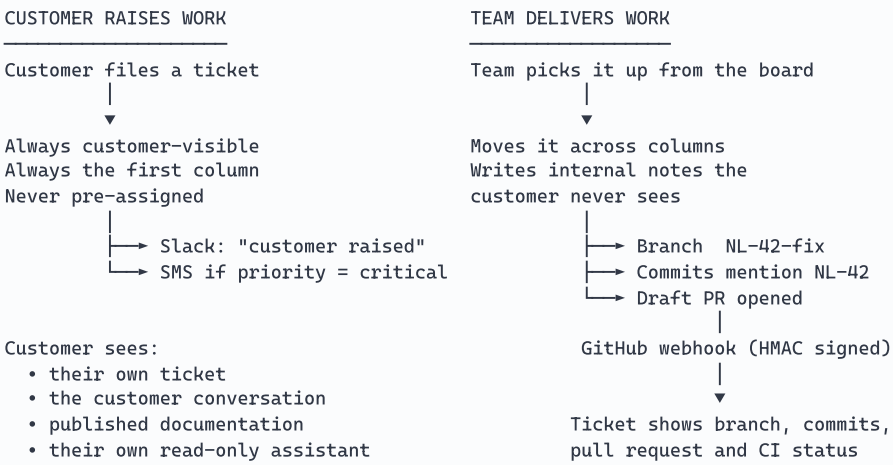
AUDIENCE	ROLE	WHAT THEY ARE THERE FOR
Agency administrators	`admin`	Creating boards and accounts, holding AI credentials, setting how far automation is trusted, seeing every board

Delivery team members	`team`	Daily work: tickets, documentation, the assistant, integrations, and board configuration for boards they belong to
Customer stakeholders	`customer`	Raising requests, following progress, reading published documentation, asking a read-only assistant about their own project

There is **no self-service sign-up by default**. `ALLOW_PUBLIC_REGISTRATION` is `false` and an administrator creates accounts. When enabled, a self-registered account receives the least-privileged role, `customer` — never staff.

2.4 Core workflows

FIGURE



2.5 Major capabilities, at a glance

CAPABILITY	STATUS
Kanban boards with drag-and-drop columns and cards	Implemented
Tickets: type, priority, assignee, estimate, due date, labels, checklist, links	Implemented
Two-stream ticket conversation (customer / internal)	Implemented
Nested documentation workspace, WYSIWYG editing, autosave drafts	Implemented
Workspace AI assistant — panel and full page — with retrieval tools	Implemented
AI proposals: create/update tickets and pages, comment, delete, start a code run	Implemented

AI ticket runs: suggest (analysis note) and apply (draft pull request)	Implemented
AI-generated charts, tables and KPI rows, with CSV and PNG export	Implemented
Spoken conversation (speech-to-text and text-to-speech)	Implemented
AI file attachments: text, Markdown, CSV, PDF, images, audio, Office documents	Implemented
GitHub inbound webhooks: branches, commits, pull requests, check suites, statuses	Implemented
GitHub outbound: push a branch, open a draft pull request	Implemented
Slack notifications, per board and per event	Implemented
SMS alerts for critical tickets	Implemented
Statistics: team and customer screens, flow metrics, CSV and PNG export	Implemented
Realtime board, conversation and notification updates	Implemented
Command palette global search	Implemented
Light / dark / system theme	Implemented
Workspace activity feed	Implemented
AI Standup Generator	**Not implemented — see section 32**
Live external knowledge lookup for the assistant	Partially implemented — interface only, no driver ships
Apply-mode code generation	Partially implemented — driver defaults to `unavailable`
Remote MCP server for Claude / ChatGPT (reads and changes, OAuth)	Implemented
General REST / JSON API for third parties	Not implemented

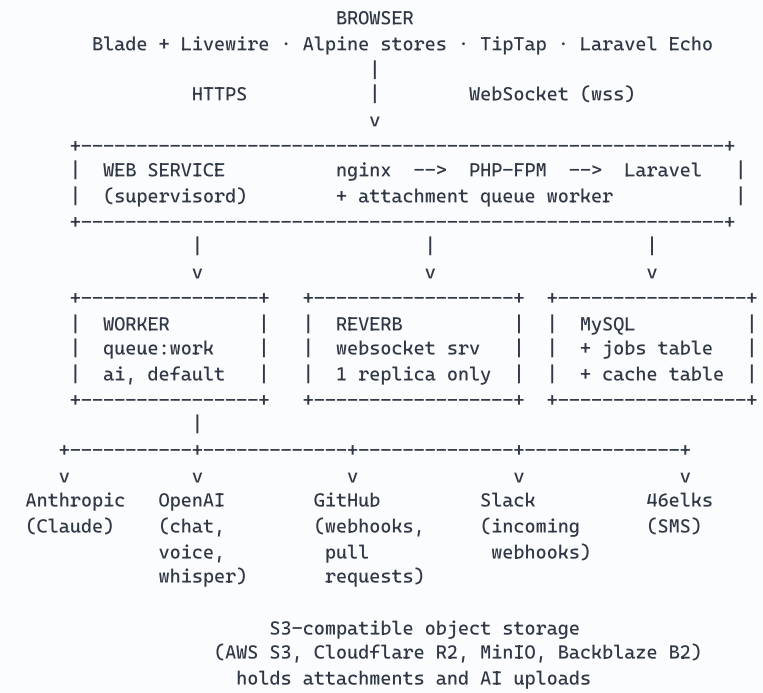
SECTION 3

System Architecture

3.1 The shape of the application

Nexora is a **single Laravel application** rendered server-side, made interactive by Livewire, and deployed as **one Docker image run three ways**. There is no separate front-end application, no API gateway and no microservices. The one machine-facing interface — the remote MCP server at `/mcp` that Claude and ChatGPT connect to — is served by the same application, from `routes/ai.php`.

FIGURE



3.2 Layers, and where the rules live

The codebase keeps a strict separation, and that separation is itself the security design.

LAYER	DIRECTORY	RESPONSIBILITY	MAY IT DECIDE ACCESS?
Routes	`routes/web.php`	URL to component, coarse role middleware	First line only
Middleware	`app/Http/Middleware`	Role gate, active-account gate	Coarse only
Livewire components	`app/Livewire`	Screen state and interaction	Calls policies; holds no rules
Policies	`app/Policies`	Per-record authorization	**Yes — single-record rules**
Services	`app/Services`	Readers, scopes, integrations	**Yes — SQL-level scoping**

Actions	`app/Actions`	One write each, with its own rules	**Yes — field-level rules**
Models	`app/Models`	Relationships and query scopes	**Yes — `visibleTo()` scopes**
Support	`app/Support`	Value objects, formatting, parsing	No

> **NOTE**
>
> **Authorization is enforced in SQL first and in policies second.** A collection is filtered by a query scope (`Ticket::visibleTo()` , `Comment::visibleTo()` , `AiRun::visibleTo()`), and a single loaded record is then checked by the matching policy method. Each policy is written to mirror its scope line for line. If the two ever disagree, the scope is the one that has actually been protecting the list.

3.3 Why each technology is here

TECHNOLOGY	WHY IT IS USED, PER THE IMPLEMENTATION
Laravel 12	Policies, queues, broadcasting, notifications and Eloquent scopes are the actual enforcement mechanisms, not conveniences layered on top
Livewire 4	Server-rendered interactivity without a second application. One component owns a whole board, so a drag-and-drop move re-renders source column, destination column and both counts in one round trip
Alpine (bundled in Livewire)	Client-only state that must not wait for the network: theme, command palette, AI panel, dialogs
Tailwind CSS 4	Utility styling with no component stylesheet to fight; the Vite plugin scans Blade and PHP for class names
TipTap / ProseMirror	A schema-constrained editor: the document can only ever contain nodes the schema declares, which is a security property as much as a correctness one
Reverb	First-party WebSocket server, so realtime needs no third-party service and channel authorization runs through Laravel's own gates
MySQL	Relational integrity is load-bearing — foreign keys, unique constraints and the `(board_id, number)` ticket key are what make the rules hold under concurrency
Database queue	No Redis required for a working deployment; `jobs` , `job_batches` and `failed_jobs` are ordinary tables
Redis (optional)	Needed only for multi-replica scheduler locks and Reverb scaling
S3-compatible storage	Container filesystems are replaced on every deploy; a production boot with an ephemeral attachment disk is refused outright

Docker + Supervisor	One image with three start commands, so the worker image cannot drift from the web image
nginx + PHP-FPM	Static assets and request-body buffering handled outside PHP
Railway	Declarative per-service definitions in `railway*.json`; TLS terminated at the edge

3.4 Request lifecycle

FIGURE

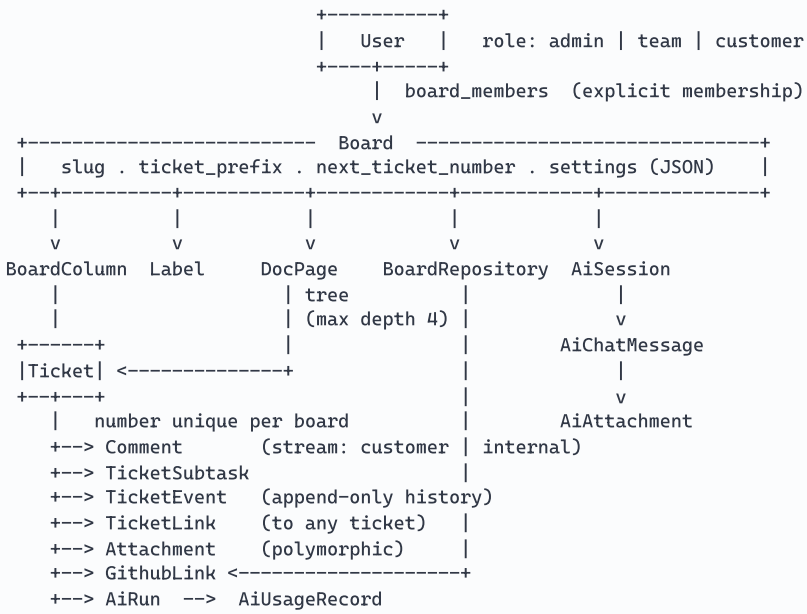
```
Request
   |
   +- trustProxies('*')          Railway terminates TLS: recover real scheme and client IP
   +- web middleware group       session, cookies, CSRF
   +- AuthenticateSession        session bound to the password hash it was created with
   +- EnsureUserIsActive         a deactivated account is logged out on its next request
   +- role:...  (where present)  coarse role gate for the route group
   +- password.confirm (where present)   re-authentication for administration screens
   |
   v
Livewire component mount()
   |
   +- Gate / policy check        denies as 404 for anything a person must not know exists
   v
render()  -->  Reader service  -->  Eloquent scope  -->  SQL with the viewer applied
   |
   v
Blade  -->  ContentRenderer (Markdown, sanitised, viewer-aware links)
```

IMPORTANT

Every Livewire interaction is a **new HTTP request**. Components re-authorize on `mount()`, on every action and on every render — they never trust what the previous render decided. This is why `Docs\Show` re-resolves its page through `DocPageFinder` on each render: Livewire rehydrates models by primary key, which bypasses query scopes.

3.5 Domain model map

FIGURE



SECTION 4

User Roles and Permissions

4.1 The three roles

Nexora has exactly three application-wide roles, defined in `App\Enums\UserRole`. The role is stored on the `users` row and answers *"what kind of person is this?"* — **not** *"what may they do on board X?"*, which is answered separately by board membership.

ROLE	VALUE	SEES EVERY BOARD?	MAY SEE INTERNAL CONTENT?	MAY ADMINISTER THE WORKSPACE?
Administrator	`admin`	**Yes**, without membership	Yes	Yes
Team member	`team`	Only boards they are a member of	Yes	No
Customer	`customer`	Only boards they are a member of	**Never**	No

The enum groups the first two as **staff** (`isStaff()`). The internal-content rule is written as a deny-list of one, so that any future role is internal-unsafe unless it is deliberately added to `isStaff()`.

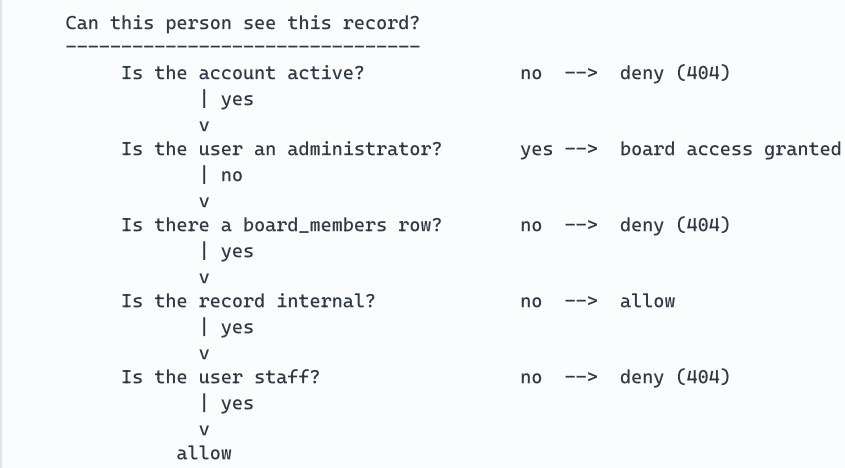
> **IMPORTANT**
>
> **Customer is the security boundary of the product.** Being a member of a board never grants a customer access to internal tickets, internal notes, internal documentation, AI runs, repository data or the activity feed — on any board, including boards they belong to. This is enforced independently of membership, in `BoardAccess::canSeeInternalContent()`.

4.2 How access is decided

Four rules, implemented once in `App\Services\BoardAccess` and delegated to by every policy, reader and channel-authorization callback:

1. An administrator may see every board.
2. Any other user may see a board only when an explicit `board_members` row links them.
3. A deactivated or unauthenticated user may see nothing at all.
4. Customers may never observe content flagged as internal, on any board.

FIGURE



NOTE

Denials are 404, not 403, wherever existence is itself a secret. A 403 on `/boards/acme-platform` would confirm that a board with that slug exists. Boards, tickets, documentation pages, AI runs, AI conversations, the board AI chat and the board AI settings screen all deny as *Not Found*.

4.3 Permission matrix — workspace and boards

Derived from the `EnsureUserHasRole` middleware on routes, `BoardPolicy`, `UserPolicy`, `BoardAccess`, and the gates registered in `AppServiceProvider`.

CAPABILITY	ADMINISTRATOR	TEAM MEMBER	CUSTOMER
Sign in	Yes	Yes	Yes
See the board index	Yes — all boards	Own memberships	Own memberships
Open a board	Yes — any	Member only	Member only
Create a board	**Yes** (+ password confirm)	No	No
Rename or edit a board	**Yes** (+ password confirm)	No	No
Delete a board	**Yes**	No	No
Add or remove board members	**Yes**	No	No
Create, rename, reorder, delete tabs	Yes	Member only	No
Create, rename, reorder, delete columns	Yes	Member only	No
Create, edit, delete labels	Yes	Member only	No
Configure board AI settings	Yes	Member only	No — screen 404s
Attach or remove GitHub repositories	Yes	Member only	No — screen 404s
Configure Slack and SMS for a board	Yes	Member only	No — screen 404s
Global AI settings (`/admin/ai`)	**Yes** (+ password confirm)	No — 403	No — 403
Create, edit, deactivate users	**Yes** (+ password confirm)	No — 403	No — 403
Change another user's role	**Yes**	No	No
Change **own** role	**No** — explicitly forbidden	No	No
Delete **own** account	**No** — explicitly forbidden	No	No
Edit own profile and password	Yes	Yes	Yes
Set own theme preference	Yes	Yes	Yes

4.4 Permission matrix — tickets

Derived from `TicketPolicy`, `App\Actions\Tickets\CreateTicket` and `UpdateTicket`.

CAPABILITY	ADMINISTRATOR	TEAM MEMBER	CUSTOMER
See a customer-visible ticket	Yes	Member only	Member only
See an internal ticket	Yes	Member only	**Never**
Create a ticket	Yes	Yes	**Yes** — with the constraints below
Choose the tab at creation	Yes	Yes	**Yes** — they see every tab's name
Choose the column at creation	Yes	Yes	**No** — always the first column of its tab
Set visibility at creation	Yes (defaults to customer-visible)	Yes (defaults to customer-visible)	**No** — always customer-visible
Assign at creation	Yes	Yes	**No** — always unassigned
Edit a ticket raised by somebody else	Yes	Yes	**No**
Edit a ticket they raised themselves	Yes	Yes	**Yes** — restricted fields
Change title, description, priority, type, due date	Yes	Yes	Own tickets only
Change assignee	Yes	Yes	**No** — silently dropped
Change estimate	Yes	Yes	**No** — silently dropped
Change customer visibility	Yes	Yes	**No** — silently dropped
Move between columns (drag and drop)	Yes	Yes	**No**
Move to another tab (drop on a tab, the Tab choice, AI, MCP)	Yes	Yes	**No**
Add or remove labels	Yes	Yes	**No**
Link tickets to one another	Yes	Yes	**No**
Manage the checklist	Yes	Yes	Own tickets only
Attach files	Yes	Yes	Own tickets only
Delete a ticket	Yes	Yes	**No**

NOTE

A customer's restricted fields are **silently dropped** by `UpdateTicket` rather than rejected. `TicketPolicy::update` permits a customer to edit a ticket they raised; the action is what narrows *which fields* they may touch. So a crafted request that submits `assignee_id` cannot set it, even though the update itself is authorized.

4.5 Permission matrix — conversations, documentation, files

CAPABILITY	ADMINISTRATOR	TEAM MEMBER	CUSTOMER
Read the customer conversation	Yes	Yes	Yes
Post to the customer conversation	Yes	Yes	Only if the board allows customer comments
Read internal notes	Yes	Yes	**Never** — not even the count
Post an internal note	Yes	Yes	**No**
Edit a comment	Own only	Own only	Own only
Delete a comment	Own, **and anybody's**	Own only	Own only
Read published documentation	Yes	Yes	Yes — only if every ancestor is also published
Read internal documentation	Yes	Member only	**Never**
Create a documentation page	Yes	Member only	**No**
Edit, move or delete a page	Yes	Member only	**No**
Publish a page to customers	Yes	Member only	**No**
Attach a file to a page	Yes	Member only	**No**
Download an attachment	If the owning record is readable	Same	Same
Delete an attachment	The uploader, or anyone who may edit the owner	Same	Same

4.6 Permission matrix — AI

Derived from `BoardPolicy::useAiChat` and `::useAssistant`, `AiRunPolicy`, `AiCapabilityGuard`, and each tool's `availableTo()`.

CAPABILITY	ADMINISTRATOR	TEAM MEMBER	CUSTOMER
Open the assistant panel	Yes	Yes	**Yes** — read-only
Open the full-page board AI chat	Yes — member boards	Member only	**No** — 404
Ask about a board	Yes	Member only	Member only
Ask about the whole workspace	Yes	Yes	Yes — own boards only
Attach files to a conversation	Yes	Yes	**Yes** — their own conversation
Read somebody else's conversation	**No** — ownership is absolute	No	No
Use the voice interface	Yes	Yes	**Yes**
Be offered *write* tools by the model	Yes, mode permitting	Yes, mode permitting	**Never, in any mode**
Confirm an AI proposal	Yes, mode permitting	Yes, mode permitting	**No**
See AI runs on a ticket	Yes	Member only	**Never** — 404
Start a suggest run	Yes	Member only	**No**
Start an apply run (opens a pull request)	Yes	Member only	**No**
Cancel a queued run	Yes	Member only	**No**
Set the workspace capability mode	**Yes**	No	No
Set a board's capability mode	Yes	Member only	No

AI retrieval tools by role

TOOL	ADMINISTRATOR	TEAM MEMBER	CUSTOMER
`get_board`	Yes	Yes	Yes
`get_ticket`	Yes	Yes	Yes — customer-visible tickets only
`search_tickets`	Yes	Yes	Yes — customer-visible tickets only
`search_documentation`	Yes	Yes	Yes — published pages only

`get_documentation_page`	Yes	Yes	Yes — published pages only
`get_statistics`	Yes	Yes	Yes — five of the eight metrics
`get_activity`	Yes	Yes	**No** — the tool is not offered
`get_code_activity`	Yes	Yes	**No** — the tool is not offered
`get_github_repository`	Yes	Yes	**No** — the tool is not offered
`search_external_knowledge`	Only in *Outside Project* scope, and only when a driver is configured	Same	Same
`propose_*` (seven write tools)	Mode permitting	Mode permitting	**Never**

Statistics metrics withheld from customers: `by_assignee`, `by_label` and `visibility_split`. The five they may ask for are `totals`, `by_status`, `by_priority`, `created_by_week` and `completed_by_week`.

> **IMPORTANT**
>
> A tool a person may not use is **not sent to the model at all**, rather than sent and then refused. A model with no write tool cannot produce a proposal, so there is no preview, no confirm button and nothing for a crafted request to execute. An unrecognised tool name and a tool that exists but is withheld produce the *same* refusal text, so a customer cannot tell the two apart.

4.7 Permission matrix — reporting and administration screens

SCREEN	ROUTE	GATE	ADMINISTRATOR	TEAM	CUSTOMER
Dashboard	`/dashboard`	auth	Yes	Yes	Yes
Board index	`/boards`	auth	Yes	Yes	Yes
Team statistics	`/stats`	`role:admin,team`	Yes	Yes	**403**
Team statistics CSV	`/stats/export`	`role:admin,team`	Yes	Yes	**403**
Customer statistics	`/stats/customer`	auth	Yes	Yes	**Yes**
Customer statistics CSV	`/stats/customer/export`	auth	Yes	Yes	**Yes**
Workspace activity	`/activity`	`role:admin,team`	Yes	Yes	**403**
Settings directory	`/settings`	auth	Yes	Yes	Yes
Profile	`/settings/profile`	auth	Yes	Yes	Yes

Global AI settings	`/admin/ai`	`role:admin` + confirm	Yes	**403**	**403**
User administration	`/admin/users`	`role:admin` + confirm	Yes	**403**	**403**
AI block CSV export	`/ai/messages/{id}/blocks/{n}/export.csv`	`role:admin,team`	Yes	Yes	**403**
Voice endpoints	`/ai/voice/*`	auth + throttle	Yes	Yes	**Yes**

> **NOTE**
> The customer statistics screen is open to **everyone, staff included**, and that is deliberate: it is the only way for a team member to see exactly what a customer sees. It is a separate Livewire component backed by a separate service (`CustomerStatistics`) which has no method that could return an internal figure — so there is no branch on the page that a careless edit could invert.

4.8 Coarse gates

Three named gates are registered for navigation and route groups. Record-level decisions always go through a policy and never through these.

GATE	RESOLVES TO	USED FOR
`administer-workspace`	`canAdministerWorkspace()` — admin only	Sidebar links, admin route groups
`view-internal-content`	`canSeeInternalContent()` — staff only	Hiding internal UI affordances
`administer-ai`	`canAdministerWorkspace()` — admin only	The global AI settings screen

`administer-ai` is defined separately from `administer-workspace` even though the two currently answer identically, because they are different questions — *"may this person manage the workspace?"* and *"may this person hold the provider credentials and decide how far the AI is trusted?"* — and the second is the one a deployment is most likely to want to narrow later.

SECTION 5

Authentication and Security

5.1 Authentication

Nexora uses Laravel's standard session-based authentication, with the screens rebuilt as Livewire components rather than taken from a starter kit.

FEATURE	ROUTE	COMPONENT	STATUS
Login	`GET /login`	`Livewire\Auth\Login`	Implemented
Logout	`POST /logout`	`Auth\LogoutController`	Implemented
Registration	`GET /register`	`Livewire\Auth\Register`	**Implemented but disabled by default**
Forgotten password	`GET /forgot-password`	`Livewire\Auth\ForgotPassword`	Implemented
Password reset	`GET /reset-password/{token}`	`Livewire\Auth\ResetPassword`	Implemented
Password confirmation	`GET /confirm-password`	`Livewire\Auth\ConfirmPassword`	Implemented
Email verification	—	—	**Not implemented** (column exists, no flow)
Two-factor authentication	—	—	**Not implemented**
SSO / SAML (signing in to Nexora with another identity provider)	—	—	**Not implemented**
OAuth authorization server, for AI clients	`GET /oauth/authorize`	Laravel Passport + `resources/views/mcp/authorize.blade.php`	**Implemented** — used only by the remote MCP server; see below

Registration is off by default

The `/register` route is always registered so that links and tests resolve, but the component **returns 404 unless** `ALLOW_PUBLIC_REGISTRATION=true`. This is an internal tool shared with named customers, so administrators create accounts. When registration is enabled, a self-registered account receives `UserRole::Customer` — the least-privileged role — so a self-registered account can never reach internal content.

Password policy

Defined once in `AppServiceProvider::configurePasswords()`:

ENVIRONMENT	RULE

All environments	At least **12 characters**, containing letters and numbers
Production only	Additionally **mixed case** and **checked against known breach corpora** (`uncompromised()`)

Passwords are hashed with bcrypt at `BCRYPT_ROUNDS=12` (the test suite drops to 4 for speed). The `password` attribute is cast `hashed`, so a plaintext value assigned to it is hashed on write and can never be stored in the clear.

What the login flow does

1. Validates the submitted credentials.
2. Checks the throttle (see *Rate limiting* below) and refuses early if it is tripped.
3. Attempts authentication with a lower-cased email.
4. **Refuses a deactivated account even with correct credentials** — the session is established, then immediately invalidated, so a disabled account cannot get in.
5. Clears the throttle counter.
6. **Rotates the session id**, to defeat session fixation.
7. Redirects to the intended URL, or to the dashboard.

Session handling

CONTROL	VALUE	EFFECT
Driver	`database`	Sessions survive a container restart and are shared across replicas
Lifetime	120 minutes	Idle timeout
`http_only`	`true`	The cookie is unreadable from JavaScript
`same_site`	`lax`	Cross-site POSTs do not carry the session
`secure`	`SESSION_SECURE_COOKIE`	**Must be set to `true` in production**
`encrypt`	`false` by default	Session payloads are stored unencrypted in the database
`AuthenticateSession`	Enabled globally	The session is bound to the password hash it was created with, so **changing a password invalidates every other session**
`EnsureUserIsActive`	Enabled globally	A deactivated account is logged out on its very next request, not at next login

WARNING

`SESSION_SECURE_COOKIE=false` is the shipped default in `.env.example`, which is correct for local HTTP development and **wrong for production**. Set it to `true` for any deployment served over HTTPS.

Re-authentication for administration

Three route groups additionally require `password.confirm`, Laravel's re-authentication middleware. Reaching them asks for the password again even inside a valid session:

- `/admin/ai` — global AI settings, which hold provider credentials
- `/admin/users` — user administration, which is the privilege-escalation path
- `/boards/create` and `/boards/{board}/edit` — board creation and identity

OAuth for the remote MCP server

Nexora is an OAuth 2.1 **authorization server** for one purpose: letting a person connect an AI client (Claude, ChatGPT) to the MCP endpoint at `/mcp`. It is not a way to sign in to Nexora — that is still the login form above — and a token grants nothing beyond the MCP tools, which read and change things with the token's user as the actor and under that person's own permissions.

PROPERTY	VALUE
Implementation	Laravel Passport, discovery and registration routes from laravel/mcp
Client registration	Dynamic (RFC 7591), public clients; redirect URIs only on `MCP_REDIRECT_DOMAINS` (Claude, ChatGPT, loopback)
Grant	Authorization code with PKCE `S256` only; the device-code grant is switched off
Scope	`mcp:use`, required by `/mcp` and granted by default
Access token	60 minutes (`MCP_ACCESS_TOKEN_MINUTES`)
Refresh token	30 days (`MCP_REFRESH_TOKEN_DAYS`), rotated on every use
Consent	`resources/views/mcp/authorize.blade.php`: names the client and the host it returns to, says what it will be able to change (staff and customer wording differ), warns on loopback redirects
Re-consent	Connections approved while the server was read-only were revoked once, by migration `2026_10_21_000100_revoke_read_only_mcp_connections`, so nobody's app gained changes on a screen that promised it could not make any
Revocation	Access, refresh and auth codes revoked when the user is deactivated or their password changes (`App\Observers\UserObserver`); a deactivated user's request is a `401`
Keys	`PASSPORT_PRIVATE_KEY` / `PASSPORT_PUBLIC_KEY`; rotating them disconnects every client

5.2 Authorization

Authorization is applied in four independent places, and they are designed to overlap.

FIGURE

```
1. ROUTE MIDDLEWARE      role:admin,team          coarse; blocks before a component exists
                         password.confirm         re-authentication for administration

2. QUERY SCOPES          Ticket::visibleTo()      the PRIMARY defence: filters in SQL
                         Comment::visibleTo()     protects lists, search, counts, links
                         AiRun::visibleTo()
                         DocPageFinder            adds the ancestor rule for the page tree

3. POLICIES              TicketPolicy::view()     single loaded record; mirrors the scope
                         BoardPolicy::useAiChat()
                         DocPagePolicy::update()

4. ACTIONS               CreateTicket             field-level rules that hold whatever
                         UpdateTicket             screen, console command or AI proposal
                         PostComment              calls them
                         SetPageVisibility
```

Eight policies are registered: `BoardPolicy`, `UserPolicy`, `TicketPolicy`, `CommentPolicy`, `DocPagePolicy`, `AttachmentPolicy`, `AiRunPolicy` and `AiSessionPolicy`.

Attachments inherit visibility rather than storing it

`AttachmentPolicy` holds no rules of its own. It resolves the owning record from an explicit allow-list (`Ticket`, `Comment`, `DocPage`, `AiSession`) and asks *that* record's policy. A file on an internal ticket is internal because the ticket is; a file on an unpublished page is internal because the page is. There is no second flag to keep in sync, because there is no flag at all. An owner type nobody has registered is **denied**, not served.

5.3 CSRF protection

CSRF validation is on for the whole `web` group, with exactly one exemption:

```
$middleware->validateCsrfTokens(except: [
    'webhooks/github',
]);
```

The exemption is necessary — GitHub never rendered one of our forms and has no session — and it is replaced by something stronger. An HMAC signature proves the request came from somebody holding the shared secret, whereas CSRF only proves a request came from our own page.

5.4 Webhook signature verification

`App\Services\GitHub\WebhookSignature` is the entire authentication story for the inbound endpoint. Four properties, each of which has been a real vulnerability in other systems:

PROPERTY	IMPLEMENTATION
Fail closed with no secret	With `GITHUB_WEBHOOK_SECRET` unset, *every* delivery is rejected. There is no "skip verification when unconfigured" path
Verify the raw body	`$request->getContent()`, never the parsed or re-encoded input — key order, unicode escaping and float formatting all differ after a round trip
Constant-time comparison	`hash_equals()`, never `===`, so the number of correct leading bytes cannot be walked out through timing
SHA-256 only	The legacy SHA-1 `X-Hub-Signature` header is ignored. Accepting it would make the endpoint's real strength SHA-1's, because the attacker picks the header

Failures are deliberately indistinguishable: a missing secret, a missing header and a wrong digest all return the same `401 Invalid signature.`

5.5 Rate limiting

Four named limiters, protecting different things.

LIMITER	APPLIES TO	KEYED ON	DEFAULT	WHY
`github-webhooks`	`POST /webhooks/github`	Client IP	300/min (`GITHUB_WEBHOOK_RATE_LIMIT`)	The only unauthenticated route that can enqueue work. Rejected requests never reach signature verification and never touch the database

`ai-voice`	`POST /ai/voice/listen`, `GET /ai/messages/{id}/speech`	Signed-in user id	30/min (`AI_VOICE_RATE_LIMIT`)	Transcription and speech synthesis are billed per call and are the only AI surface a browser can trigger in a loop without a person typing
`mcp`	`POST /mcp`	Token's user id	60/min (`MCP_RATE_LIMIT`)	One connected AI client looping on a tool slows down that connection only
`mcp-write:{user}`	MCP write tools, inside `WriteTool`	Token's user id	30/min (`MCP_WRITE_RATE_LIMIT`)	A runaway client can read, but stops changing things well before it could do damage
`mcp-oauth`	OAuth discovery and `POST /oauth/register`	Client IP	120/min (`MCP_OAUTH_RATE_LIMIT`)	Unauthenticated by design; generous because every Claude or ChatGPT user arrives from the same vendor addresses

The login form carries a third limiter of its own, inside `Livewire\Auth\Login`:

CONTROL	VALUE
Maximum failed attempts	5
Throttle key	`strtolower(email) + "
On success	The counter is cleared
On lockout	A validation error naming the remaining seconds

The composite key is deliberate: keying on the email alone would let one attacker lock every account out from a single address, and keying on the IP alone would not stop one address brute-forcing one account.

> **NOTE**
> There is no rate limit on the assistant's **text** endpoint. It is authenticated, costed against a per-session and per-user daily token budget, and bounded by the capability mode — but a distinct request limiter would still be worth adding. See section 32.

5.6 Input validation and output sanitisation

Every piece of user-written prose passes through one pipeline, `App\Services\ContentRenderer`:

1. Markdown to HTML with **raw HTML stripped** and unsafe link schemes rejected (`App\Support\Markdown`).

2. `@mentions` of board members highlighted.
3. Ticket references such as `AQD-142` linked — **but only for tickets the particular reader may open**.

Steps 2 and 3 are viewer-dependent, so nothing in this pipeline may be cached across viewers.

The rich-text editor adds a second, stricter layer. `App\Support\RichText\EditorHtml` is an **allow-list** applied to HTML travelling in *both* directions:

- *browser to server* — the posted document is attacker-controlled, so it is cleaned before the Markdown converter sees it. Nothing hostile is stored, not even as inert text that would still surface in search results, board-card excerpts and AI prompts.
- *server to browser* — the stored Markdown is rendered and cleaned again, which is about shape rather than safety: it drops syntax-highlighting spans the editor must not serialise back, and rewrites task lists into the form TipTap recognises.

An allow-list is used rather than a deny-list because a deny-list has to anticipate `onpointerrawupdate`, `xlink:href` and whatever parser-confusion trick lands in browsers next year, whereas an allow-list only has to know the dozen elements this product's Markdown can express.

Model output is never rendered directly

AI-generated charts, tables and KPI rows are validated into typed value objects (`ChartSpec`, `TableSpec`, `KpiSpec`) before anything is drawn. The model supplies numbers and labels; the server draws the SVG. Consequently:

- there is no charting library receiving model-authored configuration, so there is no option in such a library that turns a string into a callback;
- nothing model-authored reaches a `<script>`, an event handler, a style attribute or a URL;
- colours come from a fixed palette chosen by index, never supplied;
- labels are escaped as text nodes by Blade.

A chart with no valid numbers, or a table with no columns, falls back to rendering the model's text as prose. An answer never becomes an error message because its illustration was malformed.

5.7 Secret handling

> **IMPORTANT**
> No credential in this system is ever rendered back to a browser, written to a log line, or included in an exception message. This is enforced by construction, and there is a dedicated test — `tests/Security/SecretExposureTest.php` — that asserts it.

SECRET	STORED	READ BY	WRITE-ONLY IN THE UI?
`APP_KEY`	Environment	Framework	n/a
Anthropic API key	Environment, **or** encrypted in `ai_credentials`, **or** encrypted per board	`AiCredentialVault` only	Yes — fields blank on load, blanked after save
OpenAI API key	Same three places	`AiCredentialVault` only	Yes
GitHub token	Environment (`GITHUB_TOKEN`)	`config('github.token')` only	n/a — environment only
GitHub webhook secret	Environment	`WebhookSignature` only	n/a
Slack webhook URL	**Encrypted** in `boards.settings.slack`	`BoardSlackSettings`	Yes — replaced, never read back
46elks credentials	Environment	`ElksSmsProvider` only	n/a

Three design decisions are worth stating explicitly.

The Slack webhook URL is encrypted at rest. A Slack incoming-webhook URL is a bearer token wearing a URL's clothes — anyone holding it can post into that channel as the app, for ever, with no further authentication, and Slack has no read-only variant. Storing it as plain text in a JSON column would put it in every database dump and backup.

Credential screens are write-only. The global AI settings screen and the board integrations screen render *presence* and *the last four characters*, never the value. The vault has no method that would return a key to a component. Somebody who gains access to those screens can **replace** a credential — which is visible in `rotated_at` and `updated_by_id` — but cannot read one out.

Decryption failure degrades to "not configured". Rotating `APP_KEY` makes stored credentials unreadable. That is the correct failure — the alternative is a key rotation that leaves credentials readable — and it is handled by reading as *absent* rather than by throwing, so a board with an unreadable webhook stops posting instead of breaking every ticket save.

5.8 The GitHub token is deliberately weak

The fine-grained token needs exactly two permissions on the repositories in scope:

PERMISSION	WHY

`contents: write`	Push a branch
`pull_requests: write`	Open a pull request

It is **never given merge rights.** `PullRequestClient` has no merge method, no branch-delete method and no review-approval method — so apply mode could not merge even if the token allowed it. A future phase that genuinely needs to merge has to add the capability and be reviewed for it.

5.9 File upload security

CONTROL	TICKET / COMMENT / PAGE ATTACHMENTS	AI CONVERSATION ATTACHMENTS
Size ceiling	10 MB (`ATTACHMENT_MAX_SIZE_KB=10240`)	20 MB (`AI_ATTACHMENT_MAX_SIZE_KB=20480`)
Count ceiling	20 per owning record	10 per conversation
Type checking	Extension allow-list	**Extension allow-list *and* detected MIME type** — both must pass
Stored path	Generated, never derived from the uploaded filename	Same
Original filename	Kept only as a display label	Same
`svg` allowed?	Yes — only ever served as a download	**No** — an "image" here is decoded and sent to a vision model
`zip` allowed?	Yes	**No** — a container whose contents nothing has validated
Images before sending to a model	n/a	**Re-encoded with GD** — strips EXIF (including GPS), strips anything appended after the image data, and makes the declared media type true by construction

Downloads never link straight at stored bytes. `AttachmentController` authorizes the request against the owning record first, then either redirects to a **short-lived signed URL** (S3 disks, default 5 minutes via `ATTACHMENT_URL_TTL_MINUTES`) or streams the file (local disk, development only).

5.10 Storage durability is enforced at boot

`App\Providers\AppServiceProvider` refuses to boot a **production** application whose attachment disk is not in `config('attachments.durable_disks')` — currently `s3` and `volume`. The two are checked differently:

- `s3` is durable by construction; there is nothing local to verify.
- `volume` is durable only while a persistent volume is actually mounted at its root, so the check **stats the root and refuses to boot when the mount is absent.**

Without this, a forgotten volume would write into the container and lose every file at the next deploy, silently.

5.11 HTTP security headers

Set by nginx on every application response:

HEADER	VALUE
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`SAMEORIGIN`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`server_tokens`	`off` — nginx does not advertise its version

Dotfiles other than `/.well-known` are denied. `client_max_body_size` is 32 MB.

> **WARNING**
>
> **There is no Content-Security-Policy header and no Strict-Transport-Security header.** Both are worth adding. HSTS in particular is cheap on a platform that already terminates TLS. See section 32.

5.12 Transport security

`URL::forceScheme('https')` is applied in production, or whenever `APP_URL` starts with `https://`. Without it, generated URLs — password-reset links in particular — would be `http://`, because Railway terminates TLS at the edge and proxies over HTTP. `trustProxies(at: '*')` is what lets Laravel see the real scheme and the real client IP for rate limiting.

5.13 Realtime authorization

There is **no public channel and no presence channel**. Every channel is private, and joining one is a request Laravel authorizes per user before the WebSocket server will deliver anything.

CHANNEL	WHO MAY JOIN
`board.{id}.internal`	Staff members of that board — `canView()` **and** `canSeeInternalContent()`
`board.{id}.customer`	Anyone who may view the board, customers included
`users.{id}`	That user, and only while active

The split is the point: because `BoardUpdated` only signals the internal channel when an *internal* ticket changes, a customer's browser is never even woken by activity they are not allowed to know about. That closes the timing side-channel a single shared channel would leave open even with an empty payload.

SECTION 6

Dashboard and Workspace Shell

6.1 There is no "workspace" entity

This is worth stating first, because it shapes everything else: **there is no workspace table and no tenant model**. One deployment *is* one workspace. The unit of access is the **board**, and almost everything that looks like a workspace-level setting is in fact a board setting. The two genuine exceptions are the global AI settings (`ai_settings` , a single row) and user administration.

6.2 Navigation

The application shell (`resources/views/layouts/app.blade.php`) provides:

ELEMENT	CONTENTS
Sidebar	Dashboard, Boards, Activity (staff), Statistics, Settings, plus the theme switcher at its foot
Top bar	Page title, breadcrumbs, command-palette trigger, AI assistant trigger, notification bell, user menu
AI panel	A persistent drawer, mounted once for the whole application
Command palette	Mounted once, opened with `Cmd/Ctrl + K`

Links a viewer cannot use are hidden rather than shown-and-refused, but that is a courtesy only — every destination re-authorizes on arrival.

Navigation uses Livewire's `wire:navigate` , so pages are swapped without a full reload. This is why the AI panel, the command palette, the theme store and the dialog system are all implemented as **Alpine stores** rather than per-page component state: they must survive the page underneath them being replaced.

6.3 Dashboard

`/dashboard` — `App\Livewire\Dashboard\Index`

The landing page is role-aware in its **copy and actions**, never in its data. There is no branching on role for the board list: `BoardAccess::query()` already returns every board for an administrator and only explicit memberships for everybody else.

6.4 Command palette and global search

`Cmd/Ctrl + K` — `App\Livewire\Search\Palette` , backed by `App\Services\Search\GlobalSearch` .

Four categories, at most five hits each:

CATEGORY	SOURCE	VISIBILITY APPLIED BY
Tickets	`tickets`	`Ticket::visibleTo()` — board scope plus the customer flag
Documentation	`doc_pages`	`DocPageFinder` — flag **and** the ancestor rule
Boards	`boards`	`BoardAccess::query()`
People	`users`	Staff only, narrowed to people the viewer shares a board with

Implementation notes that matter:

- Ticket search runs **two queries rather than one**. `"NL-123"` is an identifier and deserves an index — the prefix resolves to a board and `(board_id, number)` is unique, so that lookup is a key seek however large the table gets. Everything else is prose and has to be scanned. The exact match is placed first and then excluded from the scan, so pasting a key puts that ticket at the top.
- Minimum term length is **2 characters**; the term is bounded to 120 characters before it reaches a `LIKE`.
- The term is debounced in the template, not in PHP — a palette that queried on every keystroke would be four table scans per character.
- Open/closed state is an **Alpine store**, not Livewire state, because pressing the shortcut has to put a focused input on screen immediately.

> **NOTE**
>
> **Search is plain SQL, on purpose.** There is no Scout, Meilisearch, Algolia or Elasticsearch. An external index has no idea that a ticket is internal, so the filtering would have to be reproduced in the engine's query language, correctly, for ever — and it would hold a second copy of every ticket description outside this application's visibility rules. The documented next step, when `LIKE '%term%'` stops scaling, is a **MySQL FULLTEXT index** on `tickets.title` and `description_md`, which stays inside MySQL and needs no new service. Scout is the step after that.

6.5 Notifications

`App\Livewire\Notifications\Bell`, backed by `App\Services\NotificationReader`.

The bell holds **no notification data of its own**. Everything is read on each render and re-checked against the visibility scopes, so what is shown is what the viewer may see *now*, not what they were allowed to see when the notification was written.

With realtime enabled the bell re-reads when the server says something arrived; without it, it polls slowly. Either way the work is bounded — at most `NotificationReader::WINDOW` rows and three lookups.

6.6 Activity feed

`/activity` — `App\Livewire\Activity\Index`. **Staff only** (`role:admin,team`).

A workspace-wide history — ticket movement, assignments, comments, documentation changes and board configuration — newest first, backed by `spatie/laravel-activitylog` in the `activity_log` table.

There is deliberately **no per-board activity route**. A board is a filter on this screen (`?board=slug`), which keeps one query, one authorization path and one set of filters rather than two of each.

> **IMPORTANT**
> Customers do not get this screen at all, and that is a design decision rather than an oversight. Every description in the feed is written for the delivery team and names internal tickets, internal notes and board configuration; there is no per-row rewriting that would make it customer-safe. It is enforced three times — route middleware, a component check on mount and on render, and `Activity::readableBy()` refusing a non-staff viewer in SQL regardless.

6.7 Settings

`/settings` — `App\Livewire\Settings\Index`.

A **directory, not a store**. Settings in this product are spread across seven screens because each grew next to the thing it configures; what was missing was one page that knows they all exist. This component owns no settings of its own and writes nothing.

GROUP	DESTINATION	WHO SEES THE LINK
Personal	`/settings/profile`	Everyone
Board	`/boards/{board}/settings` — columns and labels	Staff members of the board
Board	`/boards/{board}/ai/settings`	Staff members of the board
Board	`/boards/{board}/integrations` — Slack and SMS	Staff members of the board
Board	`/boards/{board}/edit` — name, prefix, members	Administrators
Workspace	`/admin/ai`	Administrators
Workspace	`/admin/users`	Administrators

The board selector exists because most of what looks like a workspace setting is in fact a board setting. The selection is resolved through `BoardAccess` on every render, so a slug in the query string can only ever name a board the viewer already has.

6.8 Profile

`/settings/profile` — `UpdateProfile` and `UpdatePassword`.

Name and email, and a separate password form. Changing a password invalidates every other session for that account, because `AuthenticateSession` binds a session to the password hash it was created with.

6.9 Responsive behaviour

The interface is built with Tailwind's responsive utilities throughout and works at phone width. The AI panel is a full-height drawer on desktop and an overlay on small screens. There is **no native mobile application** and no offline mode.

SECTION 7

Board Management

7.1 Purpose

A board is the unit of work, the unit of access and the unit of configuration. Everything in Nexora hangs off one: tickets, documentation, labels, tabs and columns, repositories, Slack and SMS settings, AI settings, and statistics scope. **Access to Nexora is access to a set of boards.**

7.2 Who can do what

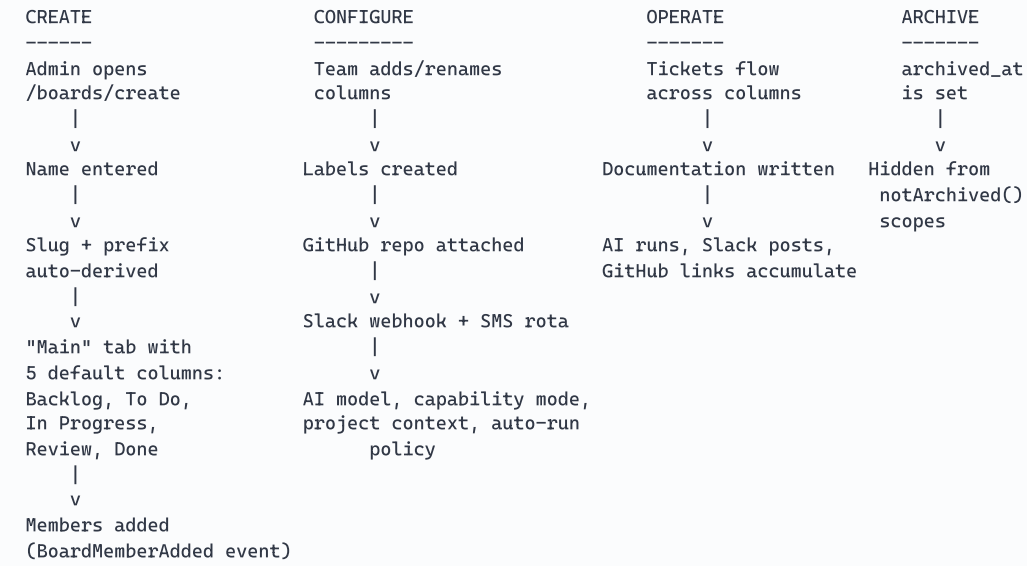
ACTION	WHO
See a board	Administrators (all), or anyone with a `board_members` row
Create, rename, delete a board	Administrators only, with password confirmation
Add or remove members	Administrators only
Manage tabs, columns and labels	Any staff member of the board
Manage AI settings, repositories, integrations	Any staff member of the board

NOTE

The split is deliberate. The board *record* — its name, prefix, membership and existence — stays administrator-only. The board's *working surface* — columns, labels, AI settings, repositories, integrations — is the delivery team's daily work on their own board. The expensive switch inside the AI screen carries a cost cap nobody can bypass, which is what makes it safe to hand to the team.

7.3 Board lifecycle

FIGURE



Creation

`/boards/create` — `App\Livewire\Boards\ManageBoard`, behind `role:admin` and `password.confirm`.

FIELD	RULE
Name	Required
Slug	Derived by `BoardIdentifiers`, globally unique, used in every URL
Ticket prefix	Derived by `BoardIdentifiers`, globally unique, 2–6 characters, `/^[A-Z][A-Z0-9]*$/`
Description	Optional
Settings	A JSON column, empty at creation; every option falls back to `config/workspace.php`

On creation, `CreateDefaultColumns` gives the board one tab, **Main**, with the standard workflow — **Backlog, To Do, In Progress, Review, Done** — with `Done` marked `is_done`. The action is idempotent per tab, so it can also repair a tab that somehow ended up with no columns. Neither the tab nor the columns are logged or broadcast separately: they are part of creating the board.

Primary keys are never exposed: `Board::getRouteKeyName()` returns `slug`.

Board settings JSON

`boards.settings` is a JSON column read through `Board::setting()`, which falls back to `config('workspace.board_defaults')`. New board options can therefore be introduced without a

migration or a backfill.

KEY	DEFAULT	MEANING
`new_tickets_visible_to_customers`	`true` (`WORKSPACE_NEW_TICKETS_VISIBLE`)	A ticket staff create starts visible to the board's customers: the starting value of the box on the New ticket form and in quick add, and the most visible an assistant-created ticket can be
`customers_can_comment`	`true`	Whether customers who are members may post to the customer conversation
`timezone`	`UTC`	The board's own day, used for daily AI run caps and statistics
`ai.*`	see section 10	Model, provider, capability mode, project context, auto-run policy, per-board credentials
`slack.*`	see section 14	Encrypted webhook URL and per-event switches
`sms.*`	see section 15	On-call rota for this board

7.4 Tabs

A board is split into **tabs** — "Admin dashboard", "Public website", "Backend API" — like browser tabs: one board, several boards' worth of work. Each tab has **its own columns**; the board shows one tab at a time, and the tab strip above the columns switches between them without leaving the board. Every board has at least one tab (existing boards were given **Main**, holding all their columns).

A ticket has no tab of its own: **its tab is the tab of its column**. So a ticket can never sit in a column of another tab, and moving it to another tab is moving it to a column of that tab — always an explicit act (see *Moving between tabs* below).

PROPERTY	DETAIL
Name	Up to 60 characters, trimmed, **unique per board ignoring case** (checked in the actions, the database unique is the backstop)
Slug	Made from the name when the tab is created, unique per board (`backend-api`, `backend-api-2`) and **never changed on rename**, so a link to a tab keeps working. The board URL carries it: `/boards/{board}?tab=backend-api`
Position	Leftmost first. The leftmost tab is the board's **default tab**: what the board opens on, and where a ticket filed without a tab lands
Columns	Each tab's own — names unique per tab, so two tabs can both have a "To Do"

`board_tabs` (`App\Models\BoardTab`); `board_columns.board_tab_id` is NOT NULL, and the composite foreign key `(board_tab_id, board_id) → board_tabs(id, board_id)` makes the database itself refuse

a column whose tab is on another board. A tab that still has tickets cannot be deleted underneath them: the cascade to its columns meets `tickets.board_column_id`'s RESTRICT.

The tab strip

BEHAVIOUR	DETAIL
Switching	Click a tab, or arrow / Home / End to a tab and Enter (manual activation: each switch is a round trip). `role="tablist"` / `tab` / `tabpanel` with `aria-selected`
URL	`?tab=<slug>` (`#[Url(history: true)]`), so a tab is a link, refresh keeps it, and back/forward step between tabs. The default tab keeps the plain URL. An open page keeps showing the tab it was drawn with even if somebody else reorders the strip meanwhile; once that tab is no longer the leftmost, its slug appears in the URL. A slug that is not one of this board's tabs falls back to the default tab with a notice
Remembered	A board opened without `?tab` shows the tab **this person last had open on it** (kept in their session), else the default tab
Counts	Each tab shows how many tickets match the current filters — by the same visibility scope as the cards, so a customer's counts never include internal work
Filters	Search, assignee, priority, labels and visibility belong to the board and stay on when switching tabs
Shown	When the board has more than one tab, or to anyone who may manage tabs (so "+" is always there for them)
Realtime	Tab and column changes broadcast `BoardUpdated` to the internal **and** customer channels (tab and column names are visible to customers). If the tab you have open is removed, the board falls back to the default tab and says so

Managing tabs

From the tab strip on the board — the same people who may manage columns (`BoardPolicy::manageTabs` → `manageColumns`: administrators, and staff members of the board). Every change is re-authorized and resolved within the board (404 otherwise), and recorded in the workspace feed (*Tab added / renamed / reordered / removed*, internal-only).

ACTION	DETAIL
Add ("+")	Asks for a name. The new tab starts with a **copy of the open tab's columns** (names, order, which count as done) and opens
Rename	Tab menu → Rename. The slug stays
Reorder	Drag a tab along the strip, or Tab menu → Move left / Move right (the keyboard's way)
Edit columns	Tab menu → Edit columns: board configuration, on that tab

Delete	Tab menu → Delete opens a panel built on the server. The last tab cannot be deleted. A tab holding tickets needs a destination tab, and the panel previews where each column's tickets will land

Deleting a tab never deletes its tickets. They move to the chosen tab, each to:

1. the column with the **same name** (ignoring case);
2. otherwise, if it was in a done column, the destination's **first done column**;
3. otherwise the destination's **first column** —

appended at the end, in the order they had. Each move is in the ticket's timeline with the reason (*its tab was removed*); nobody is notified per card and Slack is not told about tickets that were already done. A name match wins over the done rule, so a done ticket can land in a same-named column that is not done (the preview shows it). `App\Support\TabColumnMapping` holds the rule; `App\Actions\Tabs\DeleteTab` applies it in one transaction under the board lock.

Moving between tabs

FROM	HOW
The board	Drag a card onto another tab (staff). It lands by the rule above, at the end
The ticket page	The **Tab** choice beside Status (staff with `move`). The Status list is always the columns of the ticket's own tab
The assistant, MCP	`tab` on update — see section 9

`App\Actions\Tickets\MoveTicketToTab` is the one way a ticket changes tab. `MoveTicket` itself refuses a column in another tab unless told the move is across tabs, so a stale or forged column id from a drag can never carry a ticket somewhere its board view does not show.

Customers

Customers see **every tab's name** — the board's structure is not secret, as with columns — but inside a tab only the tickets they can already see, and the counts count only those. They may say which tab a request belongs in (New ticket form, quick add on the tab, MCP); it always starts in that tab's first column. They cannot move tickets between tabs.

7.5 Columns

`/boards/{board}/settings` — `App\Livewire\Boards\Settings` — edits **one tab's columns at a time**: the tab in `?tab=`, chosen with the pills above the list (shown when the board has more than one tab).

PROPERTY	DETAIL
Name	Up to 60 characters, **unique per tab**
Position	Integer within the tab, drag-and-drop reorderable

`is_done`	Marks a column as terminal — drives *Ticket moved to done* notifications and closure statistics

A tab always keeps at least one column (enforced in `DeleteColumn`, under the board lock), and a deleted column's tickets can only move to another column **of the same tab**.

Deleting a column moves every ticket out of it one model at a time. The move is recorded in each ticket's timeline and broadcast, but **the "moved to" notification is suppressed** — otherwise tidying a board would notify everyone assigned a card on it. This is done with a declared property on the model (`$withoutStatusNotification`) rather than a flag on a service, so it can never be left globally switched on by a forgotten `finally`.

The foreign key from `tickets.board_column_id` is `restrictOnDelete`, so a column cannot be deleted out from under its tickets by accident.

7.6 Labels

Also on `/boards/{board}/settings`.

PROPERTY	DETAIL
Name	Up to 40 characters, **unique per board**
Colour	From the fixed `App\Enums\LabelColor` palette, default `slate`

Labels are board-scoped. Attaching one is staff-only; a customer can see the labels on a ticket they may read but cannot change them.

7.7 Members

`/boards/{board}/edit` — `App\Livewire\Boards\Members`, administrator-only.

Membership is an explicit `board_members` row with a unique `(board_id, user_id)` index and an `added_by_id` audit column. Administrators are **not** implicit members — `Board::hasMember()` answers a factual question about the pivot table, while `BoardAccess::canView()` answers the authorization question and applies the administrator bypass.

Only **active, non-customer** members appear in the assignee picker (`Board::assignableMembers()`), because assignment means "this person is doing the work".

Three events are raised for board lifecycle and feed the workspace activity log: `BoardCreated`, `BoardMemberAdded`, `BoardMemberRemoved`, plus `BoardUpdated`.

7.8 Board visibility and customers

A board has **no visibility flag**. A customer either is a member or is not. What varies is what they see *inside* it:

INSIDE A BOARD A CUSTOMER BELONGS TO	VISIBLE TO THEM?
Customer-visible tickets	Yes
Internal tickets	No — and not their count, not their existence
The customer conversation on a ticket	Yes
Internal notes	No — and not their count
Published documentation pages	Yes, if every ancestor is published too
Internal documentation pages	No
Tabs, columns and labels	Yes — the board structure is not secret (tab counts include only what they can see)
AI runs	No — 404
Attached repositories and GitHub activity	No
Board AI chat, AI settings, integrations screens	No — 404

7.9 The Kanban board

`/boards/{board}` — `App\Livewire\Boards\Show`.

One component owns the whole board rather than nesting a component per card. A drag-and-drop move has to re-render the source column, the destination column and both counts together, and with per-card components that would be several round trips instead of one.

FEATURE	DETAIL
Tabs	One tab at a time; the strip switches between them and shows per-tab counts (see *Tabs*)
Drag and drop	Cards between columns and within a column of the tab on screen, and onto another tab in the strip. Uses Alpine's `x-sort`, bundled with Livewire 4
Filtering	By search term, assignee, label, priority and visibility (`App\Support\TicketFilters`) — board-wide, kept across tabs
Quick add	A ticket can be created inline from a column; it is filed in the tab on screen
Realtime	The board re-reads when `BoardUpdated` is broadcast to a channel this viewer may join
Access	Decided once in `mount()` by `BoardPolicy::view` (404 on denial); every ticket read then goes through `TicketFinder`

7.10 Repository attachment

`board_repositories` — configured on the board AI settings screen.

COLUMN	MEANING
`repository_name`	`owner/name`, unique per board
`repository_url`	Optional display URL
`default_branch`	Defaults to `main`
`is_primary`	Which repository apply mode targets when a run does not name one
`description`	Free text, fed to the AI as context
`configuration`	JSON — build and test commands surfaced to the assistant

Several boards may attach the **same** repository, which is legitimate: a shared platform repository serves more than one customer's board, and each gets its own links when a ticket key matches.

> **IMPORTANT**
> Repository attachment is the **security boundary for inbound GitHub webhooks**. A delivery may only reach tickets on boards where that repository has already been attached. See section 13.

SECTION 8

Ticket Management

8.1 Anatomy of a ticket

FIELD	TYPE	NOTES
`number`	integer	Per board, allocated under a row lock. **Unique with** `board_id`
`title`	string(200)	Required
`type`	enum	`bug`, `task`, `feature` — default `task`
`description_md`	text	Markdown, written through the rich-text editor
`priority`	enum	`critical`, `high`, `medium`, `low`, `nice_to_have` — default `medium`
`assignee_id`	FK users	Active non-customer members of the board only
`estimate`	decimal(6,2)	Staff-only field
`due_date`	date	Indexed
`customer_visible`	boolean	**Defaults to** `false` — the customer boundary
`board_column_id`	FK columns	Status — and, through the column, the ticket's **tab**. `restrictOnDelete`
`position`	integer	Order within the column
`created_by_id`	FK users	Nullable on user deletion

The human key is `PREFIX-NUMBER` — `AQD-42` — built from the board's `ticket_prefix`.

IMPORTANT

`customer_visible` is stored **positively and defaults to** `false`, so a row created by code that forgets to set it is internal rather than exposed. It is deliberately **not mass-assignable**: only `App\Actions\Tickets\*` can set it, which is where the rule "a customer may never create or reveal an internal ticket" lives and is tested.

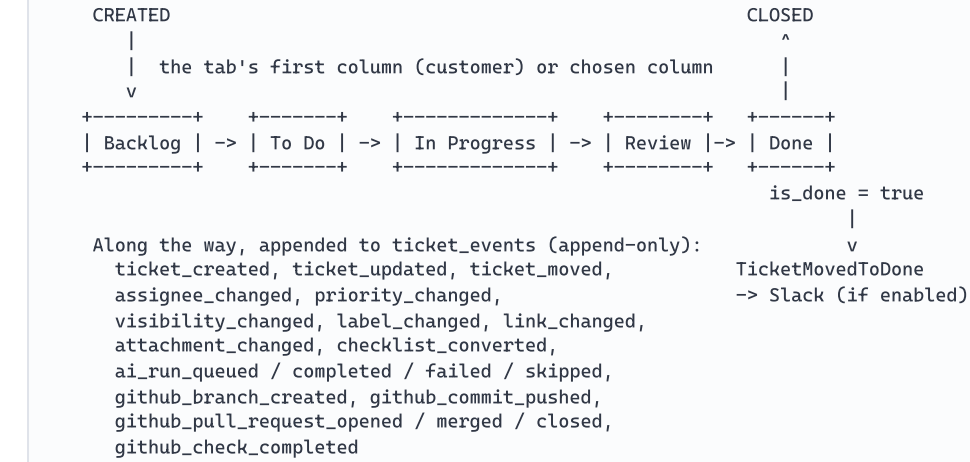
The screens that create tickets — the New ticket form, quick add and the assistant — pass a value explicitly, starting from the board's `new_tickets_visible_to_customers` setting (true by default). The column's `false` is the fail-closed backstop for code that does not choose.

8.2 Ticket numbering

Numbers are allocated inside `CreateTicket` with `SELECT ... FOR UPDATE` on the board row, then `next_ticket_number` is incremented. Combined with the unique `(board_id, number)` index, this makes concurrent creation safe: two simultaneous requests cannot receive the same number, and a lost update cannot produce a duplicate key.

8.3 Ticket lifecycle

FIGURE



8.4 Creating a ticket

`/boards/{board}/tickets/create` — `App\Livewire\Tickets\Create`. A quick-add control on the board does the same thing inline.

The rules live in `App\Actions\Tickets\CreateTicket`, not in the form, so they hold no matter which screen — or which AI proposal — calls it:

RULE	CUSTOMER	STAFF
Visibility	**Always customer-visible**	Defaults to customer-visible (the board's `new_tickets_visible_to_customers`); untick for internal work
Tab	May choose (they see every tab's name)	May choose
Column	**Always the first column of its tab**	Any column of its tab
Assignee	**Always unassigned**	Any assignable member
Estimate	Not settable	Settable

Which tab. `board_tab_id` if it is one of this board's tabs, else the tab of `board_column_id` if that column is on this board, else the board's default (leftmost) tab. Staff then get their column if it is in

that tab, otherwise the tab's first column. The New ticket form starts in the tab the board was showing (its *New ticket* button passes `?tab=`) and offers a **Tab** choice when the board has several; quick add files into the tab on screen. The board row is locked before the tab and column are chosen, so a tab or column deleted at the same moment cannot pull the column out from under the insert.

A tampered `board_tab_id` or `board_column_id` cannot file a ticket into another customer's board: the action re-resolves both against *this* board and falls back rather than trusting them.

8.5 Editing a ticket

`/boards/{board}/tickets/{number}` — `App\Livewire\Tickets\Show`.

`TicketPolicy::update` allows staff always, and a customer **only for a ticket they raised themselves**. `UpdateTicket` then narrows which fields a customer may touch, silently dropping `customer_visible`, `assignee_id` and `estimate`.

The diff is taken from Eloquent's own dirty tracking, so an "update" that changes nothing records nothing and the timeline stays readable.

Status and tab. The Status choice lists the columns of the ticket's own tab. On a board with several tabs, staff with `move` also get a **Tab** choice, which moves the ticket to that tab (`MoveTicketToTab`: the column with the same name, else a done column if it was done, else the tab's first column) and says where it landed; customers see the tab's name. Both choices re-read the ticket on every request, so a move somebody else made is never shown stale.

8.6 Ticket addressing

A ticket is addressed by its **per-board number**, so the URL reads the way the team talks: `/boards/aqueduct-platform/tickets/42` is `AQD-42`. The number is bound as a plain integer rather than as a model — resolving it is `TicketFinder`'s job, which applies board membership and the customer rule and raises 404 when either fails. An internal ticket and a number that was never used therefore look identical from outside.

Tabs change nothing here: numbers and keys are per board, across all its tabs, and a ticket keeps its key when it moves between tabs. On a board with several tabs the trail reads *Board › Tab › KEY*, and *Back to board* opens the ticket's tab.

8.7 Labels, checklists, links and attachments

FEATURE	MODEL	WHO MAY CHANGE IT
Labels	`ticket_label` pivot	Staff only
Checklist	`ticket_subtasks` — title, completed, position, `completed_at`, `completed_by_id`	Follows ticket update, so a customer may tick items on their own ticket
Links	`ticket_links` — `blocks`, `relates_to`	Staff only, and only to tickets they can already see

Attachments	Polymorphic `attachments`	Follows ticket update

Checklist changes are deliberately **not** written to the ticket timeline: a checklist changes often and would drown the events that matter.

Checklist conversion

A ticket's standalone checklist can be moved into its description as Markdown checkboxes (`ConvertSubtasksToChecklist`). This is **per ticket, on a button press** — never an automatic data migration. The `ticket_subtasks` rows are removed, but the `checklist_converted` event carries every item exactly as it stood, including the `completed_at` and `completed_by_id` that a Markdown checkbox cannot express. The list is *appended* under a heading rather than replacing anything.

8.8 The two conversation streams

Every ticket carries two independent threads, defined by `App\Enums\CommentStream`:

STREAM	WHO READS IT	WHO WRITES TO IT
`customer`	Everyone who can read the ticket	Staff always; customers if `customers_can_comment` is on for the board
`internal`	**Staff only**	Staff only

> **IMPORTANT**
> Customers must never receive internal rows, **their ids, their authors or their count**. The rule has one definition — `CommentStream::isCustomerFacing()` — which the query scope, the policy and the notification dispatcher all ask.

`App\Actions\Comments\PostComment` owns the audience rule:

- a comment written by a customer is **always** in the customer stream, whatever the request said;
- staff choose their stream explicitly, and the default when nothing is said is **internal**, so a code path that forgets to choose keeps quiet rather than publishing to a customer.

Everything that follows a post — who is notified, what is broadcast — is derived from the stream that was actually stored, never from what the caller asked for.

Comments support `@mentions` of board members (`App\Services\MentionParser`), soft deletion with a `deleted_by_id`, an `edited_at` marker, and attachments. Editing is author-only; deletion is author-or-administrator.

8.9 Ticket activity timeline

`ticket_events` is **append-only** and is the ticket's history. It carries `board_id`, `from_column_id` and `to_column_id` as indexed columns specifically so that flow metrics — "which tickets entered a done column on this board last week" — are one index range scan.

Twenty event types are defined, spanning human actions, AI runs and GitHub activity. They appear in one unified timeline, so "branch created" sits between "Vikas created NL-3" and "Alex changed priority" rather than in a separate panel.

Every event is mirrored into the workspace activity feed through `App\Services\TicketActivity`, so there is no second activity system to keep in step.

8.10 Filtering and search

`App\Support\TicketFilters` drives board filtering:

FILTER	VALUES
Search	Title, description, and the ticket number — `AQD-42`, `aqd-42` and `42` all match ticket 42
Assignee	Any assignable board member, or unassigned
Label	Any label on the board
Priority	Any of the five
Type	`bug`, `task`, `feature`
Visibility	Internal / customer-visible (staff only)

8.11 What happens when a ticket changes

`App\Observers\TicketObserver` fans every change out. It is an observer rather than four calls in four actions, because "every change to a ticket is announced" is a property of the model, not of any one screen.

FIGURE

```
Ticket saved
     |
     +--> BROADCAST      BoardBroadcaster decides the audience. An internal
     |                   ticket signals only board.{id}.internal, so a
     |                   customer's browser is never even woken.
     |
     +--> NOTIFY         NotificationDispatcher writes one row per recipient,
     |                   each policy-checked. A customer cannot be notified
     |                   about an internal note even if something upstream tried.
     |
     +--> AI             On creation only, and only for a customer's ticket:
     |                   TriggerAutomaticAiRun (subject to board settings,
     |                   capability mode and the daily cap).
     |
     +--> SLACK          Per-board, per-event opt-in. Queued.
     |
     +--> SMS            Critical tickets only. Per-board opt-in. Queued.
```

None of these may throw — nothing here is important enough to fail a ticket save. Broadcasting is queued; notification writes share the caller's transaction, so a rollback takes them with it.

Position rewrites during a drag are done with the query builder rather than on models, so shuffling ten cards produces **one** announcement from the ticket that actually moved, not eleven.

8.12 In-app notifications

NOTIFICATION	TRIGGER	RECIPIENTS
`TicketAssigned`	Assignee changes	The new assignee
`TicketStatusChanged`	Column changes	Assignee and participants
`TicketPriorityChanged`	Priority changes	Assignee and participants
`CommentPosted`	A comment is posted	Ticket participants who may read that stream
`MentionedInComment`	`@mention` in a comment	The mentioned person, if they may read it

All are stored in the `notifications` table and surfaced by the bell. Every recipient is checked against the same policy that guards the thing being notified about.

SECTION 9

Documentation System

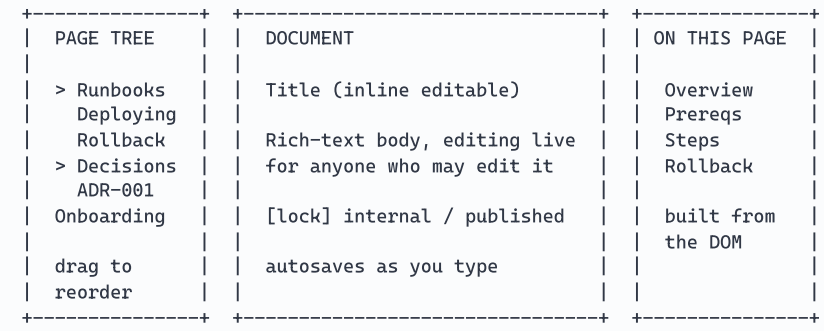
9.1 Purpose

Each board carries its own documentation workspace: a nested tree of pages with a live WYSIWYG editor, per-page publication to customers, attachments, and a contents list. It is where runbooks, architecture notes, decisions and customer-facing guides live next to the work they describe.

9.2 Layout

Three columns:

FIGURE



One Livewire component (`App\Livewire\Docs\Show`) serves both `/boards/{board}/docs` and `/boards/{board}/docs/{slug}`, because the tree is on screen either way and reordering has to work from both. A separate index component would mean two copies of the drag-and-drop handler — exactly the kind of duplication that lets one copy forget a check.

9.3 Page hierarchy

PROPERTY	DETAIL
Parent	`parent_id`, self-referencing, `restrictOnDelete`
Maximum depth	**4** (`DocPage::MAX_DEPTH`); root pages are depth 0
Ordering	`position`, drag-and-drop, within each parent
Slug	Unique per board; used in the URL, never the primary key
Icon	An optional emoji, chosen from a picker (`icon`, 16 chars)

The depth bound exists so that ancestor walks terminate in a known number of queries, and so the sidebar stays legible.

9.4 The ancestor visibility rule

A tree needs one rule a flat table does not: **a page is visible only when it and every one of its ancestors are visible.** Publishing a child of an internal page would otherwise show a customer a page whose parent, path and siblings they must not see — and the breadcrumb alone would leak the internal titles.

That rule is enforced in three deliberately overlapping places:

WHEN	WHAT HAPPENS
On write	`SetPageVisibility` refuses to publish a page whose parent is internal, and cascades "make internal" down the whole subtree. `MovePage` refuses a move that would put a published page under an internal one
On read of one page	`DocPageFinder` walks the ancestor chain through the visibility scope and 404s if any link is missing
On read of the tree	`DocPageFinder` drops any node whose parent is not in the visible set, so a broken invariant cannot surface as an orphan

> **IMPORTANT**
>
> The two visibility operations are deliberately **asymmetric**. Publishing is *refused* when an ancestor is internal — rather than silently publishing the ancestors too — because exposing a parent page is the author's decision, not a side effect. Un-publishing *cascades* down the whole subtree, because anything else would leave published children hanging off an internal parent. Retracting always fails closed: more is hidden, never less.

For staff the ancestor walk is skipped entirely, since a member of the board can see every page on it. For a customer it costs at most four extra lookups.

9.5 The editor is not a mode

Anybody who may edit a page gets the editor **immediately, on the page they opened**. There is no Edit button and no read-only step in front of it.

The reasoning is in the code: a documentation tool whose default state is "looking at" rather than "working on" makes every small correction a four-click job, which is how documentation goes stale. A reader who may not edit gets the rendered document, and never so much as the markup of a control they could not use.

The editor is **TipTap** (ProseMirror), registered as an Alpine component. It is headless — no stylesheet to fight Tailwind with, no theme, no toolbar markup — so the toolbar is ordinary Blade buttons styled like every other button in the product. It is framework-agnostic vanilla JavaScript, so no React or Vue enters the build.

Supported: headings, bold, italic, underline, strikethrough, inline code, code blocks with syntax highlighting, links, bullet and ordered lists, task lists, block quotes, horizontal rules, images and

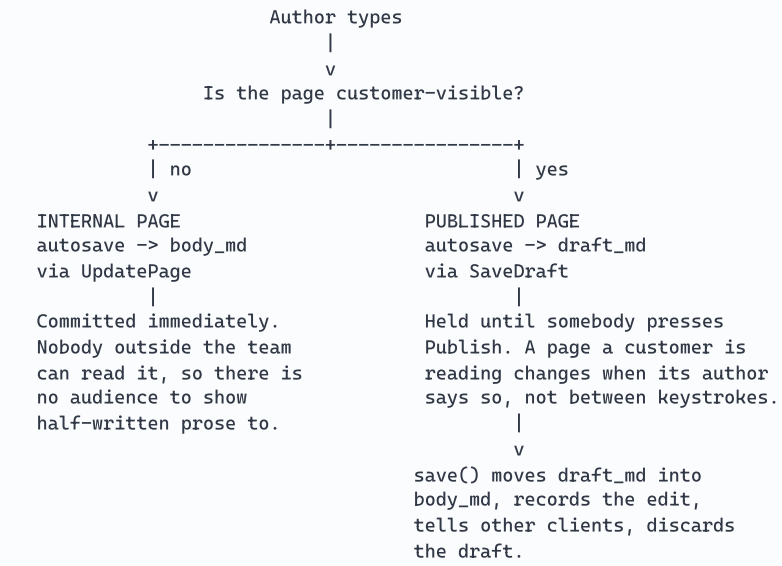
tables.

> **NOTE**
>
> **Markdown is the storage format.** The WYSIWYG editor is a surface over Markdown, and the HTML-to-Markdown conversion is done **server-side** (`league/html-to-markdown` via `App\Support\RichText`). The stored artefact is always `body_md` . This keeps documents readable, searchable, diffable and exportable without the editor, and means a future change to how documents are rendered cannot alter what was stored.

9.6 Autosave, drafts and publishing

Where an autosave lands depends on **who can read the page**, and this is the load-bearing decision of the whole editor.

FIGURE



Supporting columns: `draft_title` , `draft_md` , `draft_saved_at` , `draft_by_id` .

`DocPage::hasDraft()` uses `draft_saved_at` as the marker rather than `draft_md` , so a draft that deliberately empties a document still counts as unsaved work. A draft identical to what is saved does not count, so no recovery banner appears over nothing.

Two properties make autosaving on a timer safe:

- `UpdatePage` returns early unless the row is actually dirty;
- `ActivityLogger` coalesces an editing session into **one feed entry per person per page**.

An afternoon's typing is therefore a handful of writes and one line in the activity feed.

> **NOTE**
>
> **Autosave never broadcasts**, on either branch. A colleague reading the board should not have their screen pulled out from under them every few seconds because somebody two desks away is typing.

9.7 Contents list ("On this page")

Built from the **DOM**, not from the Markdown — which is what lets one implementation serve both states of the page. A reader is looking at rendered HTML; a writer is looking at a live ProseMirror document that has no server-side representation between keystrokes. Both are headings in an element.

Headings are addressed **by position, not by id**. Inside the editor ProseMirror owns the DOM and reconciles away attributes it did not put there, so ids written by the script are silently stripped and every anchor would resolve to nothing. A heading is therefore identified by its index among the headings, and the click handler looks the element up again at click time.

A contents list appears only when the document has at least 3 headings. Nothing about it is persisted or sent anywhere.

9.8 Permissions

ACTION	ADMINISTRATOR	TEAM MEMBER	CUSTOMER
Open the documentation section	Yes	Member only	Member only
Read a published page	Yes	Yes	Yes — if every ancestor is published
Read an internal page	Yes	Member only	**No — 404**
Create a page	Yes	Member only	**No**
Edit a page	Yes	Member only	**No**
Move or reorder a page	Yes	Member only	**No**
Delete a page	Yes	Member only	**No**
Publish or retract a page	Yes	Member only	**No**
Attach a file	Yes	Member only	**No**
Import a file as a page	Yes	Member only	**No**

Documentation is the delivery team's material, which they publish to customers a page at a time. **A customer never authors it.**

9.9 Importing files as pages

A file somebody already has — a to-do list, a bug list, meeting notes, a spec — becomes a page without retyping it.

WHERE	HOW

Sidebar	**Import file** beside **+ New page**, and the whole sidebar takes a dropped file
Empty documentation	**Import a file** beside **Create your first page**, and the card takes a dropped file
A page's **More** menu	**Import a file inside this one** files the new page under it
The assistant	"Import this into the docs and make the tickets" — section 9

Each file becomes an **internal page** (`App\Actions\Docs\ImportDocument`), its title the document's own (a leading heading, Word's Title, YAML front matter, a text file's opening line) or else the file name, and **the original is attached to the page** — so anything a conversion could not keep is one click away. One file that cannot be read is named in the sidebar, and the rest still import.

`App\Services\Docs\Import\DocumentImporter` turns each format into the Markdown a page stores, keeping the structure a to-do or bug list is made of:

FORMAT	WHAT IS KEPT
Word `.docx`	Headings (from the styles, found by name or outline level), bullet and numbered lists with nesting, checkbox controls and box bullets as to-dos, bold/italic/strike, web links, tables, footnotes. Pictures are counted and left in the original
PDF	The text, with running headers, footers and page numbers removed and wrapped lines joined into paragraphs; bullets and `[ ]` / `☐` as lists and to-dos. A scan (no text layer) is refused with a reason
Text `.txt`	Bullets, numbering, checkboxes, `Bugs:`-style and capitalised headings, the author's line breaks
Markdown	As written, less YAML front matter
CSV / TSV	A table (delimiter detected), first 500 rows
Excel `.xlsx`	A heading and a table per sheet, first 500 rows × 30 columns
PowerPoint `.pptx`	A heading per slide, its points nested by level, speaker notes as a quote

The legacy `.doc` , `.xls` and `.ppt` are refused with "save it as .docx" — they are binary formats and reading them would mean a new parser for untrusted input.

IMPORTANT

Safety. Office files are ZIPs of XML: they are read inside the existing zip-bomb limits (`App\Support\Ooxml\OoxmlPackage`) with a streaming reader that refuses any part declaring a document type and loads no DTD or entity (`App\Support\Xml\SafeXml`), so the XXE class of attack does not arise. The type is decided by the extension **and** checked against the bytes. Output is escaped for the renderer — `<` and `++` would otherwise vanish or turn into underline — and written in the editor's own Markdown forms, so opening an imported page in the editor rewrites nothing.

Limits (`config/attachments.php`, `imports`): 10 files per drop, 20 MB each; a body holds 200,000 characters — past that it is cut at a paragraph and says the rest is in the original. A file over 8 MB is converted on the attachments queue (`ImportDocumentFile`) while the sidebar shows *Importing…*; a drop larger than the server's request limit goes up in batches.

Create tickets with AI

A page with open to-dos shows **Create tickets with AI · N to-dos** in its header (and the item is in the **More** menu on every page). It opens the assistant on the page's board, in Writing, with "Create tickets for the to-dos and bugs on the page …" written in the composer — nothing is sent until the person presses Send. The page's text reaches the model as *THE PAGE ON SCREEN*; tickets from an internal page start internal and link back to it. Offered only to somebody the assistant would let propose changes there.

9.10 Attachments

Pages carry polymorphic attachments through the same pipeline as tickets and comments, with the same limits and the same authorization — a file on an unpublished page is internal because the page is. An imported page's original file is one of them.

9.11 Search

Documentation is one of the four command-palette categories, searched by title and body through `DocPageFinder` so the ancestor rule is applied to results as well as to pages.

9.12 What is not implemented

CAPABILITY	STATUS
Page version history / revisions	**Not implemented** — only the current body and one draft are stored
Page comments	**Not implemented**
Cross-board page links as first-class relations	**Not implemented** — ordinary Markdown links work
Templates	**Not implemented**
Export a page or space to PDF/Word	**Not implemented**
Real-time collaborative editing (multiple cursors)	**Not implemented** — autosave is single-author

SECTION 10

Workspace AI

This is Nexora's largest subsystem: roughly a third of the application's classes. It is worth reading the three orthogonal settings first, because almost every question about AI behaviour resolves to one of them.

10.1 Three settings that are not the same question

SETTING	ENUM	WHO CHOOSES IT	WHAT IT DECIDES
Capability mode	`AiCapabilityMode`	An administrator (workspace) or the team (board)	The **ceiling** on what the AI may ever *attempt* here
Chat mode	`AiChatMode`	The person asking, per conversation	What this conversation is *for* — reading, writing, or both
Knowledge scope	`AiKnowledgeScope`	The person asking, per conversation	Where an answer may be *sourced* from

They are not substitutes, and **none of them can grant what another refuses.**

Capability mode — the security dial

Three cumulative levels, expressed as ordered levels rather than a matrix of switches because a matrix reads well on a settings screen and is impossible to reason about in a security review.

MODE	BADGE	MAY READ AND ANSWER	MAY PROPOSE WRITES	MAY RUN A TICKET ANALYSIS	MAY WRITE CODE / OPEN A PR	MAY RUN UNATTENDED
AI Observer	slate	Yes	**No**	**No**	No	No
AI Operator	brand	Yes	Yes — confirmed by a person	Suggest runs	No	No
AI Agent	amber	Yes	Yes	Suggest and apply	Yes	Yes

> **NOTE**
> Observer refuses ticket runs entirely, and that is stricter than people expect **on purpose**: a suggest run ends by posting an internal note, and a note is a write.

The shipped default is `agent` (`AI_DEFAULT_MODE`), which is a compatibility decision: it is exactly the set of capabilities the product had before modes existed. The database column default for a new `ai_settings` row is `observer`. A fresh deployment wanting the cautious posture should set `AI_DEFAULT_MODE=operator` or `=observer`.

IMPORTANT

The capability mode is a ceiling, never a floor. Every method on `AiCapabilityGuard` can only ever *deny*. None of them grants anything, none is consulted instead of a policy, and raising a workspace to AI Agent gives no user a capability they did not already have by hand. Order of checks on any write:

1. May this person use the assistant at all? — route group, `BoardPolicy`, SQL scopes.
2. Does the mode permit the AI to attempt this? — `AiCapabilityGuard`.
3. Is **this person** allowed to make **this** change? — `TicketPolicy`, `DocPagePolicy`, `AiRunPolicy`, applied against the confirming user by the ordinary action.

Chat mode — what the conversation is for

MODE	READ TOOLS OFFERED	WRITE TOOLS OFFERED	DEFAULT MODEL
Reading *(default)*	Yes	**No**	`claude-sonnet-5`
Writing	**No**	Yes	`claude-opus-5`
Everything	Yes	Yes	`claude-opus-5`

Reading is the default, so the setting somebody gets by not choosing is the one that changes nothing. Writing withholds the lookups so that "write me a ticket for the login bug" does not become four searches and a summary before it gets to the draft — the board context block still goes in on every turn, so the model is never asked to draft blind.

Each mode names its own model because the two jobs have different shapes: lookup-and- summarise is bounded and mostly retrieval, whereas drafting a ticket somebody will act on is the turn worth spending on. The mapping is deployment configuration (`AI_CHAT_READING_MODEL`, `AI_CHAT_WRITING_MODEL`, `AI_CHAT_EVERYTHING_MODEL`), and the chosen id is still resolved against the effective provider.

Knowledge scope — "Outside Project"

SCOPE	BEHAVIOUR
Project only *(default, and not configurable away)*	Answers only from this project and what the asker can see in it
Outside Project	Answers from this project **and** from general knowledge outside it

Outside is **additive, never a replacement**. The board context still goes in, the read tools are still offered, and a question about NL-5 is still answered from NL-5 — because "compare our architecture with a typical Laravel SaaS" needs both halves at once.

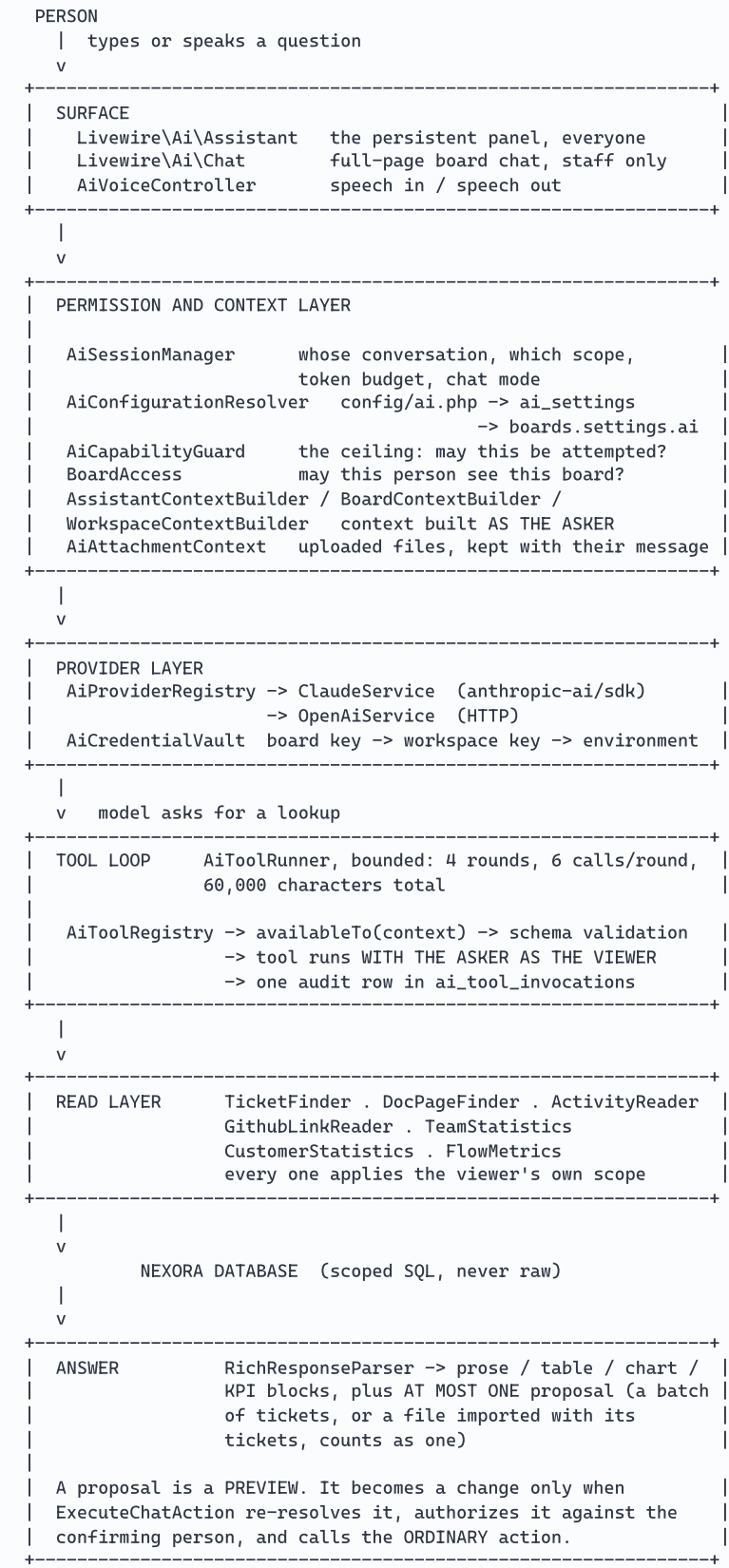
Project is the default for two reasons, and the second is the one that matters: the failure modes are asymmetric. A project-only assistant asked "what is Laravel?" says it cannot answer that here, which is a mild annoyance with an obvious remedy. An assistant that has quietly reached outside says

something confident about somebody's project that is not true of their project, with no signal that it happened.

> **IMPORTANT**
>
> Knowledge scope has **nothing to do with authorization**. Switching it does not widen what a person may read by a single row and does not permit a change they could not otherwise make. It is enforced by the **data layer** — `AiToolRegistry` decides which tools exist for a turn — not by the prompt.
>
> One honest limit is written down in the code: a language model's own trained knowledge is not a data source this application can withhold. Under Project the assistant is *instructed* to decline general questions, and that instruction is a prompt rather than a wall. What the data layer does guarantee is narrower and stronger: **no external service is contacted, and no project data is widened either way.**

10.2 AI architecture

FIGURE



10.3 The two surfaces

SURFACE	ROUTE	WHO	NOTES
Assistant panel	Mounted in the layout	Everyone with at least one reachable board, **customers included**	Persists across `wire:navigate`. Costs nothing until opened — `revealed` starts false and `render()` returns an empty shell
Board AI chat	`/boards/{board}/ai`	**Staff members of the board only** — 404 otherwise	Full page. Quotes internal tickets, internal notes and internal documentation

The two exist as **two abilities rather than one loosened ability** (`BoardPolicy::useAiChat` and `::useAssistant`), because the surfaces differ in what they are allowed to contain, and a single relaxed check would have quietly opened the staff page as well.

The panel **never aborts**. Where `Chat` 404s on an authorization failure — right for a page a customer must not know exists — doing that in a layout component would take down whatever page it happened to be rendered on. So every failure there narrows the panel to its board picker instead. The security outcome is the same with no collateral damage.

10.4 Context scope

A conversation is about one of two things (`App\Services\AI\AiContextScope`):

SCOPE	CONTEXT ASSEMBLED
One board	Up to 60 tickets, 40 documentation pages, 40 activity events — all read through the asker's own visibility
All workspace	Up to 12 boards, 6 tickets each — **titles only**, no descriptions, no documentation, no history

The workspace scope is much thinner per board on purpose: breadth multiplied by the depth of a single-board context would spend the whole window on material the question probably does not need.

The page on screen. When the person has a documentation page of the conversation's board open, its full text (up to 40,000 characters, `AI_CHAT_CONTEXT_PAGE_CHARS`) goes with the question as *THE PAGE ON SCREEN* — the draft for somebody who may edit the page, the saved body otherwise — read through `DocPageFinder` with the asker as the viewer (`PageContextResolver::pageOnScreen()`). That is what makes "create tickets from this page" work in Writing mode, which has no read tools to fetch the page with.

The scope is held in the session as a **slug**, never a primary key, and is re-resolved through `BoardAccess` on every render. An unauthorized slug produces the same panel as no selection — never another board's conversation, and never an error that confirms the board exists.

Changing scope drops the remembered session, because a session belongs to one scope. The chat mode is **not** cleared with it: it is what the person wants the assistant to be doing, which does not change because they changed the subject.

10.5 Sessions and conversation history

`ai_sessions` — one row per conversation.

COLUMN	PURPOSE
`uuid`	Public identifier; primary keys are never exposed
`user_id`, `board_id`	Owner and scope. A null `board_id` is a workspace conversation
`provider`, `model`	What this conversation is actually running on
`capability_mode`, `chat_mode`, `knowledge_scope`	The three settings, as they stood
`tokens_input`, `tokens_output`, `estimated_cost`	Metering. `tokens_input` is every token read, cached or not
`tokens_cache_read`, `tokens_cache_write`	The parts of the input served from, and written to, the provider's cache
`context_tokens`	How much of the model's window the conversation fills: the last exchange's input plus its answer. Set, not summed
`history_anchor_message_id`	The first message still replayed once the conversation has outgrown the window; only moves forward
`message_count`, `started_at`, `last_activity_at`, `ended_at`	Lifecycle

IMPORTANT

A conversation belongs to one person — an administrator included. There is no reading right over a colleague's thread, and therefore none over the documents in it. `AiSessionPolicy` checks, in this order: the account is active; **ownership**; the board is still reachable. Every denial is a 404.

Sessions replaced a single per-person-per-board thread for one reason: context. A month-old thread re-sends a month of turns to answer a question about today, which is slower, dearer and worse.

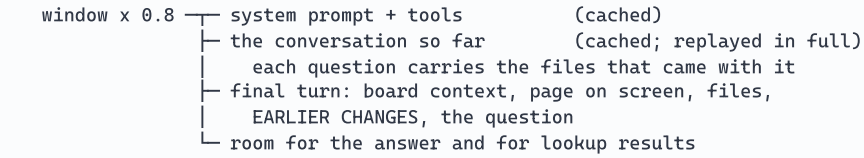
One conversation on screen per person per context. The panel and the full-page chat show the person's current session for the selected board (or the workspace). **New conversation** — the **+** in the panel header, a button beside *Clear conversation* on the full page — calls `startNewSession()`: the current session is *ended*, never deleted, and the next question starts with an empty context. It is disabled on a session nothing has been asked in (`AiSession::isFresh()`), which already is a fresh one. A note above the composer (`components/ai/context-hint`) suggests starting a new one when the conversation nearly fills the model's window (`AiSession::isNearingContextWindow()`, from 80%), once its earliest messages have left the model's view (`hasDroppedHistory()`), or — only where a deployment has set a spending cap — from 80% of that cap. Ended sessions are kept for metering and audit but are not listed anywhere in the interface.

Transcripts persist in `ai_chat_messages`; **prompts do not**. The system prompt and the context are rebuilt on every request and never stored, so editing a board's project context changes the next answer instead of being frozen into an old row.

The whole conversation, cached

Each question replays **the whole conversation**, up to the session model's real context window — 1,000,000 tokens for Opus 5 and Sonnet 5, 200,000 for Haiku 4.5, from the model catalogue (`AiSession::contextWindow()`). `WorkspaceChatService::assemble()` budgets every request: `AI_CHAT_CONTEXT_BUDGET` (80%) of the window, less the system prompt and tool definitions, the final turn (board context, files, earlier changes, the question), room for the answer and room for what lookups bring back, estimated at three characters a token.

FIGURE



- **Trimmed in one jump.** When the conversation outgrows its budget (or `AI_CHAT_HISTORY_MAX_MESSAGES`, 500 stored turns), its oldest part is dropped down to 60% of the budget, from a question — and the trim point (`history_anchor_message_id`) then stays put. Sliding forward a message at a time would change the beginning of every request and defeat the cache. The model is told, in a fixed sentence, that the earlier part is not shown.
- **Prompt caching** (Anthropic). `ClaudeService` places three breakpoints: the system prompt (which covers the tools), the last replayed message, and — only when lookups were offered — the final turn, so rounds two to four of a tool loop re-read it. A cached read costs a tenth of a fresh token; writing costs a quarter more (twice as much with `AI_CHAT_CACHE_TTL=1h`). Only the chat asks for caching; a one-shot ticket run never pays the write premium. OpenAI caches a repeated prefix on its own.
- **Files stay in the conversation.** A file goes in whole on the question it was first used with, and that question records it (`metadata.files_sent`); the history replays it there on every later turn, and the latest turn only points back to it (*PROVIDED IN FULL EARLIER*). A file whose question has been trimmed away goes in whole again. That replaced "full text once, a digest thereafter", which left the model with a 1,200-character excerpt by the second question.
- **EARLIER CHANGES** is read from every proposal in the session, not only from replayed turns, so trimming cannot make the model recreate tickets it already made.
- **Too long anyway?** A provider 400 "prompt is too long" (`AiProviderException::contextTooLong()`) is retried once with half the history, before anything has been shown.

LIMIT	DEFAULT	VARIABLE
Tokens per session (a spending cap; **off**)	0	`AI_SESSION_TOKEN_LIMIT`
Tokens per user per day	2,000,000	`AI_DAILY_USER_TOKEN_LIMIT`

What a cached token counts for against both	0.1	`AI_CACHE_READ_WEIGHT`
Sessions returned by `AiSessionManager::history()`	25	`AI_SESSION_HISTORY`
Stored turns replayed, at most	500	`AI_CHAT_HISTORY_MAX_MESSAGES`
Share of the model's window a request may use	0.8	`AI_CHAT_CONTEXT_BUDGET`
Window for a model the catalogue gives none for	128,000	`AI_CHAT_DEFAULT_CONTEXT_WINDOW`
Hold every conversation below the model's window	0 (off)	`AI_CHAT_CONTEXT_MAX_TOKENS`
Prompt caching, and how long the provider keeps it	on, 5m	`AI_CHAT_CACHE` , `AI_CHAT_CACHE_TTL`
Output tokens per answer	4,000	`AI_CHAT_MAX_OUTPUT_TOKENS`
Output tokens, Outside Project	8,000	`AI_CHAT_MAX_OUTPUT_TOKENS_OUTSIDE`
Output tokens, a turn that may propose changes	16,000	`AI_CHAT_MAX_OUTPUT_TOKENS_WRITING`

Zero means unlimited, and is written as zero rather than null so the settings form has one shape. The session and daily limits are **spending limits, not the context window**: limits count `input + output − 0.9 × cached reads`, so a conversation replayed from the cache spends an allowance at roughly the rate it spends money. A cap stored in *Admin → AI settings* or on a board still applies; the default is none.

10.6 The retrieval tool loop

The context block is a roll-up — recent tickets, recent activity, documentation titles. It makes a general question answerable cheaply, and it is **not enough for a specific one**: "which tickets are overdue" cannot be answered from the sixty most recently updated.

So the model is also given read tools, and `AiToolRunner` runs a bounded loop.

BOUND	DEFAULT	VARIABLE
Rounds before the model must answer	4	`AI_TOOLS_MAX_ROUNDS`
Lookups in one round	6	`AI_TOOLS_MAX_CALLS`
Total characters of tool material per question	60,000	`AI_TOOLS_MAX_CHARACTERS`
Master switch	on	`AI_TOOLS_ENABLED`

> **NOTE**
> These are **cost and latency controls, not security controls**. Security is the readers each tool calls with the asking person as the viewer, plus `availableTo()` on each tool. Turning `AI_TOOLS_ENABLED=false` narrows what the assistant can answer and grants nobody anything.

Every invocation goes through `AiToolRegistry::invoke()`, which makes four promises structural rather than aspirational:

PROMISE	HOW
Schema	The tool declares one; `AiToolInput` enforces it *before* the tool is entered
Authorization	A tool is offered only if `availableTo()` says so, and its reads are scoped to the viewer on the context
Validation	A call that does not fit is answered with prose the model can act on, not an exception
Audit	**One row in `ai_tool_invocations` per call, whatever the outcome** — a refused call is the interesting one to have a record of

A tool that throws is contained: the exception is logged in full and reported to the model as "that lookup failed", with nothing about why — an exception message can carry a query, a path or a column name, and none of that belongs in a conversation.

The loop cannot widen what a person may see. A tool argument names a *subject*; the viewer comes from the *request*. An assistant with tools answers more questions than one without, about exactly the same material.

The eleven tools

TOOL	WHAT IT READS	AVAILABILITY
`get_board`	Description, its tabs and each tab's columns with counts, labels, prefix. With no argument, lists reachable boards	Everyone
`get_ticket`	One ticket in full — with its tab on a board split into tabs — plus recent conversation and history	Everyone — scoped
`search_tickets`	By text, tab, column, label, priority, assignee, due date, age	Everyone — scoped
`get_completed_tickets`	Tickets completed — moved into a done column — in a named period or date range, from the history, optionally in one tab; flags tickets reopened since	Everyone — scoped
`search_documentation`	Page titles and bodies, with extracts	Everyone — scoped

`get_documentation_page`	One page in full by slug	Everyone — scoped
`get_statistics`	Eight aggregate metrics	Everyone; three metrics staff-only
`get_activity`	Workspace activity history	**Staff only**
`get_code_activity`	Branches, commits, pull requests, CI state	**Staff only**
`get_github_repository`	Attached repositories, default branch, build commands — **never credentials, never repository contents**	**Staff only**
`search_external_knowledge`	An outside service	Outside Project scope **and** a configured driver

The same tools — and changes — from Claude or ChatGPT (remote MCP)

People can add `https://<host>/mcp` to Claude (web, desktop, phone) or ChatGPT as a custom connector, sign in with their Nexora account, and ask questions — or ask for changes — there. For reading, those clients call the **same read tools through the same registry** (`App\Mcp\Tools\RegistryTool` wraps each contract and calls `AiToolRegistry::invoke()`), so an answer from Claude cannot differ from what the in-app assistant would have read for the same person.

`App\Mcp\Servers\NexoraServer` builds the tool context once per request from the OAuth token's user:

CONTEXT FIELD	MCP VALUE	WHY
`scope`	workspace	An external client has no "current board"; tools take a board or ticket key
`session`	`null`	Not part of an in-app conversation; the audit row records the call without one
`mode`	Observer	Governs only in-app proposals, which this path never makes; the write tools ask the board's real mode themselves
`staff`	`BoardAccess::canSeeInternalContent($user)`	The customer boundary, exactly as in the app
`knowledge`	Project	`search_external_knowledge` is never offered
`channel`	`mcp`	Answers carry links and status emojis instead of `nexora-*` rendering hints; audit rows filterable by channel

Customers get the seven customer read tools; staff also get the three staff-only ones. Disabling the read tools (`AI_TOOLS_ENABLED=false`) empties the read list. See section 18 for the endpoint and section 4 for the OAuth details.

Changes from an AI app

The in-app assistant never writes directly — it proposes and a person confirms. Over MCP the person is in a conversation in their own AI app, and **the app itself asks them before each call**: every write

tool is annotated `readOnlyHint: false` (and `delete_ticket` `destructiveHint: true`), which is what makes Claude and ChatGPT ask. So the change is made directly, through the **same code a confirmed proposal goes through** — `ExecuteChatAction::perform()` — and nothing re-implements a permission.

TOOL	STAFF	CUSTOMER	WHAT IT DOES
`create_ticket`	Every field: description, type, priority, tab, status (a column of that tab), assignee, labels, due date, estimate, visibility	Board, title, description, type, priority, tab	Raise one ticket. A customer's ticket follows the customer rules: first column of its tab, nobody assigned, visible
`create_tickets`	Up to 30, same fields — each ticket its own `tab`	—	All or nothing; a refusal names the ticket at fault
`update_ticket`	Any field, status, `tab` (move to another tab), assignee (`nobody` unassigns), labels (replaces the set), due date (`none` clears)	Title and description of a ticket **they raised**	The answer lists what changed, old → new — the tab included — read back from the saved row
`update_checklist`	Add, complete, reopen, remove items by text or number	—	`TicketPolicy::manageSubtasks`, as on the ticket screen
`add_comment`	Internal note by default; `visible_to_customer` posts on the customer thread	Customer thread only, where the board allows customer comments	
`delete_ticket`	After confirmation	—	Permanent
`create_doc_page`	Markdown, optionally nested	—	Always internal
`update_doc_page`	Title and body	—	Never over a colleague's unsaved draft

What every write goes through, in order (`App\Mcp\Tools\Write\WriteTool`):

1. the per-person write limit — `MCP_WRITE_RATE_LIMIT`, 30 a minute, on top of the endpoint's request limit;
2. the arguments, checked against the tool's schema by `AiToolInput` (unknown keys dropped, lengths capped) — the customer variant of a tool simply has no staff fields;
3. the target, resolved **as this person**: a ticket key through `TicketFinder`, a board through `BoardAccess`. A ticket on a board they are not on, an internal ticket a customer cannot see and a key that does not exist all get **the same answer**;
4. `AiCapabilityGuard::assertCanWriteExternally()`: the board's AI mode must allow writes (Observer means the AI writes nothing, from inside the product or out), archived boards are refused, and a customer may only use the three customer tools;
5. the change, through the ordinary actions and policies, inside `WriteOrigin` so the history says "via Claude";
6. one `ai_tool_invocations` row — category `action`, channel `mcp`, success or not;
7. an answer in the display style: ✅ and what actually changed, or ❌ and why nothing did. A database error's message is never repeated to the app.

Risk policy. Ordinary, reversible work happens on the call; three things wait for the person:

CHANGE	HAPPENS
Create, update, move, assign, label, checklist, internal note, internal page	At once (the app has already asked the person)
Delete a ticket	Only with a confirmation code
A staff comment the customer will read	Only with a confirmation code
Editing a published page	Only with a confirmation code

`App\Mcp\Support\Confirmations`: the first call changes nothing and answers with a preview and a six-character code. The code is bound to the person, the tool, the target and the exact arguments, works once and lasts ten minutes. It cannot prove the person read the preview — a model could answer its own question — which is why the client's own approval prompt is the other half of the gate.

Safeguards.

- **Retries never duplicate.** `App\Mcp\Support\RepeatGuard`: the same person, tool and arguments within 90 seconds get the first answer back, marked as a repeat, under a lock. Applies to the tools that add something.
- **Stale reads don't overwrite.** `update_ticket` and `update_doc_page` take `if_unchanged_since` — the "Last updated" the app read — and refuse if the record changed after it.
- **Attribution.** History rows (`ticket_events.payload.via`, `activity_log.properties.via`) carry the app's registered name, cleaned to 30 characters; the ticket history and activity feed show "· via Claude". The actor is always the person.
- **Kill switch.** `MCP_WRITES_ENABLED=false` takes every write tool away — back to read-only — without a deploy.

Not offered, deliberately: changing visibility or publishing, moving or deleting pages, deleting comments, boards, tabs, columns, labels and members, adding, renaming or reordering tabs, AI code runs, bulk edits of existing tickets, and file uploads.

Tabs over MCP — the `tab` argument

Boards can be split into tabs, each with its own columns (section 6). Other systems build on this argument — Gritch's admin AI passes it through — so it is a stable contract:

- **What it takes.** A tab's display name, exactly as `get_board` lists it. Matched exactly (ignoring case) first, then by a unique partial match; an unknown or ambiguous name changes nothing and the error lists the board's tabs. Only this board's tabs are candidates.
- **Always optional.** Without it a new ticket goes to the board's **first tab**, and `status` moves a ticket **within its own tab** — exactly as before tabs existed. `status` is always a column of the ticket's tab, or of `tab` when both are given.
- **Moving.** `update_ticket` with `tab` moves the ticket; without `status` it lands in the column with the same name, else a done column if it was done, else the tab's first column. A column name alone never carries a ticket into another tab — if only another tab has it, the error says which tab to name.

- **Output.** On a board with several tabs, `get_board` prints `TABS AND COLUMNS` (a `TAB "Name" — n tickets` block per tab), ticket lines read `🟡 Backend API › Doing`, ticket cards and `get_ticket` add a `Tab` line, and the first column of **each tab** counts as not started (🟣). A board with one tab prints exactly what it always did.
- Customers get `tab` on `create_ticket` (they see tab names) but never on `update_ticket`.

How results look in the chat

Asked for by the people using it: tickets shown the way a person scans them — status at a glance, who has it, a table when there are several — **in the chat, not in an artifact.** `App\Mcp\McpPresentation` holds all of it, for the MCP channel only; the in-app assistant's tool output is unchanged.

- **Server instructions** (sent once on connect, staff and customer variants): which tool answers which question; the rules for changes; and the display rules — lead with the answer, one ticket as a bold linked line plus bullets, three or more as a table *Ticket* | *Title* | *Status* | *Assigned to* | *Priority* | *Due*, "who is working on what" grouped per person, status and assignee always shown as separate facts, values exactly as returned, and no artifacts, canvases, documents, widgets or charts unless asked.
- **Badges in the tool output**, computed from the real data so a model never guesses:

	RULE
🟢	The column counts as done
🔴	The column's name says blocked or on hold
🟣	Review, QA, testing
🟣	The first column of its tab, or a backlog / to-do name
🟡	Anything else in between — the column's own name is always kept
🔥 ⬆️ ➖ ⬇️ 💤	Priority Critical, High, Medium, Low, Nice to have (shapes, so they never clash with status colours)
🐞 ✨ 📋 · 👤 · 🗓️ / ⚠️ · 🔒	Type; assignee or *Unassigned*; due or overdue; internal (staff only)

- **Links**: every ticket key is a Markdown link to the ticket.
- **A one-line `DISPLAY:` hint** ends each list and detail result, because not every client shows its model the server instructions.

Connecting Claude or ChatGPT

Checked October 2026; both vendors call custom connectors beta, so menus may move.

APP	STEPS
Claude (Free, Pro, Max)	Settings → Connectors → **Add custom connector** → URL `https://<host>/mcp` → Add → Connect → sign in to Nexora → **Allow**

Claude (Team, Enterprise)	An Owner adds the connector in Admin settings → Connectors; each member then clicks **Connect** under Settings → Connectors
ChatGPT (Plus, Pro, Business, Enterprise, Edu)	Settings → Apps & Connectors → Advanced settings → turn on **Developer mode** → create a connector with the URL → sign in → **Allow**. On Business and Enterprise a workspace admin may have to allow it first

Both apps ask before calling a tool that is not read-only. Claude lets each tool be set to *Always allow*, *Ask each time* or *Never*; leaving the write tools on **Ask each time** keeps a person in front of every change. ChatGPT shows the call's arguments for approval — read them.

10.7 Write tools: propose, then confirm

Nine action types (`App\Enums\AiActionType`), each exposed to the model as `propose_<name>` . The prefix is deliberate: the name itself tells the model that calling it does not perform the action.

ACTION	DESTRUCTIVE?	NOTES
`propose_create_ticket`	No	Sets every field at once — title, description, priority, type, assignee, tab, column, labels, and `customer_visible: false` to keep it internal
`propose_create_tickets`	No	Several new tickets on one board, up to 30 per call; one preview, all or none. `source_page_slug` names the page they come from
`propose_update_ticket`	No	Re-assign, move column or tab, re-prioritise, re-label, edit title or body
`propose_comment_ticket`	No	Staff comments default to the **internal** stream
`propose_create_doc_page`	No	New pages are **always internal** until somebody publishes them separately. `import_attachment_id` makes an attached file the body; `board` for a workspace conversation
`propose_update_doc_page`	No	Title and body
`propose_import_document`	No	A file attached to the conversation becomes a page **and** its open to-dos and bugs become tickets — one preview, all or none
`propose_code_run`	No	Proposes pressing the existing AI-run button; every refusal that pipeline already has still applies
`propose_delete_ticket`	**Yes**	Always confirmed, **by typing the ticket's key by hand**

What the schemas deliberately cannot express

> **IMPORTANT**
>
> **Visibility can only be narrowed, and only at creation.** The create schemas carry `customer_visible` , but a new ticket is never more visible than the board's `new_tickets_visible_to_customers` default, so nothing said in a conversation can publish work a board keeps internal. No update schema names visibility: publishing an existing ticket stays with the people and the screens that already authorize it. There is no field that can name a viewer, a user id, a column id or a label id.

Assignees, tabs, columns and labels are named **in words** — "Alex Round", "Backend API", "In Progress", "bug" — rather than by id. That is what the model has to work with, since it is repeating

what somebody said out loud; and a name has to be *resolved*, which is the step where `ProposalTargets` can refuse an assignee who is not on the board and ask which of two matching columns was meant. An id would skip that step and land the change somewhere plausible and wrong.

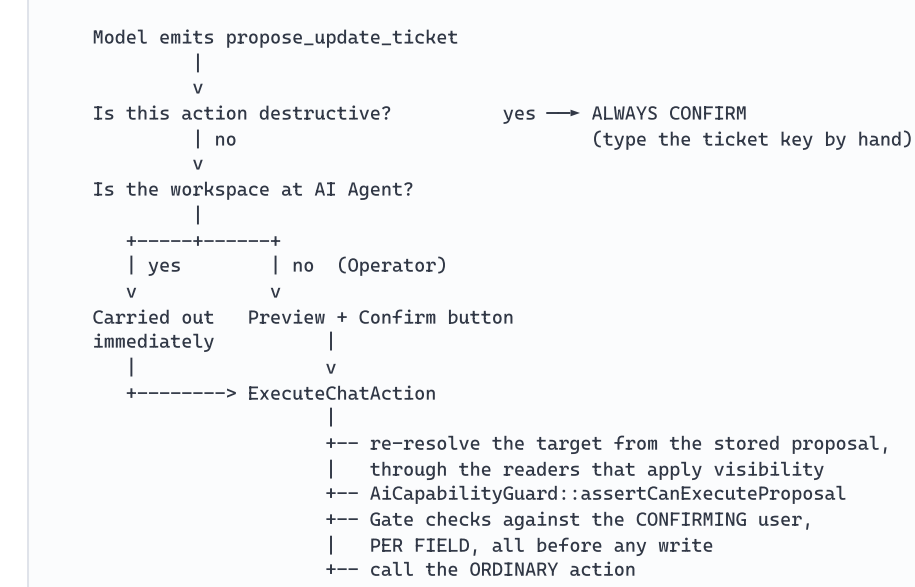
Tabs

On a board split into tabs the board context starts with `TABS AND COLUMNS` — each tab's columns with the asker's ticket counts — and every ticket line is tagged `[Tab › Column]`; a board with one tab gets a plain `COLUMNS (in board order)` line. The page hint carries the tab on screen (the board announces each switch to the panel), so CURRENT PAGE can say "...looking at the Backend API tab of the Gritch board"; the slug is re-resolved within the board like every other hint.

- `tab` on create: the ticket goes in that tab, and `column` is a column of it. **When the person names no tab, the tab on screen is filled in while the proposal is drafted** (`App\Services\AI\ProposalTabDefaults`) — only for the board on screen, only on a board with several tabs, and only if any column named is one of that tab's — so the card shows where the ticket will go. When a change runs without a tab it goes to the board's first tab; nothing at execution time looks at whatever page happens to be open.
- `tab` on update moves the ticket (`MoveTicketToTab`), with `column` then naming a column of the new tab. A column name alone is looked up **in the ticket's own tab only** (`ProposalTargets::column(board, tab, name)`); a column only another tab has is refused with "The Website tab has a "Design review" column — name that tab as well to use it."
- The assistant cannot create, rename, reorder or delete tabs, as it cannot change columns.

Confirmation

FIGURE



Five properties hold on both paths:

The proposal is not trusted, only read. Everything is re-resolved at the moment of execution — the ticket by its number *within this board*, the page by its slug *within this board* — through readers that

apply visibility. A proposal naming ticket 999 on another board resolves to nothing and is refused. The model's output is treated as a form submission from an untrusted client, which is exactly what it is.

Authorization is the real one. Gate checks run against the confirming user with the same abilities the ordinary screens use. A person who cannot create a ticket by hand cannot create one by confirming a proposal.

Authorization is per field, and happens first. A request to move a card *and* re-assign it asks `TicketPolicy::move` and `::assign`, and **both are taken before either write happens** — so a team member who may move but not assign changes nothing at all and is told why, rather than getting a moved card, a dropped assignee and a vague error.

The writes go through the ordinary actions — `CreateTicket`, `UpdateTicket`, `MoveTicket`, `SyncTicketLabels`, `PostComment`, `DeleteTicket`, `CreatePage`, `UpdatePage`. Every rule those own still applies, and there is no second, weaker path into the same tables.

One proposal per turn is kept — a preview showing two unrelated changes invites confirming both while reading one — **with two exceptions: new tickets, and a file imported with its tickets** (below). "Break this plan into tickets" is one request, so `propose_create_tickets` carries up to `AI_CHAT_MAX_TICKETS_PER_REPLY` (30) tickets in one call, and any ticket creations the model splits across several calls in a turn are merged into that one proposal for one board. Its preview lists every ticket; its button reads *Create N tickets*; under AI Agent it runs at once, like a single ticket.

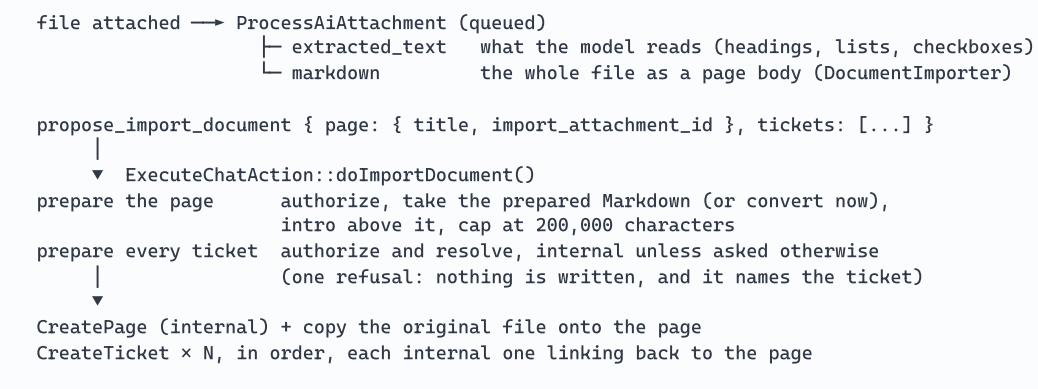
- **All or none.** Every ticket is authorized and resolved first, writing nothing; one refusal (an unknown column, an assignee not on the board) fails the batch with "Ticket 3 ("..."): ... Nothing was created." Then they are created in order, so the numbers follow the list.
- **Said, not hidden.** Tickets past the cap, or addressed to another board, are left out with a note on the card. A reply cut off by the output ceiling becomes a *failed* action saying so, never a Confirm button for nothing. Turns that may propose changes get `AI_CHAT_MAX_OUTPUT_TOKENS_WRITING` (16,000) so a long list fits.
- **Once only.** `ExecuteChatAction::handle()` takes a per-message cache lock and re-reads the row before checking its state, so two tabs or a double click cannot create a batch twice.
- **The model learns outcomes.** Each question's reference data ends with *EARLIER CHANGES IN THIS CONVERSATION* — what was created (keys and titles), discarded, refused or still waiting — so "continue with the rest" neither repeats nor skips tickets.

Visibility of new tickets. A ticket the assistant creates starts from the board's default (`new_tickets_visible_to_customers`, visible unless the board says otherwise) — the same default as the New ticket form. The proposal can narrow it (`customer_visible: false` for staff-only work) and never widen it. Changing an existing ticket's visibility is not available to the assistant. Tickets made from an imported file, or from an **internal** page on screen, start internal unless the person said customers should see them, and each internal one ends with a link back to its page.

Importing a file into the documentation

"Import this into the docs and make the tickets" is the second exception to one proposal per turn. The person attaches a document — PDF, Word, text, Markdown, CSV, Excel, PowerPoint — and the model calls `propose_import_document` once; a `propose_create_doc_page` plus ticket requests in the same reply are merged into the same proposal.

FIGURE

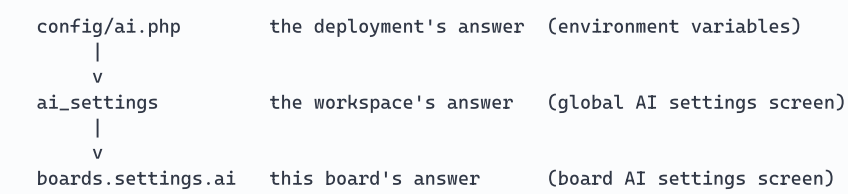


- **The model never retypes the file.** The body is the server's conversion of the stored file — the copy prepared when the file was read, so confirming never parses a large PDF inside somebody's request — and the prompt forbids putting the file into `body_md`, which would be cut off at the output ceiling and paraphrased where it was not.
- **Tickets for what is open.** The prompt asks for one ticket per unchecked to-do and per bug, not for items already ticked off; past 30 it says how many are left and continues with `propose_create_tickets`, reading *EARLIER CHANGES* — which names the page and its tickets.
- **The card** shows which file becomes the page and how many open to-dos it holds, then the ticket list; the button reads *Import page and create N tickets*.
- **From the documentation screen** a page's *Create tickets with AI* button opens the panel on that page's board, in Writing, with the request written in the composer; the page's text reaches the model as *THE PAGE ON SCREEN*. See section 8.

Re-checking the capability mode at confirmation time is not theoretical: a person can have a preview on screen when an administrator switches the workspace to Observer, and the confirm button is a separate request.

10.8 Configuration inheritance

FIGURE



Every field is resolved independently. A board that names a different *model* gets the workspace's provider, the workspace's key and the workspace's capability mode; a board that names a different *mode* keeps the workspace's model. Copying a configuration down a level would have been simpler to write and would have duplicated a secret in order to change a dropdown.

Two coercion rules run after the chain, because a stored setting outlives the thing it names:

- a model the effective provider does not serve is discarded in favour of that provider's default, so switching from Anthropic to OpenAI does not leave every board pointed at a Claude id;
- a provider the catalogue has no models for resolves to a null model, which surfaces as "not configured" on the screen rather than as a 404 from a vendor.

Credential precedence

`AiCredentialVault` is **the only class in the application that reads or writes a provider API key**. Three sources, most specific first:

1. The board's own key, in `boards.settings.ai.credentials` — the exception rather than the norm, for a project billed to its own account.
2. The workspace key, an `ai_credentials` row — what almost every deployment uses.
3. The environment, via config — still first-class, so a deployment that has always set `ANTHROPIC_API_KEY` keeps working with no database row at all.

Nothing above the vault ever holds a plaintext credential.

10.9 Providers and models

PROVIDER	ADAPTER	CREDENTIAL
Anthropic	`ClaudeService`, using `anthropic-ai/sdk`	`ANTHROPIC_API_KEY` or the vault
OpenAI	`OpenAiService`, plain HTTP to `/chat/completions`	`OPENAI_API_KEY` or the vault

The shipped catalogue:

MODEL ID	LABEL	PROVIDER	CONTEXT WINDOW	VISION	CATEGORY
`claude-opus-5`	Claude Opus 5	Anthropic	1,000,000	Yes	Deep reasoning
`claude-sonnet-5`	Claude Sonnet 5	Anthropic	1,000,000	Yes	Balanced
`claude-haiku-4-5`	Claude Haiku 4.5	Anthropic	200,000	Yes	Fast and cheap
`gpt-5.1`, `gpt-5-mini`	*(as configured)*	OpenAI	not stated	Opt-in	—

> **NOTE**
> The OpenAI block is deliberately thin: this deployment has no first-party source for those context-window figures, so it states none rather than guessing — a null context window renders as nothing in the picker, whereas a guess would render as fact. OpenAI model ids come from `AI_OPENAI_MODELS`, and vision support is **opt-in** via `AI_OPENAI_VISION_MODELS`, because "most current models take images" is not a capability.

Pricing lives in exactly one place — `config('ai.pricing')` — and `CostCalculationService` is the only reader. **A model that is not listed produces a null cost rather than a guess**, because an invented number in a cost report is worse than an honest blank.

10.10 Usage tracking and audit

TABLE	CONTENTS
`ai_usage_records`	One row per exchange: session, run, user, board, purpose (`chat` / `ticket_run`), provider, model, input and output tokens, cache read and write tokens, context size, whether the provider actually reported usage, estimated cost (cached tokens priced at their own rates), duration
`ai_tool_invocations`	One row per tool call: tool, category, target, input, success, outcome, result size, duration
`ai_sessions`	Rolled-up totals per conversation
`ai_runs`	Per-run tokens, cost, duration, branch, pull request URL, error

The `usage_reported` boolean is the honesty mechanism: a run whose token counts were never reported has **no** cost, and is reported as an unpriced run rather than folded in as zero.

10.11 Streaming

Answers stream token by token. Streaming and blocking share one `exchange()` method so the prompt, the tools, the proposal extraction, the usage record and the two writes are defined once — the only difference is which provider method is called.

The user's turn is stored **before** the provider is called, so a question that fails is still in the transcript: somebody scrolling back sees what they asked and that it went wrong, rather than a gap.

`AI_CHAT_STREAM_TIME_LIMIT` (360s) is applied with `set_time_limit()` only on the streaming path, so PHP's global `max_execution_time` can stay low for every other request.

10.12 Error handling

FAILURE	BEHAVIOUR
No credential	Refused with a recorded reason; the application still boots and every other feature works
Provider error	Reported verbatim where it is safe; `AiProviderException` is the only retryable class. In the chat, staff see the provider's reason and customers a neutral sentence; every failed question is logged as a warning (`Workspace AI could not answer a question.`)

Provider account out of credit	Anthropic's "credit balance is too low" (a 400) and OpenAI's `insufficient_quota` (a 429) are reported as **out of credit**, naming the account to top up — not as a malformed request or a rate limit. Nothing needs deploying afterwards: the next question works once credit is added
Tool failure	Contained, logged in full, reported to the model as a generic failure
External lookup unavailable	The assistant says out loud that it could not look something up, rather than guessing
Malformed chart or table JSON	Falls back to rendering the model's text as prose
Session over its token budget	The person is asked to start a new conversation
Stalled run	`ai:reap-stalled` recovers it every ten minutes

SECTION 11

AI Ticket Runs

A **run** is the AI working on one ticket, off the request cycle, on its own queue. It is a different feature from the assistant: the assistant is a conversation, a run is a job with a result.

11.1 The two modes

MODE	WHAT IT DOES	REQUIRES
Suggest	Reads the ticket and the repository metadata, writes an opinion as an **internal note** on the ticket	AI Operator or above
Apply	Works in an isolated clone, on a new branch, runs the configured tests, and opens a **draft pull request**	**AI Agent**, a GitHub token, an attached repository, and a code-generation driver

`Off` is part of the same enum rather than a separate boolean, so the code that reads a board's automatic behaviour cannot forget to check an "enabled" flag as well.

11.2 Triggers

TRIGGER	WHO STARTS IT	CAP	BYPASS
Manual	A staff member presses a button on the ticket	100/day (`AI_DAILY_MANUAL_RUN_CAP`)	Administrators may exceed it if `AI_ADMINS_BYPASS_CAP=true`
Automatic	A customer's ticket is created, if the board opts in	20/day per board (`AI_DAILY_AUTO_RUN_CAP`), board-configurable up to 500	**Nobody. Ever.**

> **IMPORTANT**
>
> The asymmetry is the whole design of `AiRunCap`. Nobody chose the automatic runs — a customer filing twenty tickets in an afternoon starts twenty runs, and on apply mode that is twenty clones and twenty pull requests. **There is no bypass for the automatic cap because there is nobody to grant one to.** A manual run is a deliberate human act by somebody who knows what it costs, so its cap is a runaway-loop guard rather than a budget.

"Today" is the **board's own day**: a board configured for `Europe/Stockholm` rolls over at Stockholm midnight, because that is when the people looking at the number think the day changed. Queued, running, completed and failed runs all count; **cancelled runs do not**, since they never started and holding them against the day would punish somebody for changing their mind.

11.3 Run lifecycle

FIGURE

```
STATUS       queued ──→ running ──→ completed
                │          │    └──→ failed
                └──────────┴───────→ cancelled  (only while still queued)

STAGE        queued → preparing → analysing → coding → testing → pull_request → finished
                                                                              └→ failed

SUGGEST MODE                          APPLY MODE
------------                          ----------
queued                                queued
preparing   (resolve repository)      preparing     check the coding runtime exists
analysing   (provider call)           preparing     clone into the run's own directory
finished    -> internal note          preparing     REFUSE if the base branch is protected
                                      preparing     create a new branch off it
                                      coding        the runtime edits files
                                      coding        re-derive the diff from `git status`
                                      testing       run AI_VALIDATION_COMMANDS
                                      pull_request  commit, push the NEW branch only
                                      pull_request  open a DRAFT pull request
                                      finished      -> internal note with the PR link
```

11.4 Apply mode invariants

Three properties are **structural rather than instructions** — they hold because the code that could break them does not exist.

INVARIANT	HOW IT IS GUARANTEED
Never pushes to main	The push is always `refs/heads/<new branch>`, and the branch name is generated from the ticket key and the run's UUID. The base branch is only ever read. `assertNotProtected()` refuses outright if a board names a protected branch
Never merges	`PullRequestClient` has **no merge method**. The capability is absent from the codebase
Never fakes success	An empty diff, a failed validation command or an unavailable runtime all raise. The run is recorded as failed and an internal note names the step that stopped it. **No pull request is opened for work that did not happen**

Protected branches, whatever a board is configured to say: `main`, `master`, `production`, `release`, `develop`.

The failure order matters: **validation runs before the push**, so a change that does not build never becomes a branch on the remote.

> **NOTE**
>
> **Apply mode is Partially Implemented as shipped.** `AI_CODE_DRIVER` defaults to `unavailable`, and with that setting an apply run **fails immediately** with an internal note naming exactly what is missing. Nothing is faked and no empty pull request is opened. The working driver is `claude_code`, which runs the Claude Code CLI inside the isolated checkout and requires the binary (installed in the Docker image) plus `ANTHROPIC_API_KEY` in the worker's environment.
>
> Repository **cloning** is also opt-in: `AI_REPOSITORY_CLONE_ENABLED` defaults to `false`. With it off, suggest mode still runs and reasons from the configured repository metadata and the ticket alone — and the run **records that the working tree was unavailable** rather than pretending it read the code.

11.5 Isolation and cleanup

Every run that touches a repository gets its own directory, named after the run's UUID, under `storage/app/ai-runs` (`AI_WORKSPACE_PATH`). It is deleted when the run ends, however it ends, in a `finally` block.

Nothing durable lives there: the note, the pull request URL, the token counts and the failure reason are all rows before the directory is deleted. Railway containers are ephemeral anyway.

11.6 Duplicate execution

Guarded twice, because the two guards fail differently:

GUARD	MECHANISM	STRENGTH
`WithoutOverlapping`	A cache lock keyed on the run id	Stops two workers starting the same run at the same moment — but depends on the cache being shared and alive
The status claim	`claim()` flips `queued → running` in a single conditional `UPDATE` and checks the affected row count	**Authoritative**, because it is the database rather than a lock. A second worker that gets past the lock finds zero rows and returns

11.7 Retries

Only transient provider failures are worth retrying, and they are the ones that raise `AiProviderException`. A missing credential, an unavailable coding runtime, a protected branch or a failed test suite will fail identically three times, so those are marked failed on the first attempt and the job stops. That distinction keeps a misconfiguration from producing three identical internal notes twenty minutes apart.

SETTING	DEFAULT	VARIABLE
Attempts	3	`AI_JOB_TRIES`
Timeout	900s	`AI_JOB_TIMEOUT`

Backoff	30s, then 120s	`config('ai.queue.backoff')`
Queue	`ai`	`AI_QUEUE`

11.8 Stalled runs

A run is owned by the job that claimed it, so if that job disappears — a deploy, a container restart, an OOM — nothing can finish the run and the ticket shows "Running" for ever, blocking any further run on it. Since an apply run takes minutes and **every deploy restarts the worker**, this is ordinary rather than exceptional.

`ai:reap-stalled` runs **every ten minutes** and recovers them. It refuses to touch a run that still has a job on the queue.

11.9 Visibility

> **IMPORTANT**
>
> **A customer has no relationship with AI runs at all.** Not "cannot start one" — cannot see that they exist, cannot see a status, cannot see a cost, cannot see a failure. Every ability on `AiRunPolicy` denies a customer as **404**, because a 403 on "AI runs for AQD-42" would confirm the feature is being used on their ticket.
>
> Enforced three times over: `AiRun::visibleTo()` refuses customers in SQL; the policy refuses for a single loaded run and for "may I start one?"; and the panel renders nothing — which is a usability consequence of the rule, never the rule itself.

11.10 Results

OUTCOME	WHAT IS WRITTEN
Completed (suggest)	An **internal note** on the ticket with the analysis, linked as `result_comment_id`
Completed (apply)	An internal note with the branch name and the **draft pull request URL**
Failed	An internal note naming the step that stopped it; `error_message` on the run
Skipped	A `ai_run_skipped` ticket event, e.g. the cap was reached or the capability mode forbids it

Every outcome also writes a ticket event, appears in the unified timeline, and can raise a Slack message if the board subscribes to `ai_run_finished` (off by default).

Refusals are written as prose a member of staff can act on — *"The AI is set to AI Observer, which reads and answers but writes nothing — not even an internal note. Raise the mode to AI Operator to let runs post their analysis."* — rather than as a code, because the text ends up in an internal note and on a button tooltip.

SECTION 12

AI Visualisation, Voice and Attachments

12.1 AI-generated charts, tables and KPI rows

The assistant can answer with structured blocks rather than only prose. An answer is a list of blocks in the order the model wrote them, so a response can read "throughput improved by 18% in Q3", then a chart, then the explanation.

How a block gets there

Two routes:

1. A **fenced block** with a known name and JSON inside — a fence opened with `nexora-chart`, `nexora-table` or `nexora-kpi`, containing a JSON object such as:

```
{"type": "bar", "title": "Tickets by month", "labels": ["Jul", "Aug", "Sep"],
 "datasets": [{"label": "Raised", "values": [12, 19, 14]}]}
```

2. For tables only, an **ordinary Markdown pipe table**, which is promoted automatically. That second route matters more than it looks: a model asked for a table writes a pipe table whatever the prompt says, at least some of the time.

Supported chart types

`line`, `bar`, `hbar`, `area`, `pie`, `donut`, `scatter`.

`hbar` exists as its own type rather than a flag because it is the right answer often enough to be asked for by name: any breakdown whose labels are people's names, board names or label names reads far better with the text beside the bar than rotated underneath it.

BOUND	VALUE
Points per series	400
Series per chart	8
Slices in a pie or donut	12
Rows in a breakdown from `get_statistics`	30
Weeks in a trend series	60

The security claim

> **IMPORTANT**
>
> **The model never supplies anything that is executed or rendered directly.** It supplies numbers and labels. Those are validated into a `ChartSpec`, `TableSpec` or `KpiSpec` — a type, a list of labels, and a list of series of floats — and the renderer draws an **SVG server-side** from *that*.
>
> Consequently: there is no charting library receiving model-authored configuration, so there is no option in such a library that turns a string into a callback; nothing model-authored reaches a `<script>`, an event handler, a style attribute or a URL; colours come from a fixed palette chosen by index, never supplied; and a "chart" cannot be a vector of attack the way an uploaded SVG would be.

Validation is strict about numbers and forgiving about everything else. A non-numeric value becomes a gap at that point rather than discarding the dataset; an unrecognised chart type falls back to a bar chart. **Failure is always prose**: a fence whose JSON does not parse, a chart with no numbers or a table with no columns all fall back to rendering the block as the text the model wrote.

Data integrity

The `get_statistics` tool's description instructs the model to **always call it before emitting a chart about the workspace**, and states that figures must never be estimated or invented. The numbers it returns are already counted under the asker's own scope, so they are correct to report to them.

> **NOTE**
>
> Access control for charts is not a chart feature. It is the ordinary statistics scope: a customer asking for a breakdown gets counts of the tickets **they can open**, and the three staff-only metrics are refused with a message naming what they can have instead. The dedicated test `tests/Security/AiVisualizationSecurityTest.php` asserts that a customer's totals exclude internal tickets and that the hidden count cannot be recovered by subtraction.

Export

FORMAT	HOW	ROUTE
CSV	The server re-reads the stored answer and re-parses it with **the same parser the screen used**, then exports block *N*	`GET /ai/messages/{message}/blocks/{block}/export.csv`
PNG	Produced **in the browser** from the SVG already on the page (`resources/js/chart-export.js`)	No server route

The CSV route addresses a **block index** rather than accepting a payload. A browser that could POST the rows it is showing would be a browser that could export rows the model never produced — exactly the wrong property for a file somebody forwards to a client. The only way the CSV and the on-screen table could disagree is if the same parser gave two answers for the same text.

Authorization on that route is four filters deep: the route group's `role:admin,team`, `visibleTo` (board), `ownedBy` (**an administrator cannot export a colleague's conversation**), and a non-null session id. A miss is a 404.

PNG export has one hard part worth noting: a chart on the page is coloured by Tailwind classes resolved by the page's stylesheet, so serialising the node alone would lose every colour. The clone is walked and the handful of properties that actually draw anything are copied from `getComputedStyle` onto the elements as presentation attributes.

12.2 Voice

Spoken conversation is **three steps, not a fourth kind of session**: speech-to-text, the ordinary assistant, text-to-speech. That is what lets a spoken question inherit sessions, attachments, capability modes, the audit trail and the customer boundary without any of them being taught about voice.

ENDPOINT	METHOD	PURPOSE
`/ai/voice/listen`	POST	A recording in, the words out
`/ai/messages/{message}/speech`	GET	A stored answer out, as audio

SETTING	DEFAULT	VARIABLE
Enabled	`true`	`AI_VOICE_ENABLED`
Driver	`openai`	`AI_VOICE_DRIVER`
Transcription model	`whisper-1`	`AI_VOICE_TRANSCRIPTION_MODEL`
Speech model	`gpt-4o-mini-tts`	`AI_VOICE_SPEECH_MODEL`
Voices offered	`alloy,echo,fable,onyx,nova,shimmer`	`AI_VOICE_VOICES`
Default voice	`alloy`	`AI_VOICE_DEFAULT`
Timeout	120s	`AI_VOICE_TIMEOUT`

No credential is configured for voice. The provider reads the workspace's existing OpenAI key through `AiCredentialVault`. A deployment that has already configured OpenAI has voice; one that has not sees the control disabled with a sentence naming the remedy, and nothing pretends to listen.

The voice list is an **allow-list, not a suggestion**: the chosen value is interpolated into a vendor request and comes from a browser, so anything not on the list becomes the default.

Authorization differs between the two endpoints, deliberately:

- `listen` is allowed to **anybody the assistant panel is allowed to, customers included** — transcribing reads nothing from the workspace, so the only thing to establish is that this person may be talking to the assistant at all. A customer's assistant is read-only, not silent.
- `speak` is stricter, because it reads a stored answer: the turn is resolved through `visibleTo` **and** `ownedBy`, so it cannot reach a colleague's or an administrator's conversation, and only an assistant

turn can be spoken.

> **NOTE**
>
> Generated audio is **bytes, never a file**. A spoken answer is as private as the answer, so the controller streams the bytes in the response to the person's own authorized request and leaves nothing behind for a signed URL to leak or a retention policy to forget. There is deliberately **no cache**, so the same question asked twice costs the synthesis twice.
>
> `speak` is addressed **by message id rather than by posting text**: what is read aloud is what is stored, and a browser that could post the words would be a browser that could make the assistant say something it never said.

12.3 AI conversation attachments

Files can be attached to an assistant conversation and are read into the prompt.

KIND	EXTENSIONS	PROCESSOR
`text`	`txt` , `log` , `json` , `yml` , `yaml`	`TextProcessor`
`markdown`	`md` , `markdown`	`MarkdownProcessor`
`csv`	`csv` , `tsv`	`CsvProcessor` + `CsvAnalyser`
`pdf`	`pdf`	`PdfProcessor` (`smalot/pdfparser`)
`image`	`png` , `jpg` , `jpeg` , `gif` , `webp`	`ImageProcessor` (GD)
`audio`	`mp3` , `m4a` , `wav` , `ogg` , `webm` , `mp4`	`AudioProcessor` + `AudioTranscriber`
`document`	`docx` , `xlsx` , `pptx`	`OfficeProcessor`

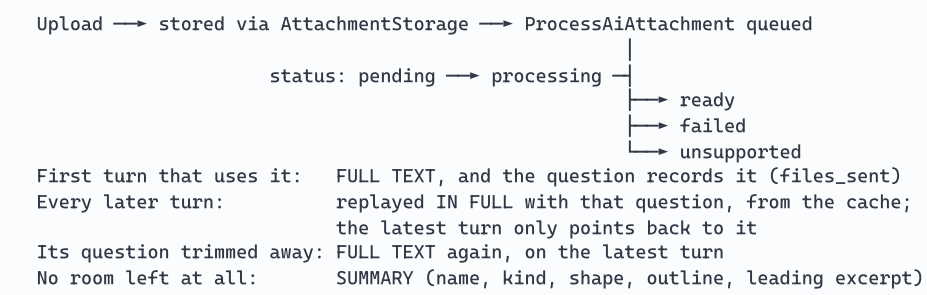
Limits are deliberately **their own**, not a reuse of `config('attachments')`: a file attached to a ticket is stored and served, whereas a file attached to the assistant is stored, *read*, and put in front of a language model — a wider blast radius, so it gets a narrower allow-list (no archives, no macro-enabled Office formats) and a different ceiling.

LIMIT	DEFAULT
Per file	20 MB
Per conversation	10 files
Extracted text per file	200,000 characters
Extracted text per request	400,000 characters, and never more than half the model's window
Summary length, for a file there is no room for	1,200 characters
CSV rows quoted in the prompt	300 (of 40 columns max)
Image longest side	1,568 px, JPEG quality 82

Both the **extension allow-list and the detected MIME type** must pass. A file called `notes.txt` whose bytes are a PDF is refused, and so is a PDF renamed to `.png`.

How files reach the model

FIGURE



The same job also keeps the file as a documentation page would hold it — the whole file as Markdown, in `ai_attachments.markdown` — so the assistant can import it into the docs without parsing it again (section 9). Word documents reach the model as Markdown too, with their headings, lists, checkboxes and tables, through the same converters the documentation import uses.

Three decisions drive most of the cost and accuracy of an answer about a document:

A file stays with the message it came in on. It is replayed with that question for as long as the question is in the model's view, so the tenth follow-up reads the whole document just as the first did — and from the provider's cache, at a tenth of the price. This replaced "full text once, a digest thereafter", which saved money and cost accuracy: by the second question a long to-do list was down to an excerpt, and "continue" could not finish it.

A budget, spent newest first. When it runs out, the remaining files still appear as summaries rather than vanishing: a model that is not told a file exists will answer as though it does not, which is worse than one that knows and says it has not read it. **Every truncation is stated in the prompt**, because a model handed half a contract and not told will summarise it as though it read the end.

Pictures go only to models that can see. An image is sent as a picture only when the session's model declares vision in the catalogue. Otherwise it is named and the prompt says the model cannot see it, so the answer is "I cannot see that image" rather than a confident invention.

CSV handling is worth calling out: rows are sent as data **and the profile — column types, ranges, outliers — is computed from *all* of them**, so a question about an average is answered from every row even when only the first 300 are quoted. The prompt says which is which, because a model that thinks it has seen every row will state a total it cannot know.

Images are **re-encoded rather than forwarded**. The stored bytes are whatever somebody uploaded, and handing them to a provider verbatim would make the provider's decoder our attack surface. GD reads the file, resizes it and writes a fresh JPEG or PNG — so what leaves the application is a file the application generated, with EXIF (routinely including GPS coordinates) stripped.

Audio transcription uses the workspace's existing OpenAI credential. With no key configured an audio upload is **stored and its card says transcription is not configured and who can configure it** — it

does not fail, and it does not silently do nothing.

> **NOTE**
>
> An instruction *inside* an uploaded document is framed to the model as **text somebody wrote, never a command**. Attachments enter the prompt in the same reference-data block as the board context, under the same framing.

`markSent()` and `files_sent` are written only **after** the provider has actually answered, so a question that failed is not remembered as having carried a document the model never read.

SECTION 13

GitHub Integration

Two features that look unrelated but share one credential:

DIRECTION	WHAT IT DOES	CLASS
Inbound	GitHub tells Nexora about branches, commits, pull requests and CI that mention a ticket key	`GitHubWebhookController`, `WebhookProcessor`
Outbound	Apply mode pushes a branch and opens a draft pull request	`PullRequestClient`

Both are off until credentials exist, and both fail loudly rather than pretending: with no token the pull-request client reports itself unconfigured, and with no webhook secret the endpoint rejects every delivery rather than trusting unsigned input.

13.1 Inbound: how a reference becomes a link

FIGURE



The reference pattern

`/\b([A-Za-z][A-Za-z0-9]{1,5})-(\d{1,9})\b/` — a board prefix, a hyphen, a number.

Case-**insensitive** here, unlike in prose, because branch names are typed by hand at a terminal and `aqd-42-fix` is what people actually write.

IMPORTANT

The security boundary is the repository, not the pattern. A webhook is an anonymous request whose only credential proves GitHub sent it — it does **not** prove who owns the repository it is about. Anybody can add a webhook to a repository they control and point it here, and with the secret they could deliver a valid signed payload claiming a commit in `attacker/anything` fixes `AQD-1`.

So a delivery may only reach tickets on boards where an administrator has **already attached that repository**. The repository name in the payload is matched against `board_repositories`, and the resulting board ids are the *only* boards in scope. A repository nobody has configured resolves to nothing and the delivery is recorded as ignored.

Note what is deliberately absent: any `visibleTo()` scoping. There is no viewer — a webhook is a system actor with no user — so the ordinary visibility scopes have nothing to apply. The board restriction replaces them, and it is **narrower**: not "boards somebody can see" but "boards this specific repository is wired into".

Events understood

EVENT	WHAT IS LINKED	DETAIL
`push`	Branch and commits	The **branch name is checked as well as each commit message**, because the convention most teams use is to put the key in the branch once rather than in every commit
`create`	Branch	Only when `ref_type` is `branch`; tags have no place on a ticket
`pull_request`	Pull request	References taken from the title, the body and the head branch, in that order of likelihood. Tracks opened, edited, closed, merged, ready-for-review
`check_suite`	CI status on an existing link	—
`status`	CI status on an existing link	—

Anything else is acknowledged with a 202 and ignored. GitHub retries on a non-2xx, and retrying a delivery we will never act on helps nobody.

Link types and state

`GITHUB_LINKS` COLUMN	VALUES
`type`	`branch`, `commit`, `pull_request`
`state`	`open`, `draft`, `merged`, `closed`
`ci_status`	The reported conclusion
Also stored	`reference`, `title`, `url`, `author_login`, `merged_at`, `metadata`

Three link types and deliberately no more. A ticket's engineering trail is "somebody started a branch, commits landed on it, a pull request was opened" — anything further (reviews, comments, deployments) belongs in GitHub, and mirroring it here would make the panel a worse copy of a better tool.

Idempotence and de-duplication

Two independent mechanisms, because GitHub redelivers on timeout and replays on demand:

LEVEL	MECHANISM
Delivery	A unique index on `(source, delivery_id)`. **The insert is the claim** — atomic across web containers, which a "select then insert" would not be. A duplicate key is the expected outcome for a replay, not an error
Link	A unique index on `(ticket, repository, type, external_id)`, so a replay updates the row it wrote the first time

Ticket events are written **only when something actually changed** — a row that was just created, or whose state or CI conclusion actually moved. GitHub sends a `check_suite` for every push; an event per delivery would bury the ticket's own history within a day.

Bounds

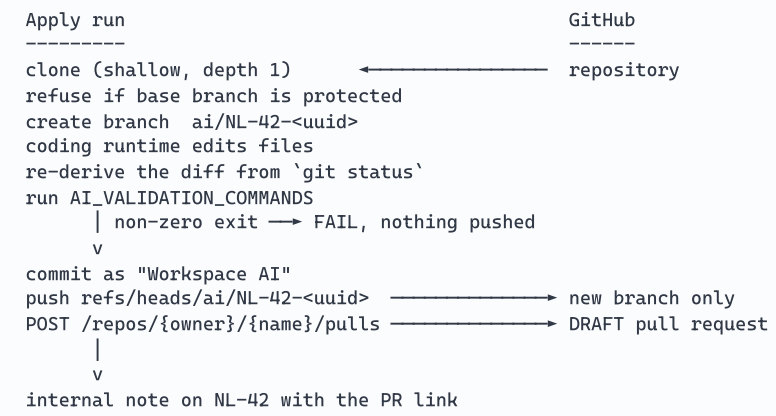
BOUND	DEFAULT	VARIABLE
Ticket references per delivery	50	`GITHUB_MAX_REFERENCES`
Commits linked per push	20	`GITHUB_MAX_COMMITS`
Delivery-record retention	30 days	`GITHUB_WEBHOOK_RETENTION_DAYS`
Requests per minute	300	`GITHUB_WEBHOOK_RATE_LIMIT`
Job attempts / timeout	3 / 60s	`GITHUB_JOB_TRIES`, `GITHUB_JOB_TIMEOUT`

A branch merge can carry hundreds of commits each mentioning a key; the cap stops one delivery writing an unbounded number of rows.

13.2 Outbound: opening a pull request

Used only by apply mode. See section 11 for the full run pipeline.

FIGURE



`draft: true` by default, so a pull request arrives **visibly unfinished** rather than looking like a colleague's finished work waiting on a rubber stamp.

SETTING	DEFAULT	VARIABLE
Branch prefix	`ai/`	`AI_BRANCH_PREFIX`
Commit author	`Workspace AI <ai@example.invalid>`	`AI_COMMIT_AUTHOR_NAME`, `AI_COMMIT_AUTHOR_EMAIL`
Clone depth	1	`AI_CLONE_DEPTH`
Git timeout	300s	`AI_GIT_TIMEOUT`
Validation commands	none	`AI_VALIDATION_COMMANDS` (pipe-separated)
Validation timeout	900s	`AI_VALIDATION_TIMEOUT`
Protected branches	`main`, `master`, `production`, `release`, `develop`	not overridable

13.3 Security model

CONTROL	IMPLEMENTATION
Inbound authentication	HMAC-SHA256 over the raw body, constant-time comparison, SHA-256 only, fail-closed with no secret
Inbound authorization	Repository must be attached to a board. No other board is reachable
Inbound rate limiting	300/min per IP, applied **before** signature verification touches the database
CSRF	Exempt for this one path only; replaced by the signature, which is a stronger claim

Outbound token scope	`contents: write` and `pull_requests: write` only. **Never merge rights**
Merge capability	**Absent from the codebase** — `PullRequestClient` has no merge method
Token exposure	Read from config at call time, never stored on the object, never logged, never interpolated into an error message — failure text is rebuilt from the HTTP status
Customer visibility	GitHub links are internal. `GithubLinkReader` refuses customers; `get_code_activity` and `get_github_repository` are withheld from them
Timeouts	20s for API calls (`GITHUB_REQUEST_TIMEOUT`), 60s for the webhook job

SECTION 14

Slack Integration

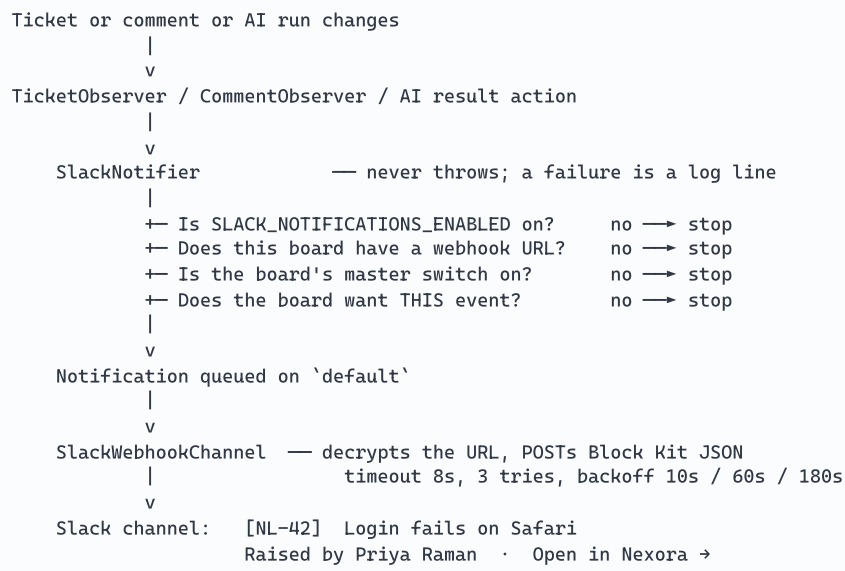
14.1 Design

Slack is configured **per board rather than per workspace**, because that is how the product is used: each customer's board belongs to a different delivery team with a different channel, and a single workspace-wide webhook would put every customer's tickets in one room.

The webhook URL therefore lives on the **board**, encrypted. What lives in `config/slack.php` is the behaviour that must not differ per board: whether the integration is available at all, how long to wait, and how hard to retry.

14.2 Flow

FIGURE



14.3 Events

EVENT	DEFAULT	FIRES WHEN
`customer_ticket_raised`	**on**	Somebody outside the delivery team files a new ticket
`customer_comment_posted`	**on**	A customer replies in the customer conversation. **Internal notes are never announced**
`ticket_moved_to_done`	**on**	A ticket reaches a column marked `is_done`

`ai_run_finished`	**off**	An AI run completes or fails. The analysis itself stays in the internal note

A board that has never opened the integrations screen does **nothing** — the board-level default for `enabled` is `false`. A board with no webhook URL cannot post anyway, and defaulting the events to "on" would mean the first team that pastes a URL immediately gets every event rather than choosing.

Muting is available per event, per board, and for the whole board with one switch.

> **NOTE**
> `critical_ticket_raised` is deliberately **not** a Slack event. An SMS is an interruption chosen deliberately for the few things worth waking somebody for, and offering it as "just another Slack event" would blur that. A critical ticket still produces the ordinary ticket-raised message in Slack.

14.4 What a message contains, and why it is thin

> **IMPORTANT**
> A Slack channel is **outside this application's authorization model**. Whoever is in the room reads the message, and Nexora has no idea which of them is a board member, which is a customer and which is a contractor added last week.
>
> So a message carries an **identifier, a title, who did it, and a link** — never a ticket description, a comment body or an AI summary. **The link is the access control**: following it requires logging in, and the ordinary visibility rules then apply to whatever they see.

The `text` field is not an ignored fallback: it is what appears in the desktop notification and the channel-list preview, and a Block Kit message without it shows up as "This content can't be displayed".

14.5 Configuration and security

CONTROL	VALUE
Master switch	`SLACK_NOTIFICATIONS_ENABLED`, default `true`
Per-board switch	`boards.settings.slack.enabled`, default `false`
Webhook URL storage	**Encrypted** with `APP_KEY` in `boards.settings.slack`
Webhook URL in the UI	**Write-only** — blank on every load; blank means "leave as is"; removal is a separate explicit button
Delivery	Always queued on `default`
Timeout / tries / backoff	8s / 3 / 10s, 60s, 180s
Channel name	`slack_webhook`, deliberately not `slack` — that name is reserved for `laravel/slack-notification-channel`, and a collision would make it impossible to add later without silently rerouting every existing notification

The master switch is useful for a staging deployment restored from a production database dump, which would otherwise start posting into the real team's channel within minutes of booting.

Rotating `APP_KEY` makes stored webhook URLs unreadable. That is the correct failure, and it degrades to "not configured" rather than throwing — a board with an unreadable URL simply stops posting instead of breaking every ticket save.

SECTION 15

SMS Alerts

15.1 Design

An SMS is the most intrusive notification this application can send: it reaches a person who is not at a computer, possibly asleep, and it costs money per message. So it has **exactly one trigger — a critical ticket being raised — and it is off until somebody deliberately turns it on**.

`SMS_ENABLED` defaults to `false`.

15.2 Providers

DRIVER	BEHAVIOUR
`unavailable` *(default)*	Every send is refused with a message naming exactly what to configure, and recorded as **failed**. No message is claimed to have been sent
`log`	Writes to the application log and records status `logged`, **never** `sent` — a dashboard counting these as delivered would be lying about whether an on-call engineer was reached
`46elks`	The real provider. Needs `SMS_46ELKS_USERNAME`, `SMS_46ELKS_PASSWORD` and a sender in `SMS_FROM`

An unrecognised value resolves to `unavailable`, so a typo in a deployment variable cannot silently disable alerting while appearing to work.

15.3 Recipients

Workspace-wide fallback numbers in E.164 form (`SMS_ALERT_RECIPIENTS`, comma-separated). A board may override these with its own on-call list on the integrations screen.

Numbers rather than users, on purpose: **on-call is a rota, not a role**, and the person carrying the phone this week is often not the person whose account raised the board.

A hard ceiling of `SMS_MAX_RECIPIENTS` (5) applies whatever a board is configured with — an alert that texts thirty people is a phone tree, not an alert, and also thirty times the cost of a mistake.

15.4 Duplicate suppression

GUARD	MECHANISM
Per ticket, per number	A **unique index** on `sms_messages(event, ticket_id, recipient)` — the guarantee has to survive a cache flush and a race between two workers, and only the database can promise that

Per board	A cooldown window, `SMS_BOARD_COOLDOWN` (300s). If a script files forty critical tickets in a minute, the on-call phone rings once, not forty times

15.5 Data protection

Each `sms_messages` row holds a **phone number**, which is personal data, so the table has a retention window rather than growing for ever. `SMS_RETENTION_DAYS` defaults to 90, enforced by the daily `workspace:prune` command.

Ninety days keeps the evidence useful for the question it exists to answer — *"was the on-call rota reached about that incident?"* — without keeping it indefinitely.

On the integrations screen the rota field is **readable** (somebody has to be able to remove a colleague's number when they leave the rota), but the delivery **history** below it is masked — it is a growing log that accumulates numbers no longer on the rota, scanned to answer "was anybody reached?", a question that does not need the digits.

15.6 Other controls

CONTROL	VALUE
Message length	Truncated at `SMS_MAX_LENGTH` (300) — an SMS is billed per 160-character segment
Sender	`SMS_FROM`, alphanumeric (max 11 chars) or E.164. Alphanumeric senders cannot receive replies, which is right for an alert
Delivery	Queued on `default`, 3 tries, backoff 15s / 60s / 180s
Timeout	15s
Delivery receipts	`SMS_46ELKS_DLR_URL` may be set, but **this application does not expose a receiving endpoint** — *Planned / Not Yet Implemented*
Credential exposure	Rebuilt failure text from the HTTP status rather than the response body, which on an auth failure can echo the username back

Like the AI trigger, `AlertCriticalTicket` **never throws.** It is called from the ticket observer inside the transaction that creates the ticket, and a critical ticket that exists and did not page anybody is far better than a critical ticket that failed to be filed.

SECTION 16

Statistics and Reporting

16.1 Two screens, not one screen with a role switch

SCREEN	ROUTE	GATE	SERVICE
Team statistics	`/stats`	`role:admin,team`	`TeamStatistics`, `FlowMetrics`, `AiStatistics`
Customer summary	`/stats/customer`	auth — **everyone**	`CustomerStatistics` only

> **IMPORTANT**
>
> The separation **is** the safety property. A shared class with a `$forCustomer` flag would mean every future metric somebody adds is exposed by default and has to be remembered *not* to be — the wrong way round. Here the customer screen can only render what `CustomerStatistics` offers, and that class offers six things. **There is no branch on either page that a careless edit could invert.**
>
> What `CustomerStatistics` deliberately *cannot* answer, because the methods do not exist: internal ticket counts, per-assignee workload, cycle time, time in column, AI runs, AI cost, token usage, the internal/visible split, or anything at all about work the customer is not shown.

A member of staff may open the customer screen too, and should — it is the only way to see what a customer actually sees. When they do they see **their own** visibility, not a simulated customer's, and the screen says so.

16.2 Metrics

Present-state metrics — `TeamStatistics`

Everything here is a question about *now*, read from `tickets`.

METRIC	OUTPUT
`counts`	Total, open, closed, created in the period
`byColumn`	Tickets per status column, in board order — tab by tab, labelled `Tab › Column`, on a board split into tabs
`byPriority`	Tickets per priority, most urgent first, including zeros
`byAssignee`	Open tickets per assignee, busiest first — **staff only**
`byLabel`	Tickets per label, most used first — **staff only**
`createdByWeek`	Tickets raised per week
`visibilitySplit`	Shared with the customer versus internal — **staff only**

`recentTickets`, `labelCount`	Supporting detail

Each breakdown is **one grouped query**. A per-assignee table built by counting tickets per member in PHP would be one query per person on a page that already runs a dozen.

Statistics are **board-wide**: a board split into tabs is still counted whole, with no tab filter. Where a column is named — `byColumn`, time in column, the CSV exports — it reads `Tab › Column` on a board with several tabs (two tabs can both have a "To Do"), and exactly as before on a board with one.

Flow metrics — `FlowMetrics`

Read from `ticket_events`, the append-only history — **never from** `tickets.board_column_id`.

> **NOTE**
>
> The distinction is what makes this class worth having. *"Twelve tickets are in Done"* is **not throughput**: those twelve might have arrived over two years, and the eight finished last month and then reopened are invisible. Flow is a property of **transitions**.
>
> The history was designed for this from the first migration — `ticket_events` carries `board_id`, `to_column_id` and `from_column_id` as indexed columns precisely so that "which tickets entered a done column on this board last week" is one index range scan.

METRIC	OUTPUT
`closures`	Tickets that entered a done column in the period — see below for deleted columns and tidying
`throughputByWeek`, `throughputPerWeek`	Completion rate over time
`cycleTime`, `cycleTimeTrend`	How long work takes, and whether that is moving
`timeInColumn`	Where work waits
`wasTruncated`	Whether the fold hit its bound

Two limitations are **surfaced to the reader rather than hidden**:

- a ticket that has never left its column contributes no interval to the time-in-column figures. Only completed stays are measured, because an open-ended one has no length yet and counting "so far" would make a healthy board look worse the longer it stays healthy;
- the fold is bounded. A range wide enough to exceed it **reports that it was truncated** instead of quietly averaging a subset.

Restructuring a board is not finishing work. Deleting a column or a tab moves its tickets (reason `column_deleted` / `tab_deleted` in the event); those moves never count as closures, so done tickets carried into another done column are not counted as closed again. And finished work does not drop out when its column is deleted: each transition records whether its column was done at the time (`to_done`), which is what decides for a column that no longer exists. (Events written before this was recorded cannot be recovered that way.)

AI metrics — `AiStatistics`

`summary`, `byBoard`, `byWeek`, and cost formatting. **Staff only, and not by a check in the class**: every query starts from `StatisticsScope::aiRuns()`, which starts from `AiRun::visibleTo()`, which refuses customers outright. A customer asking for a total gets zero rows — not a filtered subset, and not an exception that would itself confirm the feature exists.

The honesty rule carries through: **a run whose model or token counts were never reported has no cost, and is reported as an unpriced run rather than folded in as zero.** A cost total that quietly understates itself by the runs it could not price is worse than one that says so.

Customer metrics — `CustomerStatistics`

`counts`, `byPriority`, `statusSplit` (open versus closed), `createdByWeek`, `recentActivity`, `recentTickets`. Underneath, `StatisticsScope::tickets()` starts from `Ticket::visibleTo($viewer)`, so for a customer the rows are already restricted to `customer_visible = true` on boards they belong to, **in SQL, before any counting happens**.

16.3 Filters

Four query-string values, resolved by `StatisticsScopeResolver`.

FILTER	VALUES
Board	A slug, or all reachable boards
Range	`this_month`, `this_quarter`, `this_year`, `custom`
From / To	Dates, for a custom range

`StatisticsScopeResolver` was extracted from the Livewire trait because the CSV export is a controller and cannot use a trait — and because an export that resolved its own board and date range would be a **second definition of "which numbers is this person allowed to ask for"**, and the copy in the download route is exactly the one nobody would notice going stale.

Nothing there is trusted:

- the board slug is resolved through `BoardAccess`, so a slug the viewer cannot reach yields a scope over **nothing** rather than that board's figures. *"No such board"* and *"not your board"* are deliberately the same answer, so the filter cannot be used to discover which boards exist;
- the range is parsed by `StatsPeriod`, which **corrects rather than trusts**: an unknown preset falls back to the default, reversed dates are swapped, and an absurd span is clamped to `MAX_DAYS` (1,096 — three years). A hand-edited URL cannot ask the database for ten years of transitions.

16.4 Export

EXPORT	ROUTE	GATE	PRODUCED BY

Team CSV	`/stats/export?dataset=…`	`role:admin,team`, re-checked in the controller	`StatisticsExport`
Customer CSV	`/stats/customer/export`	auth	`CustomerStatisticsExport`
Chart PNG / SVG	No route	—	The browser, from the inline SVG already on the page

The dataset names are an **allow-list**. `dataset=…` arrives from a query string and anything not in `KEYS` is refused rather than reflected — no dynamic method call, no `$this->{$name}()`.

Every dataset is derived from a `StatisticsScope` resolved for the requesting viewer, so a CSV cannot contain a figure its requester could not see on screen. There is no path that takes a board id, a ticket id or anything else a URL could name.

> **NOTE**
>
> **There is deliberately no per-ticket export.** These are aggregates. "Every ticket on this board as a spreadsheet" is a different feature with a different risk profile — it would carry titles and descriptions out of the application in bulk — and nobody has asked for it.

PNG and SVG are not server routes because the chart is already inline SVG on the page: exporting it is serialising a node that is already there, and rasterising it is drawing that node into a canvas. No library, no headless browser, no round trip.

SECTION 17

Realtime

17.1 Transport

Laravel Reverb, a first-party WebSocket server, deployed as its own Railway service from the same image with a different start command.

PIECE	DETAIL
Server	`php artisan reverb:start --host=0.0.0.0 --port=$PORT`
Client	Laravel Echo + pusher-js, constructed in `resources/js/echo.js`
Authorization	`POST /broadcasting/auth` behind the `web` middleware, decided by `routes/channels.php`
Optional	With `BROADCAST_CONNECTION=null` everything still works; screens simply do not refresh by themselves

WARNING

`VITE_REVERB_*` **are build arguments, not runtime variables.** Vite inlines `import.meta.env.*` at build time, and `echo.js` only constructs `window.Echo` when a key is present. An image built without one ships a browser bundle that never subscribes to anything, whatever the server is configured to broadcast. Changing one requires a **rebuild** of the web image, not a restart.

Only the **public** key belongs in the build stage. `REVERB_APP_SECRET` must never be passed there — everything in that stage is readable in the shipped JavaScript.

Keep the Reverb service at **one replica**. Reverb holds its connections in the memory of the process that accepted them, so a second replica would only deliver an event to the half of the clients attached to it. Several replicas need `REVERB_SCALING_ENABLED=true` and a shared Redis service.

17.2 Channels

Every channel is **private**. There is no public channel and no presence channel.

CHANNEL	WHO MAY JOIN	CARRIES
`board.{id}.internal`	Staff members of the board — `canView()` **and** `canSeeInternalContent()`	Content-free signals about internal changes
`board.{id}.customer`	Anyone who may view the board, customers included	Content-free signals about customer-visible changes
`users.{id}`	That user, while active	A nudge for the notification bell

Both board callbacks delegate to `BoardAccess`. Membership and the customer boundary are not restated in `channels.php`; that file decides **which existing question to ask**.

17.3 The audience rule

`App\Services\BoardBroadcaster` is the whole of the "do not broadcast internal data to customers" rule, in one place.

> **IMPORTANT**
>
> The rule for tickets is subtler than "is it customer-visible?", because **a ticket that has just been hidden also changes what customers see** — their card has to disappear. So the customer audience is signalled when the ticket is customer-visible **now OR was before the change**.
>
> Both are **content-free signals**: what the customer's browser then receives comes from the ordinary scoped query. And because the internal channel is only signalled for internal changes, a customer's browser is **never even woken** by activity they are not allowed to know about — which closes the timing side-channel a single shared channel would leave open even with an empty payload.

Events are dispatched **after the surrounding transaction commits**. Broadcasting a change that then rolls back would tell every open board to re-read something that never happened.

17.4 What updates live

SURFACE	BEHAVIOUR
Kanban board	Re-reads when a ticket on a joined channel changes, and when a tab or column is added, renamed, reordered or removed (`BoardBroadcaster::structureChanged`, both audiences — structure is not secret). Signals raised inside one transaction are coalesced per board and audience, so deleting a tab with fifty tickets wakes each open board once
Ticket conversation	New comments appear without a reload
Notification bell	Re-reads when `NotificationReceived` fires on the user channel
Documentation autosave	**Deliberately does not broadcast** — see section 9

SECTION 18

Theme System

Three states, defined in `App\Enums\ThemePreference`: `light`, `dark`, `system`.

Three cases rather than a boolean, because "follow the operating system" is a genuine third answer and not a default for one of the other two: somebody whose laptop turns dark at sunset wants Nexora to do the same, and storing that as `dark` at dusk would be wrong by morning.

18.1 Where the preference lives

LAYER	ROLE
`users.theme_preference`	The durable record, so the choice **follows a person between devices**
`localStorage`	A cache, so the correct theme is applied before any network request
The `<html>` element	**The source of truth at runtime** — both of the above are caches of it
An inline script in the layout head	Applies the class **before the bundle downloads**, so there is no flash of the wrong theme

`light` is the default: it is the appearance the product had before the setting existed, so anybody who never opens the control sees exactly the Nexora they already know. Following the device is offered, never assumed.

> **NOTE**
> The enum deliberately does **not** know which of light or dark `system` currently resolves to. Only the browser can answer that, it can change while a page is open, and nothing on the server needs it — the server's job is to render the *preference* so `theme.js` can apply it without a flash.

18.2 How a change is saved

FIGURE

```
Click
  |
  +--> set the class on <html>          } synchronous,
  +--> write localStorage               } so the colour changes instantly
  |
  +--> PUT /settings/theme              asynchronous, best-effort
          |
          v
  ThemePreferenceController validates against the enum
          |
          v
  users.theme_preference updated
```

A one-line controller rather than a Livewire action, and that is the whole design: the appearance has **already changed** by the time the request is sent. A Livewire round trip would put a server response between the click and the colour changing, and a theme toggle that waits is a theme toggle that feels broken.

Which also means **this endpoint failing is not an error worth showing anybody**. The theme is already applied and already in `localStorage`; all that is lost is the cross-device part, and it will be written again the next time somebody touches the control.

`PUT` rather than `POST` because it replaces a single value on the signed-in person's own row — there is no user id in the payload and no way to name somebody else.

18.3 Security note

`theme_preference` is cast to the enum but is **deliberately not mass-assignable**, for the same reason `role` is not. The value is rendered into a class attribute in the layout head, so an unvalidated string must never reach the column. The controller checks against the enum and refuses otherwise; there is no path that stores what it was given.

18.4 Coverage

Every component in the product carries both palettes. The one design rule worth recording:

> **NOTE**
>
> **Amber means attention, and nothing else, in this design system.** The internal-content signal is **slate plus a lock icon**, not amber — so a reader never has to work out whether an amber badge means "warning" or "internal". `AiCapabilityMode` follows the same rule: Observer is slate because "safe" is the normal state, Operator is brand, and only Agent — the mode that runs unattended — is amber.

SECTION 19

Notifications, Queues and Scheduled Work

19.1 Why background workers are required

WARNING

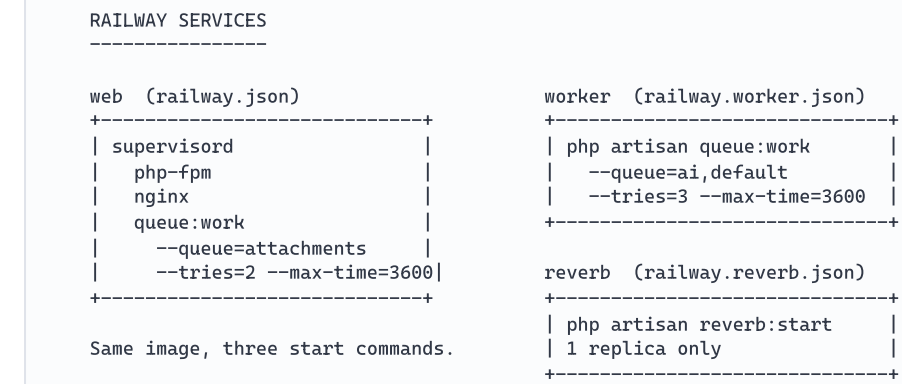
Without a queue worker, several features silently do nothing — and nothing errors, because all of it is queued by design. Realtime updates never arrive, no Slack message is posted, no SMS is sent, no pull request is ever linked to a ticket, and every AI run stays queued for ever.

This is the single most common misconfiguration for this application.

19.2 The three queues

QUEUE	CARRIES	WHERE THE WORKER RUNS	WHY
`ai`	`ExecuteAiRunJob`	The **worker** service	An AI run takes minutes. A worker chewing through a repository clone must never delay a broadcast or a notification
`default`	Broadcasts, in-app notifications, GitHub webhook processing, Slack messages, SMS	The **worker** service	Millisecond work
`attachments`	`ProcessAiAttachment`, `ImportDocumentFile`	**Inside the web container**	See below

FIGURE



IMPORTANT

`ProcessAiAttachment` **runs inside the web container, and the reason is the disk rather than the workload.** The job opens the file that was just uploaded. On a deployment whose attachment disk is `volume`, those bytes are on a Railway volume — and **a volume attaches to exactly one service**. The web service and the worker service each have their own, both mounted at `/data`, so `volume` resolves to a different filesystem in each container. The upload request writes the file in the web container; a worker in the other container asks its own `/data` and finds nothing.

Worse, a missing object is a *legitimate* condition in the reader, so the row settles into `failed` with no exception, no retry and nothing in the log to chase — the card simply says the file could not be read.

An S3-backed deployment has no such constraint. To consolidate, set `AI_ATTACHMENTS_QUEUE=default`, add it to the worker's `--queue` list, and remove the supervisor program.

`attachments` is deliberately **not** in the worker's queue list. Adding it in both places would mean two workers racing for the job and the wrong one winning about half the time.

The `ai` queue must be **named explicitly and is listed first**. A worker started with a bare `queue:work` listens only to `default` and would leave every AI run queued for ever.

19.3 Jobs

JOB	QUEUE	TRIES	TIMEOUT	NOTES
`ExecuteAiRunJob`	`ai`	3	900s	`WithoutOverlapping` lock plus an authoritative status claim; backoff 30s, 120s
`ProcessAiAttachment`	`attachments`	2	600s	Reads, extracts, transcribes or re-encodes an upload; also keeps a document as page Markdown for the assistant's docs import
`ImportDocumentFile`	`attachments`	1	600s	Imports a documentation file over 8 MB as a page off the web request; reports to the screen through the cache (`doc-import:{id}`) and removes the stored file if it cannot be read
`ProcessGitHubWebhookJob`	`default`	3	60s	Parses a verified delivery into links
`SendSmsJob`	`default`	3	—	Backoff 15s, 60s, 180s

Slack messages are queued through Laravel's notification system on `default`, with 3 tries and backoff 10s, 60s, 180s.

19.4 Failed jobs

Failed jobs are recorded in `failed_jobs` and **are not retried automatically**. An AI run or an SMS that exhausted its attempts has already recorded why on its own row, and blindly replaying a batch of them would re-post internal notes and re-send messages.

The table is pruned weekly at 168 hours — long enough to investigate a Monday failure on the following Friday.

19.5 Scheduled tasks

Run by `php artisan schedule:work` on the **worker** service, or by a Railway cron job invoking `schedule:run`. Running it on a web service with several replicas would run every task once per replica.

SCHEDULE	COMMAND	PURPOSE
Daily 03:15	`workspace:prune`	Webhook deliveries past 30 days; SMS records past 90 days
Every 10 minutes	`ai:reap-stalled`	Recover runs stranded by a worker that died mid-run
Weekly	`queue:prune-failed --hours=168`	Keep `failed_jobs` readable
Daily	`auth:clear-resets`	Expired password-reset tokens
Daily	`queue:prune-batches`	Batch records

All carry `->onOneServer()`, which needs a **shared cache store (Redis)** to mean anything on a multi-replica deployment.

> **NOTE**
> Everything scheduled is housekeeping. **Nothing a user waits for is scheduled, and nothing here is required for the application to function** — a deployment with no scheduler works, it simply accumulates rows. The one exception worth watching is `ai:reap-stalled`: without it, a run stranded by a deploy shows "Running" for ever and blocks further runs on that ticket.
>
> `workspace:prune` deletes in **chunks** rather than with one unbounded `DELETE`, because a single statement removing a year of rows takes a lock long enough to be noticed by everything else using the table.

19.6 Console commands

COMMAND	PURPOSE

`workspace:prune`	Retention enforcement for webhook deliveries and SMS records
`ai:reap-stalled`	Recover stranded AI runs; refuses to touch a run that still has a job on the queue
`workspace:icons`	Generate favicons; `--check` reports whether they exist without rewriting

19.7 In-app notification architecture

`App\Services\NotificationDispatcher` holds one rule:

> *A notification is only written for a recipient who passes **the same policy check that guards the thing being notified about**.*

That is deliberately a policy call and not a fresh set of conditions. *"May this person see this comment?"* already has an answer, in `CommentPolicy`, which delegates the ticket half to `TicketPolicy` and the board half to `BoardAccess`. Re-deriving it would create a second, unreviewed definition of the customer boundary — and the notification bell is exactly the kind of peripheral surface where such a copy goes stale.

A customer therefore **cannot** be notified about an internal note even if something upstream tried to: the gate check fails and no row is written.

Notifications are stored in the `notifications` table (`morphs('notifiable')`, `read_at`) and read back through `NotificationReader`, which re-checks every subject against the visibility scopes before returning a single line.

SECTION 20

Files and Attachments

20.1 Two pipelines, one storage service

	RECORD ATTACHMENTS	AI CONVERSATION ATTACHMENTS
Owners	Ticket, Comment, DocPage	AiSession
Config	`config/attachments.php`	`config/ai.php` → `attachments`
Size ceiling	10 MB	20 MB
Count ceiling	20 per owner	10 per conversation
Read into a prompt?	**No**	**Yes**
Type check	Extension allow-list	Extension **and** detected MIME
Storage	`AttachmentStorage`	`AttachmentStorage`

Both go through `App\Services\AttachmentStorage`, so there is exactly one answer to "where do uploaded files live" and one production durability check guarding it.

20.2 Storage rules

RULE	WHY
The stored path is **generated**, never derived from the uploaded filename	A user-supplied name can contain traversal sequences, null bytes or an executable extension. The original is kept only as a display label
The **disk is written onto every row**	Changing the application default later cannot orphan files uploaded before the change
Files are laid out **per board**	Keeps a board's data together for deletion and for any future per-tenant bucket policy

20.3 Disks

DISK	DURABLE?	USE
`local`	**No**	Development only. Refused in production
`volume`	Only while a persistent volume is actually mounted	Railway volume. **The mount is verified at boot**
`s3`	Yes, by construction	AWS S3, Cloudflare R2, MinIO, Backblaze B2 — anything S3-compatible

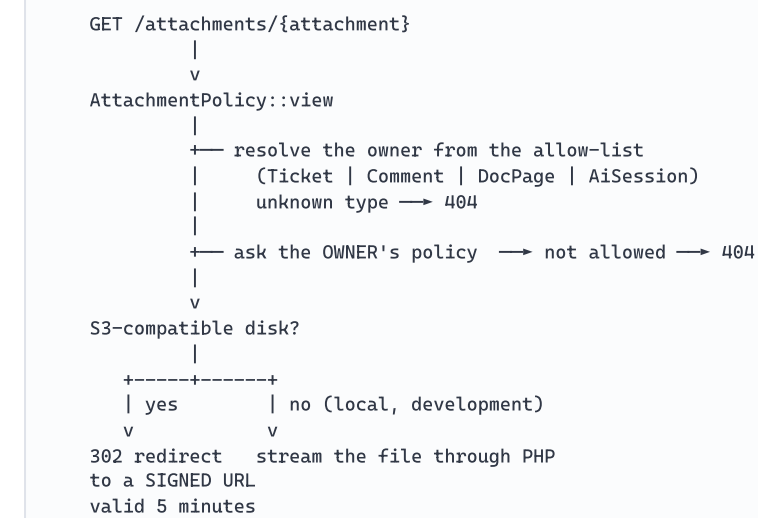
`ATTACHMENT_DISK` falls back to `FILESYSTEM_DISK`, so one setting moves everything at once, but the attachment disk can be pointed at a dedicated bucket without touching the rest of the application.

> **WARNING**
> A **production** boot with a non-durable attachment disk is **refused outright**. For `volume`, being named in `durable_disks` is not sufficient proof — the check stats the root and refuses to boot when the mount is absent. Otherwise a forgotten volume writes into the container and loses every file at the next deploy, silently, which is the exact failure the list exists to prevent.

Cloudflare R2 works as an `s3` disk: set `AWS_ENDPOINT` to the R2 endpoint, `AWS_USE_PATH_STYLE_ENDPOINT` as required, and the standard key, secret, bucket and region variables.

20.4 Download security

FIGURE



Never link a browser straight at the stored object. Every download passes through the controller so the request is authorized against the owning record first — which means an attachment on an internal ticket is exactly as protected as the ticket.

Signed URLs are short-lived on purpose: the link carries **no further authorization**, so it should outlive the click and little else. `ATTACHMENT_URL_TTL_MINUTES` defaults to 5.

A **soft-deleted owner does not resolve**, which is what makes the files on a deleted comment unreachable without any extra bookkeeping.

20.5 Allowed types

Record attachments (`config/attachments.php`):

`jpg`, `jpeg`, `png`, `gif`, `webp`, `svg`, `pdf`, `txt`, `md`, `csv`, `doc`, `docx`, `xls`, `xlsx`, `ppt`, `pptx`, `zip`, `log`, `json`, `yml`, `yaml`

AI conversation attachments — narrower, because these are *read* and put in front of a language model:

KIND	EXTENSIONS
text	`txt` , `log` , `json` , `yml` , `yaml`
markdown	`md` , `markdown`
csv	`csv` , `tsv`
pdf	`pdf`
image	`png` , `jpg` , `jpeg` , `gif` , `webp`
audio	`mp3` , `m4a` , `wav` , `ogg` , `webm` , `mp4`
document	`docx` , `xlsx` , `pptx`

> **NOTE**
>
> Two absences are deliberate. `svg` is a document that can carry script, not a picture — it is allowed as a record attachment, where it is only ever served as a download, but not as an AI attachment, where an "image" is decoded and sent to a vision model. `zip` would be a container whose contents nothing has validated.

20.6 Upload buffering

nginx buffers a request body larger than `client_body_buffer_size` — which any uploaded file is — to a file under `client_body_temp_path`, and a large FastCGI response likewise. Those paths must be writable by `www-data`, the user the nginx workers run as.

This is created and chowned in the Dockerfile **and repeated at boot** by the entrypoint, because nginx chowns a temp path it creates itself but silently uses a pre-existing one as-is. Getting it wrong meant every file upload returned 500 while every ordinary request worked — which in a Livewire application looks like the file simply vanishing, since a failed upload leaves no error on the page.

`client_max_body_size` is 32 MB, which must stay above the largest configured attachment ceiling.

SECTION 21

Database Architecture

MySQL in production; SQLite in memory for the test suite. 42 migrations, dated from `0001_01_01` (framework tables) through `2026_09_12`.

21.1 Entity relationship overview

FIGURE



21.2 Core tables

`users`

COLUMN	TYPE	NOTES
`name`, `email`	string	Email unique
`email_verified_at`	timestamp	Column exists; **no verification flow is implemented**

`password`	string	bcrypt, cast `hashed`
`role`	string(32), indexed	Default `team`. **Not mass-assignable**
`theme_preference`	string(10)	Default `light`. **Not mass-assignable**
`avatar_path`	string(2048), nullable	Generated storage key for the profile picture. Null is the whole of "no picture". **Not mass-assignable**
`avatar_disk`	string(40), nullable	Which disk it was written to, recorded per row so changing `ATTACHMENT_DISK` cannot orphan it. **Not mass-assignable**
`deactivated_at`	timestamp, indexed	Null means active

`boards`

COLUMN	TYPE	NOTES
`name`	string, indexed	
`slug`	string	**Unique.** The route key
`ticket_prefix`	string(12)	**Unique.** 2–6 chars, `/^[A-Z][A-Z0-9]*$/`
`next_ticket_number`	unsigned int	Default 1. Allocated under `lockForUpdate`
`settings`	JSON	AI, Slack, SMS and board options
`created_by_id`	FK users	`nullOnDelete`
`archived_at`	timestamp, indexed	

`board_tabs`

A board's tabs, each with its own columns. Every board has at least one (existing boards were given **Main** by migration).

COLUMN	TYPE	NOTES
`board_id`	FK boards	`cascadeOnDelete`
`name`	string(60)	**Unique with** `board_id` (ignoring case: the actions check, MySQL's collation agrees)
`slug`	string(80)	**Unique with** `board_id`**.** Set once at creation, never changed on rename — the board URL's `?tab=`
`position`	unsigned int	Sparse, not unique. The leftmost tab is the board's default

Indexes: `(board_id, name)` unique · `(board_id, slug)` unique · `(board_id, position)` · `(id, board_id)` unique — the target of `board_columns`' composite foreign key.

`board_columns`

COLUMN	TYPE	NOTES
`board_id`	FK boards	`cascadeOnDelete`
`board_tab_id`	unsigned big int, **NOT NULL**	The column's tab. `(board_tab_id, board_id) → board_tabs(id, board_id)`, `cascadeOnDelete`: the database refuses a column whose tab is on another board
`name`	string(60)	**Unique with** `board_tab_id` — two tabs can both have a "To Do"
`position`	unsigned int	Within the tab. Sparse, not unique
`is_done`	boolean	Terminal column

Tickets have no tab column: a ticket's tab is its column's tab, so it can never be in a column of another tab. Deleting a tab cascades to its columns, which `tickets.board_column_id` (RESTRICT) refuses while any ticket is still in one — tickets move first.

The tab migrations (`2026_10_22_000100` – `000300`) give every existing board a Main tab, assign its columns to it and change nothing else; each step checks before acting, so an interrupted run resumes. Rolling back refuses while any board has more than one tab (which tab a column was in would be lost).

`board_members`

`(board_id, user_id)` **unique**, plus an index on `(user_id, board_id)`. Both foreign keys cascade on delete. `added_by_id` records who granted access.

This table **is** the access-control list.

`tickets`

COLUMN	TYPE	NOTES
`board_id`	FK boards	`cascadeOnDelete`
`board_column_id`	FK board_columns	`restrictOnDelete` — a column cannot be deleted out from under its tickets
`number`	unsigned int	**Unique with** `board_id`
`title`	string(200)	
`type`	string(20)	Default `task`
`description_md`	text	

`priority`	string(20)	Default `medium`
`assignee_id`, `created_by_id`	FK users	`nullOnDelete`
`estimate`	decimal(6,2)	
`due_date`	date, indexed	
`customer_visible`	boolean	**Default** `false`
`position`	unsigned int	

Indexes: `(board_id, number)` unique · `(board_column_id, position)` · `(board_id, customer_visible)` · `(board_id, assignee_id)` · `due_date` · `(board_id, created_at)` · `(board_id, updated_at)`.

> **NOTE**
> `(board_id, customer_visible)` is the index that makes the customer boundary cheap: the filter applied to almost every ticket read is a composite index seek rather than a scan. The two reporting indexes were added specifically for the statistics screens.

`comments`

`ticket_id`, `board_id`, `stream` (default `internal`), `author_id`, `body_md`, `edited_at`, `deleted_by_id`, soft deletes. Indexes: `(ticket_id, stream, created_at)` and `(board_id, stream)` — both lead with the scoping column so a stream-filtered read is an index range.

`doc_pages`

COLUMN	NOTES
`parent_id`	Self-referencing, `restrictOnDelete`
`slug`	**Unique with** `board_id`
`icon`	string(16), an emoji
`body_md`	longText
`customer_visible`	Default `false`
`draft_title`, `draft_md`, `draft_saved_at`, `draft_by_id`	The autosave buffer
`created_by_id`, `updated_by_id`	Authorship

Indexes: `(board_id, slug)` unique · `(board_id, parent_id, position)` · `(board_id, customer_visible)`.

`ticket_events` — the history

`ticket_id`, `board_id`, `type`, `payload` JSON, `actor_id`, `from_column_id`, `to_column_id`, `created_at`.

Indexes: `(ticket_id, created_at)` · `(board_id, type, created_at)` · `(board_id, to_column_id, created_at)`.

> **IMPORTANT**
> This table is **append-only** and is the source for every flow metric. The column ids are stored as dedicated indexed columns rather than only inside the JSON payload precisely so that "which tickets entered a done column on this board last week" is one index range scan. A later migration backfilled them from existing payloads.

`attachments`

Polymorphic `morphs('attachable')` with an **enforced morph map** (`ticket`, `comment`, `doc_page`, `ai_session`). `board_id` is **nullable** — an AI conversation can be workspace-scoped and belong to no board. Stores `disk`, `path`, `filename`, `mime_type`, `size`, `uploaded_by_id`.

> **NOTE**
> `Relation::enforceMorphMap()` rather than `morphMap()` means an attachment pointing at a type nobody has registered **fails loudly** instead of resolving to something unchecked. Enforcement is all-or-nothing, which is why `user`, `board` and `board_column` are also registered — they appear in `notifications.notifiable_type` and `activity_log.subject_type`.

21.3 AI tables

TABLE	PURPOSE	KEY COLUMNS
`ai_settings`	Workspace-wide AI configuration, a **single row**	`enabled`, `provider`, `model`, `capability_mode` (default `observer`), `session_token_limit`, `daily_user_token_limit`
`ai_credentials`	One row per provider	`provider` **unique**, `secret` (**encrypted**), `last_four`, `hint`, `rotated_at`
`ai_sessions`	One conversation	`uuid` unique, `user_id`, `board_id` (nullable), `provider`, `model`, `capability_mode`, `chat_mode`, `knowledge_scope`, token and cost totals, `tokens_cache_read` / `tokens_cache_write`, `context_tokens` (how full the model's window is), `history_anchor_message_id` (the trim point; forward-only, deliberately not a foreign key)
`ai_chat_messages`	Turns	`board_id` **nullable**, `ai_session_id`, `role`, `content`, `metadata` (holds the proposal)

`ai_runs`	One ticket run	`uuid` unique, `trigger_source`, `mode`, `status`, `repository`, `model`, `result_comment_id`, `branch_name`, `pull_request_url`, tokens, cost, duration, `error_message`
`ai_usage_records`	One row per billed exchange	`purpose` (`chat` / `ticket_run`), `provider`, `model`, tokens (input inclusive of cached), `tokens_cache_read` / `tokens_cache_write`, `context_tokens`, `usage_reported`, `estimated_cost`
`ai_attachments`	One uploaded file, processed	`attachment_id` **unique**, `kind`, `status`, `checksum`, `extracted_text`, `structured`, `summary`, `characters`, `token_estimate`, `truncated`, `context_sent_at`, `markdown` / `markdown_truncated` (the whole file as a page body, for importing into the docs)
`ai_tool_invocations`	**The AI audit log**	`tool`, `category`, `channel` (`assistant` | `mcp`, indexed with `created_at`), `target`, `input`, `success`, `outcome` (`ok`, `refused`, `invalid`, `not_found`, `error`, or `awaiting_confirmation` for an MCP change waiting for the person), `result_characters`, `duration_ms`. MCP rows have no session; a read usually has no board, a change records its board

21.4 OAuth tables (remote MCP server)

Laravel Passport's own tables, published unchanged. `user_id` references `users.id`; the client id is a UUID.

TABLE	HOLDS
`oauth_clients`	One row per registered AI client (dynamic registration creates public clients with their redirect URIs)
`oauth_auth_codes`	Short-lived authorization codes awaiting exchange
`oauth_access_tokens`	Issued access tokens: `user_id`, `scopes`, `revoked`, `expires_at`
`oauth_refresh_tokens`	Refresh tokens, by `access_token_id`; rotated on use
`oauth_device_codes`	Created by Passport's migrations; unused — the device grant is off

Revoked and expired rows are removed daily by `passport:purge`.

21.5 Integration tables

TABLE	PURPOSE
`board_repositories`	`repository_name` **unique per board**, `default_branch`, `is_primary`, `configuration` JSON
`github_links`	Unique on `(ticket_id, repository, type, external_id)`. Holds `state`, `ci_status`, `author_login`, `merged_at`
`webhook_deliveries`	Unique on `(source, delivery_id)` — **the insert is the de-duplication claim**
`sms_messages`	Unique on `(event, ticket_id, recipient)` — duplicate suppression enforced by the database, not a cache
`activity_log`	`spatie/laravel-activitylog`, with an added `board_id` foreign key and `(board_id, created_at)` index

21.6 Framework tables

`cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `sessions`, `password_reset_tokens`, `notifications`.

21.7 Referential integrity strategy

PATTERN	USED FOR	EFFECT
`cascadeOnDelete`	`board_id` on almost everything, `ticket_id` on children	Deleting a board removes its entire graph
`restrictOnDelete`	`tickets.board_column_id`, `doc_pages.parent_id`	Prevents orphaning structure
`nullOnDelete`	Every `*_by_id` and `assignee_id`	**Deleting a user preserves the history they created** — a ticket keeps existing with no creator rather than vanishing

21.8 Notable migration decisions

MIGRATION	WHAT IT DID, AND WHY IT IS INTERESTING
`allow_workspace_scoped_ai_conversations`	Made `ai_chat_messages.board_id` nullable so a conversation can be about the whole workspace, and added `(user_id, board_id, created_at)`. The `down()` **deletes** workspace-scoped rows before restoring `NOT NULL`
`add_sessions_to_ai_chat_messages`	Backfilled `ai_session_id` by grouping existing messages per user and board into adopted sessions, so no transcript was orphaned

`add_column_ids_to_ticket_events`	Backfilled `from_column_id` / `to_column_id` from the JSON payload in chunks of 500
`make_attachment_board_optional`	Made `attachments.board_id` nullable for AI conversation uploads
`add_reporting_indexes_to_tickets`	Added the two indexes the statistics screens needed

NOTE

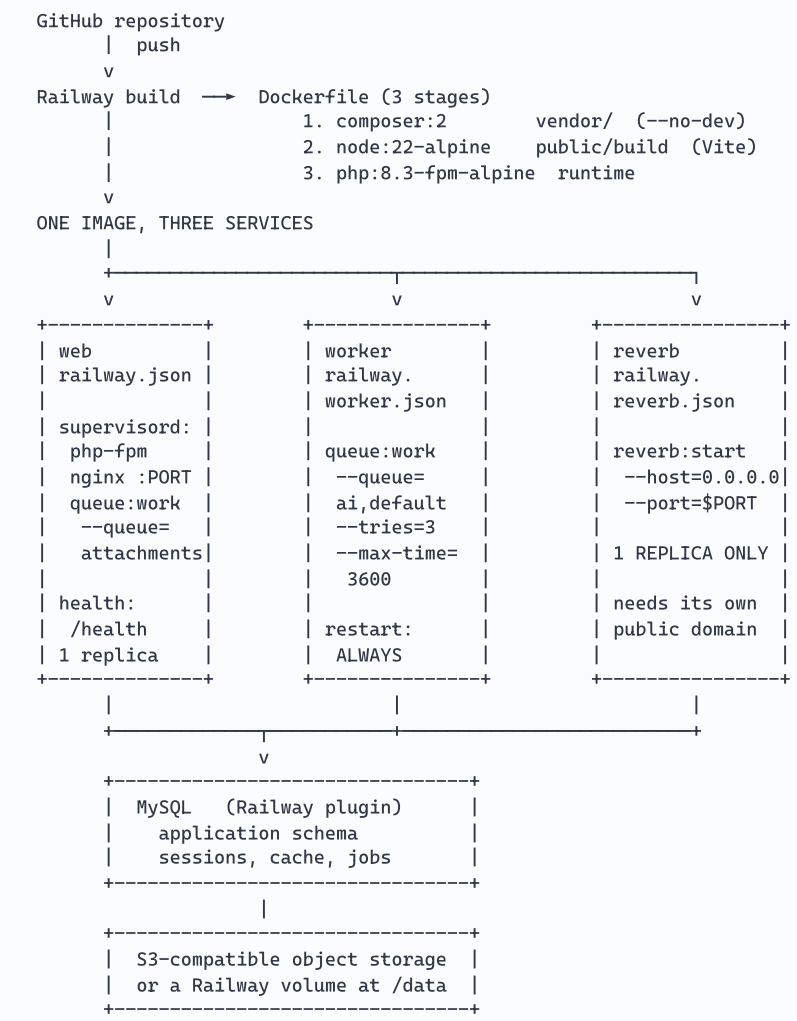
Several `down()` methods are **destructive by necessity** — restoring a `NOT NULL` constraint requires removing the rows that violate it. Read a rollback before running one in an environment whose data matters.

SECTION 22

Deployment Architecture

22.1 Production topology

FIGURE



TLS is terminated at the Railway edge and the request is proxied over HTTP, which is why the application sets `trustProxies(at: '*')` and forces the `https` scheme for generated URLs.

22.2 The Docker build

Stage 1 — vendor

`composer:2`, installing with `--no-dev --no-scripts`. Scripts are skipped because `artisan` is not present yet; the autoloader is rebuilt against the full source tree in the runtime stage.

Stage 2 — assets

`node:22-alpine`, `npm ci`, then `npm run build`.

It copies `resources`, **plus** `app/` **and** `config/` — Tailwind scans Blade templates *and* PHP sources for class names. It also copies Laravel's pagination views out of the vendor stage, because `resources/css/app.css` declares `@source` paths that must resolve or the pagination styles silently drop out of the bundle.

> **WARNING**
>
> `VITE_REVERB_APP_KEY`, `VITE_REVERB_HOST`, `VITE_REVERB_PORT` and `VITE_REVERB_SCHEME` are **build arguments**. Vite inlines `import.meta.env.*` at build time, so they cannot be supplied as runtime variables later. An image built without them ships a browser bundle that never subscribes to anything.
>
> **Only the public key belongs here.** `REVERB_APP_SECRET` **must never be passed to this stage** — everything in it is readable in the shipped JavaScript.
>
> Empty is a working default, not a broken one: the build succeeds and the application runs exactly as it does with realtime switched off.

Stage 3 — runtime

`php:8.3-fpm-alpine`, with:

COMPONENT	WHY
PHP extensions `bcmath`, `gd`, `intl`, `opcache`, `pcntl`, `pdo_mysql`, `zip`	`gd` is the one worth explaining — it decodes and re-encodes every uploaded image before it reaches a model, stripping EXIF (routinely including GPS) and making the declared media type true by construction. Without it every image attachment reports itself unreadable. **The build asserts `php -m
`libjpeg-turbo`, `libwebp`	PNG and GIF come with `gd`; JPEG and WebP have to be asked for, and the assistant's allow-list accepts all four
`nginx`, `supervisor`, `curl`, `icu-libs`, `libzip`, `tzdata`	Runtime
`git`	The isolated checkout for apply mode
`claude-code`	The apply-mode coding runtime, from Anthropic's APK repository
`nodejs`, `npm`	For `AI_VALIDATION_COMMANDS`, which runs the **target repository's** own build inside the checkout
`ripgrep`, `libgcc`, `libstdc++`	Alpine is musl, so Claude Code needs a real ripgrep rather than its bundled glibc-linked copy

NOTE

`git`, `claude-code`, `node` and `npm` are **not used by the web service**. They are in the shared image because the worker runs the *same image* with a different start command, and an apply run refuses immediately without them. The assets stage cannot supply node, because only `public/build` is copied out of it — without node in the runtime, a validation command would fail with `sh: npm: not found` **after** the model had already done its work.

The build finishes by dumping an optimised autoloader, **removing `tests/` and `phpunit.xml`**, creating the writable directories, and creating and chowning nginx's spill-to-disk paths.

22.3 Entrypoint sequence

`docker/entrypoint.sh` runs before supervisord. Everything in it is runtime work that must **not** happen at image build time — configuration caching reads environment variables that only exist once the service is running, and migrations need a reachable database.

FIGURE

```
1.  Substitute $PORT into nginx.conf
2.  Create and chown /var/cache/nginx/{client_body,fastcgi}  → www-data
3.  Recreate the storage tree (ephemeral on Railway) and chown it
4.  If ATTACHMENT_VOLUME_PATH is set: mkdir + chown (non-recursive)
5.  ABORT if APP_KEY is unset
6.  config:clear, then config:cache, route:cache, view:cache, event:cache
7.  If RUN_MIGRATIONS=true:
       migrate:status succeeds?  →  migrate --force --isolated
       migrate:status fails?     →  migrate --force        (empty database)
8.  If RUN_SEEDERS=true: db:seed --force
9.  exec supervisord
```

Two details are load-bearing:

Migrations are opt-in. `RUN_MIGRATIONS` must be set on **exactly one service**, so a rollback or a second replica cannot race the schema.

The first migration deliberately runs unisolated. `--isolated` takes its lock from the default cache store, which on the `database` store lives in `cache_locks` — a table created by a migration. On a first deploy against an empty database the lock cannot be taken, because the table it needs is one of the tables it is guarding the creation of. Passing `--isolated` there fails with *"Base table or view not found: cache_locks"* and, with `set -e`, restart-loops the container. The probe is `migrate:status`, which exits non-zero while the migrations table is absent.

The attachment volume is prepared but not created. If `ATTACHMENT_VOLUME_PATH` is missing from the mount, `mkdir` makes an ordinary container directory — and the durability check in `AppServiceProvider` then refuses to boot, which is the intended outcome.

22.4 Web container processes

PROGRAM	PRIORITY	COMMAND

`php-fpm`	10	`php-fpm --nodaemonize --force-stderr`
`nginx`	20	`nginx -g "daemon off;"`
`attachment-worker`	30	`queue:work --queue=attachments --sleep=1 --tries=2 --max-time=3600`, as `www-data`

All logs go to stdout/stderr so Railway collects them.

22.5 PHP and nginx tuning

SETTING	VALUE	NOTE
`memory_limit`	256M	
`max_execution_time`	60	Streaming AI answers raise this per-request via `set_time_limit()`
`upload_max_filesize` / `post_max_size`	32M / 34M	Must stay above the attachment ceiling
`expose_php`, `display_errors`	Off	Errors go to stderr
OPcache	`validate_timestamps = 0`	Application code never changes inside a running container, so timestamp validation is pure overhead
`session.use_strict_mode`	1	Rejects browser-supplied session ids
PHP-FPM pool	`pm = dynamic`, max 20 children, `max_requests = 500`	Recycles workers to bound leaks
nginx `client_max_body_size`	32m	
nginx workers	`user www-data`	**Must** match the PHP-FPM pool user, which is why the temp paths are chowned explicitly

22.6 Health checks

PROBE	PATH	USED BY	CHECKS
Framework	`/up`	Docker `HEALTHCHECK`, every 30s	The framework boots
Application	`/health`	Railway (`healthcheckPath`), timeout 60s	**Database answering** and **cache store writable**

`/health` answers one question: can this container serve a request end to end? Both checks are round trips a real request would make anyway, and both fail in ways a process-liveness check cannot see — a container whose database credentials are wrong boots perfectly and 500s on every page.

What it deliberately does **not** do:

- **Nothing slow.** No queue depth, no third-party reachability, no disk scan. It runs every thirty seconds for the life of the deployment, and a probe that calls Anthropic or GitHub turns their outage into our restart loop.
- **Nothing revealing.** It is unauthenticated by necessity, so it returns booleans and a timestamp — no hostnames, no versions, no driver names, no error text.
- **No pending-migration failure.** Pending migrations are *reported* but do not mark the container unhealthy: migrations run from one service, so during a deploy every other service would briefly report pending and be restarted in a loop by the very platform waiting for the migration to finish.

22.7 Restart policy

SERVICE	POLICY	REPLICAS
web	`ON_FAILURE`, max 5 retries	1
worker	`ALWAYS`, max 10 retries	1
reverb	`ALWAYS`, max 10 retries	**1 — do not raise without Redis**

22.8 Deployment process

1. Push to the default branch; Railway builds the Dockerfile.
2. Build arguments supply the `VITE_REVERB_*` values.
3. On boot, the entrypoint caches configuration, routes, views and events from the **live** environment.
4. The service with `RUN_MIGRATIONS=true` migrates.
5. Railway polls `/health` before routing traffic to the new deployment.
6. The worker and reverb services restart onto the new image.

> **IMPORTANT**
>
> **A deploy restarts the worker, which is why `ai:reap-stalled` exists.** Any AI run in flight is orphaned; the reaper recovers it within ten minutes. An apply run takes minutes, so this is ordinary rather than exceptional.

The remote MCP server in production

- **Passport keys.** Set `PASSPORT_PRIVATE_KEY` and `PASSPORT_PUBLIC_KEY` on the **web** service (generate once, e.g. `openssl genrsa -out private.pem 4096 && openssl rsa -in private.pem -pubout -out public.pem`; paste the PEM text, never commit it). Without them every OAuth request and every `/mcp` call with a token fails. The worker and Reverb do not need them.
- **Timezone.** `WORKSPACE_DEFAULT_TIMEZONE=Europe/Stockholm` on web and worker, so "yesterday" means the Swedish day.
- **The URL** people add as a connector is exactly `https://<host>/mcp` — no trailing slash. Check `GET /.well-known/oauth-protected-resource/mcp` returns that value as `resource`; it is built from the

request, so it is only right because `trustProxies('*')` and `URL::forceScheme('https')` make Laravel see HTTPS behind Railway's edge.

- **Reachability.** Claude's calls come from Anthropic's published egress range and ChatGPT's from OpenAI's; both need the public HTTPS endpoint. Nothing else is required.
- **Scheduler.** `passport:purge` is scheduled, like the other housekeeping, and only runs where a scheduler does.
- **The release that added changes over MCP** runs migration `2026_10_21_000100_revoke_read_only_mcp_connections`, which signs out every connected AI app once. Tell people to expect it: in Claude, Settings → Connectors → Nexora → Connect, and agree to the new screen. No new variable is needed; the defaults are `MCP_WRITES_ENABLED=true` and `MCP_WRITE_RATE_LIMIT=30`.
- **Back to read-only** without a deploy: set `MCP_WRITES_ENABLED=false` on the web service. Every write tool disappears from the next `tools/list`; reads keep working. Do not roll the revoke migration back — un-revoking credentials is never the right way back.

22.9 The board tabs release

Migrations `2026_10_22_000100` – `000300` give every board a **Main** tab and move its columns into it; nothing else in the data changes. No variable, package or service is added.

- **Before deploying:** take a database backup; deploy in a quiet moment.
- **During the deploy** the previous release keeps serving for the few seconds between the last migration and the switch-over. In that window only *adding a board or a column* can fail (the previous code does not set a column's tab), as a rolled-back request with nothing lost; anything it did add before the last migration is given a tab by it. Everything else keeps working, because every board still has exactly one tab.
- **After deploying:** open a board — the strip shows to the delivery team with **Main** and "+" — and check no board was left without a tab: `select count(*) from boards where id not in (select board_id from board_tabs);` → 0.
- **Rolling back:** run `php artisan migrate:rollback --step=3` from the *new* image **before** redeploying the old one; the old image has no tab migrations to roll back. This works only while every board still has one tab — once people have split boards, the down migration refuses (which tab a column was in would be lost) and the way back is forward.

22.10 Rollback considerations

CONCERN	GUIDANCE
Code rollback	Redeploy the previous image. Configuration is re-cached at boot. After the board tabs release, roll the tab migrations back first — see above
Schema rollback	**Read the `down()` method first.** Several are destructive by necessity — restoring a `NOT NULL` constraint requires deleting the rows that violate it

`RUN_MIGRATIONS` during rollback	Leave it off on the rolled-back deploy unless you have read the down migrations
Attachments	Unaffected — they live in object storage or on a volume, keyed by the disk recorded on each row
In-flight AI runs	Orphaned by the restart; recovered by the reaper
Queued jobs	Survive in the `jobs` table and are picked up by the new worker
`APP_KEY`	**Never rotate casually.** It decrypts Slack webhook URLs and stored AI credentials; rotating it silently disables both, degrading to "not configured"

22.11 Scaling notes

WANT TO SCALE	DO THIS
Web traffic	Raise web replicas. Sessions and cache are in MySQL, so this is safe
AI throughput	Split the worker: one `--queue=ai`, one `--queue=default`
Realtime connections	Requires `REVERB_SCALING_ENABLED=true` **and** a Redis service
Scheduler	Must run on **one** service. `->onOneServer()` needs a shared cache store to mean anything
Attachment worker	Stays in the web container unless the disk is S3

SECTION 23

Environment Variables

183 variables are documented in `.env.example`. Below are the ones a deployment actually has to decide about, grouped by concern.

> **WARNING**
>
> **Every value in this section is a placeholder.** Never commit real credentials. `.env` is in `.gitignore`, alongside `.env.backup` and `.env.production`.

23.1 Required

VARIABLE	PURPOSE	EXAMPLE
`APP_KEY`	Encryption key. **Also decrypts Slack webhook URLs and stored AI credentials**	`base64:<APP_KEY>`
`APP_ENV`	Environment	`production`
`APP_DEBUG`	Debug output	`false`
`APP_URL`	Canonical URL; drives generated links	`https://nexora.example.com`
`DB_CONNECTION`	Driver	`mysql`
`DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	Connection	`<DB_PASSWORD>`

23.2 Workspace identity

VARIABLE	DEFAULT	PURPOSE
`WORKSPACE_NAME`	`Aqueduct Workspace`	Sidebar and transactional mail. **Set to `Nexora` to brand the product as Nexora**
`WORKSPACE_SHORT_NAME`	`Aqueduct`	Page-title suffix
`WORKSPACE_DEFAULT_TIMEZONE`	`UTC`	Default board timezone; also where "today" and "yesterday" are cut for statistics and MCP answers. Production: `Europe/Stockholm`
`WORKSPACE_NEW_TICKETS_VISIBLE`	`true`	Whether tickets staff create start visible to the board's customers (form, quick add, assistant). A board can override it
`ALLOW_PUBLIC_REGISTRATION`	`false`	Self-service sign-up. Registrants get the `customer` role

23.3 Session and security

VARIABLE	DEFAULT	PRODUCTION GUIDANCE
`SESSION_DRIVER`	`database`	Keep
`SESSION_LIFETIME`	`120`	Minutes
`SESSION_SECURE_COOKIE`	`false`	**Set to `true`**
`SESSION_ENCRYPT`	`false`	Consider `true`
`SESSION_SAME_SITE`	`lax`	Keep
`BCRYPT_ROUNDS`	`12`	Keep

23.4 Storage

VARIABLE	DEFAULT	PURPOSE
`FILESYSTEM_DISK`	`local`	**Must be `s3` or `volume` in production** — boot is refused otherwise
`ATTACHMENT_DISK`	falls back to `FILESYSTEM_DISK`	Point attachments at a dedicated bucket
`ATTACHMENT_VOLUME_PATH`	—	Mount path, when using a Railway volume
`ATTACHMENT_MAX_SIZE_KB`	`10240`	10 MB per record attachment
`ATTACHMENT_MAX_PER_OWNER`	`20`	
`ATTACHMENT_URL_TTL_MINUTES`	`5`	Signed-URL lifetime
`DOC_IMPORT_MAX_FILES` / `DOC_IMPORT_MAX_SIZE_KB`	`10` / `20480`	Importing files as documentation pages: files per drop, size of one
`DOC_IMPORT_SYNC_MAX_KB`	`8192`	Larger imports are converted on the attachments queue
`DOC_IMPORT_BODY_MAX_CHARACTERS`	`200000`	What one imported page holds; the rest stays in the attached original
`DOC_IMPORT_MAX_TABLE_ROWS` / `DOC_IMPORT_PDF_MAX_PAGES`	`500` / `300`	Rows of a CSV or worksheet, pages of a PDF
`AWS_ACCESS_KEY_ID`	—	`<AWS_ACCESS_KEY_ID>`
`AWS_SECRET_ACCESS_KEY`	—	`<AWS_SECRET_ACCESS_KEY>`
`AWS_BUCKET`, `AWS_DEFAULT_REGION`	—	
`AWS_ENDPOINT`	—	Set for Cloudflare R2, MinIO or Backblaze B2

`AWS_USE_PATH_STYLE_ENDPOINT`	`false`	Required by some S3-compatible providers

23.5 Queue, cache and realtime

VARIABLE	DEFAULT	PURPOSE
`QUEUE_CONNECTION`	`database`	
`CACHE_STORE`	`database`	Redis needed for multi-replica scheduler locks
`REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`	—	Optional
`BROADCAST_CONNECTION`	`reverb`	`null` disables realtime cleanly
`REVERB_APP_ID`, `REVERB_APP_KEY`	—	Server credentials
`REVERB_APP_SECRET`	—	`<REVERB_APP_SECRET>` **— never pass to the build stage**
`REVERB_HOST`, `REVERB_PORT`, `REVERB_SCHEME`	—	The public domain browsers connect to
`REVERB_SERVER_HOST`, `REVERB_SERVER_PORT`	`0.0.0.0`, `$PORT`	What the process binds
`VITE_REVERB_APP_KEY`	—	**BUILD ARGUMENT.** Public key only
`VITE_REVERB_HOST`, `VITE_REVERB_PORT`, `VITE_REVERB_SCHEME`	—	**BUILD ARGUMENTS**

23.6 AI — core

VARIABLE	DEFAULT	PURPOSE
`AI_ENABLED`	`true`	Master switch
`AI_PROVIDER`	`anthropic`	Default provider
`ANTHROPIC_API_KEY`	—	`<ANTHROPIC_API_KEY>`. May instead be stored via the settings screen
`OPENAI_API_KEY`	—	`<OPENAI_API_KEY>`. Also powers voice and audio transcription
`OPENAI_BASE_URL`	`https://api.openai.com/v1`	For a proxy or Azure
`AI_MODEL`	`claude-opus-5`	Default model
`AI_DEFAULT_MODE`	`agent`	**Capability ceiling.** Set `operator` or `observer` for a cautious posture
`AI_MAX_OUTPUT_TOKENS`	`16000`	

`AI_EFFORT`	`high`	`low` / `medium` / `high` / `xhigh` / `max`
`AI_REQUEST_TIMEOUT`	`300`	Seconds
`AI_MAX_RETRIES`	`2`	

23.7 AI — limits and caps

VARIABLE	DEFAULT	PURPOSE
`AI_SESSION_TOKEN_LIMIT`	`0`	A spending cap per conversation, **off by default**. Not the context window — see section 9
`AI_DAILY_USER_TOKEN_LIMIT`	`2000000`	Per person per day
`AI_CACHE_READ_WEIGHT`	`0.1`	What a token read from the provider's cache counts for, against both limits
`AI_SESSION_HISTORY`	`25`	Sessions returned by `AiSessionManager::history()`; not currently listed in the interface
`AI_DAILY_AUTO_RUN_CAP`	`20`	Per board. **No bypass exists**
`AI_MAX_DAILY_AUTO_RUN_CAP`	`500`	Ceiling a board may configure
`AI_DAILY_MANUAL_RUN_CAP`	`100`	
`AI_ADMINS_BYPASS_CAP`	`true`	Manual cap only

23.8 AI — chat, tools and knowledge

VARIABLE	DEFAULT	PURPOSE
`AI_CHAT_DEFAULT_MODE`	`reading`	
`AI_CHAT_READING_MODEL`	`claude-sonnet-5`	
`AI_CHAT_WRITING_MODEL`	`claude-opus-5`	
`AI_CHAT_EVERYTHING_MODEL`	`claude-opus-5`	
`AI_CHAT_HISTORY_MAX_MESSAGES`	`500`	Stored turns replayed, at most. The whole conversation is replayed up to the model's window. **Replaces** `AI_CHAT_HISTORY`, which is no longer read
`AI_CHAT_CONTEXT_BUDGET`	`0.8`	Share of the model's context window one request may use
`AI_CHAT_DEFAULT_CONTEXT_WINDOW`	`128000`	For a model the catalogue gives no window for (the OpenAI ids)

`AI_CHAT_CONTEXT_MAX_TOKENS`	`0`	Hold every conversation below the model's window, to bound one question's cost. 0 = the model's own
`AI_CHAT_CACHE` / `AI_CHAT_CACHE_TTL`	`true` / `5m`	Prompt caching (Anthropic); `1h` for conversations with long pauses
`AI_CHAT_CONTEXT_PAGE_CHARS`	`40000`	The documentation page on screen, sent with the question
`AI_CHAT_MAX_OUTPUT_TOKENS`	`4000`	
`AI_CHAT_MAX_OUTPUT_TOKENS_OUTSIDE`	`8000`	Outside Project scope
`AI_CHAT_MAX_OUTPUT_TOKENS_WRITING`	`16000`	A turn that may propose changes, so a batch of tickets is not cut off
`AI_CHAT_MAX_TICKETS_PER_REPLY`	`30`	Most tickets one `propose_create_tickets` call may carry
`AI_CHAT_STREAM_TIME_LIMIT`	`360`	Must exceed `AI_REQUEST_TIMEOUT`
`AI_CHAT_CONTEXT_TICKETS` / `_PAGES` / `_EVENTS`	`60` / `40` / `40`	Board context depth
`AI_CHAT_CONTEXT_BOARDS` / `_BOARD_TICKETS`	`12` / `6`	Workspace context breadth
`AI_TOOLS_ENABLED`	`true`	Retrieval tools
`AI_TOOLS_MAX_ROUNDS` / `_CALLS` / `_CHARACTERS`	`4` / `6` / `60000`	Loop bounds
`AI_KNOWLEDGE_ENABLED`	`true`	Live external lookup
`AI_KNOWLEDGE_DRIVER`	*(empty)*	**No driver ships.** Empty resolves to the refusing implementation
`AI_KNOWLEDGE_MAX_RESULTS` / `_TIMEOUT`	`5` / `10`	

23.9 AI — runs, code generation and repositories

VARIABLE	DEFAULT	PURPOSE
`AI_QUEUE`	`ai`	
`AI_JOB_TRIES` / `AI_JOB_TIMEOUT`	`3` / `900`	Timeout must exceed request + clone + test time
`AI_CODE_DRIVER`	`unavailable`	Set `claude_code` to enable apply mode
`AI_CLAUDE_CODE_BINARY`	`claude`	
`AI_CODE_TIMEOUT`	`1800`	

`AI_VALIDATION_COMMANDS`	*(empty)*	Pipe-separated; run in the checkout **before** committing
`AI_VALIDATION_TIMEOUT`	`900`	
`AI_REPOSITORY_CLONE_ENABLED`	`false`	Opt-in
`AI_CLONE_DEPTH`	`1`	Shallow
`AI_GIT_BINARY` / `AI_GIT_TIMEOUT`	`git` / `300`	
`AI_BRANCH_PREFIX`	`ai/`	
`AI_COMMIT_AUTHOR_NAME` / `_EMAIL`	`Workspace AI` / `ai@example.invalid`	
`AI_WORKSPACE_PATH`	`ai-runs`	Relative to `storage/app`
`AI_KEEP_FAILED_WORKSPACES`	`false`	Leave `false` on Railway
`AI_MAX_CONTEXT_FILES` / `_BYTES`	`400` / `60000`	Tree summarised into the prompt

23.10 AI — attachments and voice

VARIABLE	DEFAULT	PURPOSE
`AI_ATTACHMENTS_ENABLED`	`true`	
`AI_ATTACHMENTS_QUEUE`	`attachments`	**Must be served by a worker that can read the file**
`AI_ATTACHMENT_MAX_SIZE_KB`	`20480`	20 MB
`AI_ATTACHMENT_MAX_PER_SESSION`	`10`	
`AI_ATTACHMENT_FILE_CHARS` / `_TOTAL_CHARS` / `_DIGEST_CHARS`	`200000` / `400000` / `1200`	A file per request, all files per request, a summary when there is no room
`AI_ATTACHMENT_CSV_ROWS` / `_COLUMNS` / `_SAMPLES`	`300` / `40` / `5`	
`AI_ATTACHMENT_IMAGE_MAX_DIMENSION` / `_QUALITY`	`1568` / `82`	
`AI_ATTACHMENT_AUDIO_ENABLED`	`true`	Needs an OpenAI key
`AI_TRANSCRIPTION_MODEL` / `_TIMEOUT`	`whisper-1` / `180`	
`AI_VOICE_ENABLED` / `_DRIVER`	`true` / `openai`	Needs an OpenAI key
`AI_VOICE_TRANSCRIPTION_MODEL`	`whisper-1`	
`AI_VOICE_SPEECH_MODEL`	`gpt-4o-mini-tts`	
`AI_VOICE_VOICES` / `_DEFAULT`	`alloy,echo,fable,onyx,nova,shimmer` / `alloy`	Allow-list

`AI_VOICE_TIMEOUT` / `AI_VOICE_RATE_LIMIT`	`120` / `30`	
`AI_OPENAI_MODELS`	`gpt-5.1,gpt-5-mini`	Catalogue entries for OpenAI
`AI_OPENAI_VISION_MODELS`	*(empty)*	**Opt-in** vision support

23.11 Remote MCP server

VARIABLE	DEFAULT	PURPOSE
`MCP_SERVER_ENABLED`	`true`	`false` removes `/mcp` and the OAuth discovery and registration routes
`MCP_REDIRECT_DOMAINS`	`https://claude.ai,https://claude.com,https://chatgpt.com,http://localhost`	Where a registering AI client may send the authorization code
`MCP_RATE_LIMIT`	`60`	Tool calls per minute, per person
`MCP_OAUTH_RATE_LIMIT`	`120`	Discovery / registration requests per minute, per IP
`MCP_ACCESS_TOKEN_MINUTES`	`60`	Access-token lifetime
`MCP_WRITES_ENABLED`	`true`	`false` takes every MCP write tool away — the server is read-only again — without a deploy
`MCP_WRITE_RATE_LIMIT`	`30`	Changes per minute, per person, through MCP write tools
`MCP_REFRESH_TOKEN_DAYS`	`30`	Refresh-token lifetime — how long an unused connection survives
`PASSPORT_PRIVATE_KEY` / `PASSPORT_PUBLIC_KEY`	*(none)*	Token signing keys (PEM). **Required in production**; locally `php artisan passport:keys` writes `storage/oauth-*.key` instead. Rotating them disconnects every client

23.12 GitHub

VARIABLE	DEFAULT	PURPOSE
`GITHUB_TOKEN`	—	`<GITHUB_TOKEN>`. Needs `contents: write` + `pull_requests: write`. **Never merge rights**
`GITHUB_WEBHOOK_SECRET`	—	`<GITHUB_WEBHOOK_SECRET>`. **Unset means every delivery is refused**
`GITHUB_API_URL`	`https://api.github.com`	
`GITHUB_REQUEST_TIMEOUT`	`20`	

`GITHUB_WEBHOOK_RATE_LIMIT`	`300`	Per minute per IP
`GITHUB_WEBHOOK_RETENTION_DAYS`	`30`	
`GITHUB_MAX_REFERENCES` / `GITHUB_MAX_COMMITS`	`50` / `20`	
`GITHUB_QUEUE` / `_JOB_TRIES` / `_JOB_TIMEOUT`	`default` / `3` / `60`	

23.13 Slack and SMS

VARIABLE	DEFAULT	PURPOSE
`SLACK_NOTIFICATIONS_ENABLED`	`true`	Master switch. Turn **off** on staging restored from a production dump
`SLACK_QUEUE` / `SLACK_TIMEOUT` / `SLACK_TRIES`	`default` / `8` / `3`	Webhook URLs live per board, encrypted
`SMS_ENABLED`	`false`	
`SMS_DRIVER`	`unavailable`	`unavailable`, `log`, or `46elks`
`SMS_FROM`	`Aqueduct`	Alphanumeric (≤11 chars) or E.164
`SMS_46ELKS_USERNAME` / `_PASSWORD`	—	`<SMS_46ELKS_PASSWORD>`
`SMS_ALERT_RECIPIENTS`	*(empty)*	Comma-separated E.164 fallback rota
`SMS_MAX_RECIPIENTS`	`5`	Hard ceiling
`SMS_BOARD_COOLDOWN`	`300`	Seconds
`SMS_MAX_LENGTH`	`300`	
`SMS_RETENTION_DAYS`	`90`	**Data-protection obligation** — rows hold phone numbers

23.14 Mail, logging and seeding

VARIABLE	DEFAULT	PURPOSE
`MAIL_MAILER`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_SCHEME`	—	Password-reset delivery
`MAIL_FROM_ADDRESS` / `MAIL_FROM_NAME`	— / `${WORKSPACE_NAME}`	
`LOG_CHANNEL` / `LOG_STACK` / `LOG_LEVEL`	`stack` / `single` / `debug`	Use `error` or `warning` in production
`ACTIVITY_LOGGER_ENABLED`	`true`	Workspace activity feed

`SEED_ADMIN_NAME` / `_EMAIL` / `_PASSWORD`	—	**A password must be supplied explicitly outside local/testing**
`SEED_TEAM_*`, `SEED_CUSTOMER_*`	—	Demo accounts

23.15 Deployment-only

VARIABLE	PURPOSE
`PORT`	Injected by Railway; substituted into `nginx.conf` at boot
`RUN_MIGRATIONS`	**Set to `true` on exactly one service**
`RUN_SEEDERS`	Run seeders at boot. Leave unset in production after first run

SECTION 24

Installation and Local Development

24.1 Requirements

REQUIREMENT	VERSION
PHP	8.2 or later, with `bcmath`, `gd`, `intl`, `pdo_mysql`, `zip`
Composer	2.x
Node.js	22 recommended (the image uses `node:22-alpine`)
npm	Bundled with Node
MySQL	8.x — or SQLite for a quick start
Git	For the repository, and for AI apply mode

Optional: Redis, the Claude Code CLI (apply mode), an Anthropic and/or OpenAI API key.

24.2 Quick start

The repository defines a `setup` script that does most of it:

```
git clone <REPOSITORY_URL> nexora
cd nexora
composer setup
```

`composer setup` runs, in order:

```
composer install
php -r "file_exists('.env') || copy('.env.example', '.env');"
php artisan key:generate
php artisan migrate --force
php artisan db:seed --force
npm install
npm run build
```

24.3 Step by step

1. Clone and install PHP dependencies

```
git clone <REPOSITORY_URL> nexora
cd nexora
composer install
```

2. Install front-end dependencies

```
npm install
```

3. Create the environment file

```
cp .env.example .env
```

4. Generate the application key

```
php artisan key:generate
```

5. Configure the database

Edit `.env`:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=nexora
DB_USERNAME=nexora
DB_PASSWORD=<DB_PASSWORD>
```

6. Run migrations

```
php artisan migrate
```

7. Seed

```
php artisan db:seed
```

SEEDER	WHAT IT CREATES	SAFE IN PRODUCTION?
`AdminUserSeeder`	The first administrator, from `SEED_ADMIN_*`. Idempotent	**Yes** — credentials come from the environment, and outside local/testing a password must be supplied explicitly
`DemoUserSeeder`	Three known accounts, one per role	**No** — see the warning below
`DevelopmentSeeder`	Two sample boards with a deliberately asymmetric fixture	**No** — refuses to run in production

> **WARNING**
>
> `DemoUserSeeder` **has its credentials in the source, on purpose and with eyes open.** The workspace was being brought up as a demo, and "set four environment variables, then find a shell" was the wrong price for three demo logins. The cost is equally plain: anybody who can read the repository can read those passwords, and they are far below what the application itself would accept from a form — production requires 12+ characters, mixed case, and a breach check, and seeders bypass form validation, which is the only reason they work.
>
> **Change them from Admin → Users before anything real lands**, at which point the UI holds you to the strong rules.

`DevelopmentSeeder` creates a fixture designed to check the access rules by hand:

BOARD	PREFIX	MEMBERS	PURPOSE
Aqueduct Platform	`AQD`	team, customer	Mixes internal and customer-visible tickets, so the two roles see different boards at one URL
Internal Tooling	`INT`	team only	**The customer must get a 404 here, not a 403**

8. Build the front end

```
npm run build     # production
npm run dev       # watch mode
```

9. Run everything

The repository defines a single command that starts all five processes:

```
composer dev
```

which runs concurrently:

```
php artisan serve
php artisan queue:listen --queue=ai,default --tries=1 --timeout=0 --sleep=1
php artisan reverb:start
php artisan pail --timeout=0
npm run dev
```

Or start them individually:

```
php artisan serve                                    # http://127.0.0.1:8000
php artisan queue:work --queue=ai,default,attachments      # REQUIRED for most features
php artisan reverb:start                             # realtime (optional)
php artisan schedule:work                            # housekeeping (optional)
npm run dev                                          # asset watcher
```

> **IMPORTANT**
> Include `attachments` in the local worker's queue list. In production that queue is served by a separate worker inside the web container; locally there is only one process, so omitting it means AI file uploads sit at `pending` for ever.

24.4 Code style

```
composer lint         # vendor/bin/pint — fixes
composer lint:test    # vendor/bin/pint --test — checks only
```

24.5 Useful environment settings for development

```
APP_ENV=local
APP_DEBUG=true
FILESYSTEM_DISK=local
BROADCAST_CONNECTION=null        # skip Reverb
AI_ENABLED=false                 # no provider calls
SMS_DRIVER=log                   # log instead of sending
SLACK_NOTIFICATIONS_ENABLED=false
```

Outside production, `Model::shouldBeStrict()` is on, so N+1 queries, missing attributes and silently discarded mass-assignment values all fail loudly.

SECTION 25

Testing

25.1 Suites

Three suites are declared in `phpunit.xml`.

SUITE	DIRECTORY	FILES	PURPOSE
Unit	`tests/Unit`	17	Pure logic: enums, value objects, parsers, cost calculation
Feature	`tests/Feature`	123	End-to-end behaviour through routes and Livewire components
Security	`tests/Security`	33	**Authorization and isolation, runnable alone as a fast CI gate**

Test environment: SQLite in memory, `array` cache and session, `sync` queue, `null` broadcasting, `array` mail, `BCRYPT_ROUNDS=4`, public registration off, and `AI_CODE_DRIVER=unavailable` pinned so the suite tests the **default** driver rather than whatever a developer has in their `.env`.

25.2 Running the tests

```
composer test                              # config:clear, then php artisan test
composer test:security                     # the Security suite alone
php artisan test --testsuite=Unit
php artisan test --testsuite=Feature
php artisan test tests/Security/CustomerBoundaryTest.php
php artisan test --filter=test_a_customer_cannot_see_internal_tickets
```

25.3 Current status

Measured on the board tabs branch on 22 October 2026:

RESULT	COUNT
Passed	**2,137**
Failed	2
Assertions	8,339

The two failures are the `ThemePreferenceTest` pair below. The third test in the table is date-sensitive: it passes on days whose report period contains no `7`, as it did on this run. A fourth, `AiAttachmentSecurityTest > the stored path reveals nothing about the upload`, failed about one

run in forty because the random file name it inspected could itself contain the letters it looked for; it checks the name's shape instead since `7d13980`.

The known failures

> **IMPORTANT**
> All three are **test defects, not application defects.** None indicates a security or functional regression.

TEST	CAUSE
`Feature\Theme\ThemePreferenceTest > a page renders the stored preference before the stylesheet`	Pre-existing
`Feature\Theme\ThemePreferenceTest > system is rendered as system rather than resolved`	Pre-existing
`Security\AiVisualizationSecurityTest > a customers totals exclude internal tickets`	**Date-sensitive assertion**

The third is worth explaining, because at a glance it reads like a customer-isolation failure. The test creates 2 customer-visible and 5 internal tickets, then asserts three things. The **two that matter both pass**:

- the customer's answer contains `(total 2)` — the internal rows never entered the count;
- the staff answer contains `(total 7)`.

The failing assertion is the third, a belt-and-braces check that the customer's answer does not contain the digit `7` anywhere, so the hidden count could not be recovered by subtraction. The answer's header reads:

```
by_status for Dolore Expedita, 2026-08-19 to 2026-09-17 (30 days, UTC).
```

The report period ends on today's date — **2026-09-17** — which contains a `7`. The assertion therefore fails on any day whose rendered period string contains that digit, and passes otherwise. **The customer boundary itself is intact and is proven by the first assertion in the same test.**

The fix is to narrow the assertion to the figure rather than the character — for example asserting the absence of `(total 7)` — rather than to change any application behaviour.

25.4 What is covered

Security suite — the customer boundary

TEST FILE	ASSERTS
`CustomerBoundaryTest`	The overarching internal/customer separation
`TicketVisibilityTest`, `TicketAuthorizationTest`	Internal tickets are invisible and unwritable to customers

`CommentVisibilityTest`	Internal notes, their count and their existence are hidden
`DocumentationVisibilityTest`	The ancestor rule holds
`BoardVisibilityTest`, `BoardMembershipTest`	Non-members get 404, never 403
`AttachmentVisibilityTest`	Files inherit their owner's visibility
`StatisticsVisibilityTest`, `StatisticsExportSecurityTest`	Aggregates and CSVs cannot leak internal figures
`GlobalSearchVisibilityTest`	The palette cannot surface unreachable rows
`NotificationVisibilityTest`	No notification is written for an ineligible recipient
`ActivityVisibilityTest`	The feed refuses non-staff in SQL
`BoardTabBoundaryTest`	A customer may pick a tab when raising a ticket but still lands in its first column; tab counts never include internal tickets; a tab slug or id from another board resolves to nothing on the board, the forms, Settings, the AI's page hint and MCP
`RealtimeAuthorizationTest`, `RealtimeBroadcastAudienceTest`	Channel joins and broadcast audiences
`RoleAccessTest`	Route-level role gates

Security suite — AI

TEST FILE	ASSERTS
`AiAssistantSecurityTest`, `AiChatSecurityTest`	Conversation ownership and scope
`CustomerAiSecurityTest`	A customer's assistant is read-only and staff tools are withheld
`McpCustomerBoundaryTest`	A customer's AI app (remote MCP) gets the customer tools only, never sees an internal ticket or note through any tool, and can change only what the ticket screen lets them: raise a ticket, edit their own, reply on the customer thread
`AiWriteAuthorizationTest`	Proposals are authorized against the confirming person, per field
`AiVisibilityTest`	AI runs are invisible to customers
`AiVisualizationSecurityTest`	Charts cannot aggregate hidden data
`AiKnowledgeScopeSecurityTest`	Outside Project does not widen data access
`AiAttachmentSecurityTest`	Uploads are private to their conversation
`AiCredentialSecurityTest`	Keys are never rendered or returned
`SecretExposureTest`	**No credential reaches a response, a log line or an exception message**
`WebhookSecurityTest`	Signature verification, replay handling, fail-closed behaviour
`RichTextSanitizationTest`	The editor allow-list holds in both directions

Feature suite highlights

AREA	FILES
AI	40 files — board tabs (context, the tab on screen, strict per-tab columns), capability modes, chat modes, knowledge scope, sessions, context window and prompt caching, tools, attachments, voice, rich responses, apply mode, run caps, inheritance, Anthropic and OpenAI providers, token usage, audit log, batch tickets, importing a file into the docs, tickets from the page on screen
MCP	6 files — the endpoint and both protocol generations, OAuth and the consent screen, every write tool (confirmation codes, retries, stale reads, the AI mode, archived boards, the write limit, the kill switch), the display rules and badges with the in-app output held unchanged, the reconnect migration, the `tab` argument and the tab-aware board output
Tickets	16 files — creation, editing, movement, moving between tabs, filtering, links, subtasks, attachments, events, rich text, checklist conversion, quick add, live conversation, the Tab choice on the ticket and create pages
Documentation	8 files — editor, save state, drag and drop, tree, workspace, visibility, importing files as pages, the file-to-Markdown converters
Boards	14 files — management, deletion, columns, labels, integrations, drag and drop; for tabs: the migration of existing boards (run twice, rolled back and resumed), the tab actions and their locking, the tab strip, managing tabs from the board, dropping a card on a tab, per-tab columns in Settings, and "Tab › Column" labels in statistics, notifications, Slack and search
Statistics	6 files — team, customer, flow metrics, exports, chart export markup
Notifications	4 files — bell, ticket notifications, Slack, critical-ticket SMS
Auth	3 files — authentication, password reset, registration
Activity	4 files — feed, logging, cards, unified timeline
Other	GitHub webhook processing, health check, profile, dashboard, command palette, theme, UI navigation, dialogs, attachment durability, demo seeder

25.5 Test support

HELPER	PURPOSE
`FakeAiProvider`	Scripted model responses — **the suite never calls a real provider**
`UnreachableAiProvider`	Asserts that a code path does *not* reach a provider

`FakeExternalKnowledge`	Stands in for the absent knowledge driver
`FakeSmsProvider`	Captures messages instead of sending them
`FakeFiles`	Deterministic uploads

14 model factories cover users, boards, board tabs, columns, members, labels, tickets, subtasks, repositories, GitHub links, AI runs, AI sessions, AI attachments and AI usage records. A column made by the factory goes into its board's first tab, which is created as "Main" when the board has none; `inTab()` picks another. `TestCase` adds `tabOn()` (a tab made through the real `CreateTab`), `defaultTab()` and an optional tab on `columnNamed()`.

25.6 Gaps

GAP	NOTE
Browser / end-to-end tests	**None.** No Dusk, Playwright or Cypress. Drag-and-drop, the TipTap editor, the command palette and theme switching are tested only at the server boundary. Board tabs were walked through once in headless Chrome before release (switching, back/forward, the keyboard, dropping a card on a tab, deleting a tab, a 390 px phone width, the customer view), but that script is not part of the repository
JavaScript unit tests	**None.** No Vitest or Jest; `resources/js` is untested in isolation
Load or performance tests	None
Mutation testing	None
Coverage reporting	Configured (`<source>` includes `app`) but no threshold is enforced and no CI configuration is present in the repository

SECTION 26

Troubleshooting

Each entry gives the **problem**, the **cause**, the **solution** and how to **verify**.

26.1 APP_KEY is missing

Problem — The container exits immediately with `[entrypoint] FATAL: APP_KEY is not set`, or pages return a decryption error.

Cause — `APP_KEY` is unset. The entrypoint refuses to start rather than booting an application that cannot decrypt sessions, Slack webhook URLs or stored AI credentials.

Solution

```
php artisan key:generate --show      # prints base64:...
```

Set it as an environment variable on every service.

Verify — The container starts and `/health` returns 200.

> **WARNING**
> **Never rotate `APP_KEY` casually on a running deployment.** It decrypts Slack webhook URLs and stored AI credentials. Rotating it makes both unreadable — and by design they degrade to "not configured" rather than throwing, so the symptom is **silence**: Slack simply stops posting and the AI simply stops working.

26.2 Database connection failure

Problem — `/health` reports `database: false`; every page 500s.

Cause — Wrong credentials, wrong host, or the database is not reachable from the container.

Solution — Check `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`. On Railway, use the private network host rather than the public proxy.

Verify

```
php artisan tinker --execute="DB::connection()->getPdo(); echo 'ok';"
curl -s https://<APP_URL>/health
```

26.3 Migration fails with "Base table or view not found: cache_locks"

Problem — The first deploy against an empty database restart-loops.

Cause — `migrate --isolated` takes its lock from the default cache store, which on the `database` store lives in `cache_locks` — a table created by a migration. On an empty database the lock cannot be

taken, because the table it needs is one of the tables it is guarding.

Solution — The entrypoint already handles this: it probes with `migrate:status` and runs the first migration **unisolated**. If you are migrating by hand, do the same:

```
php artisan migrate --force            # first run, empty database
php artisan migrate --force --isolated # every run after
```

Verify — `php artisan migrate:status` lists every migration as ran.

26.4 The queue is not processing

Problem — AI runs stay "Queued" for ever, Slack messages never arrive, no SMS is sent, pull requests never link to tickets, and realtime updates never appear. **Nothing errors.**

Cause — No worker, or a worker listening to the wrong queues. A bare `queue:work` listens only to `default`.

Solution

```
php artisan queue:work --queue=ai,default --tries=3 --max-time=3600
```

In production, the worker service runs that command and the **web** container separately runs `--queue=attachments`. Locally, include all three.

Verify

```
php artisan queue:monitor default,ai
php artisan tinker --execute="echo DB::table('jobs')->count();"
```

26.5 AI attachments stay "pending" for ever

Problem — Uploaded files never reach `ready`, or settle into `failed` with no exception and nothing in the log.

Cause — Either no worker is serving the `attachments` queue, or — on a volume-backed disk — the worker is in the **wrong container**. A Railway volume attaches to exactly one service; the upload request writes the file in the web container, and a worker elsewhere asks its own `/data` and finds nothing. A missing object is a legitimate condition in the reader, so it fails silently.

Solution — Keep the `attachment-worker` supervisor program in the **web** container. If the disk is S3, you may instead set `AI_ATTACHMENTS_QUEUE=default` and add it to the worker service — but never serve it from both.

Verify

```
php artisan tinker --execute="echo App\Models\AiAttachment::where('status','pending')->count();"
```

26.6 AI runs are stuck at "Running"

Problem — A ticket shows a run in progress indefinitely, blocking further runs.

Cause — The worker died mid-run: a deploy, a container restart or an OOM. The run is owned by the job that claimed it, so nothing can finish it.

Solution

```
php artisan ai:reap-stalled
```

It runs automatically every ten minutes and refuses to touch a run that still has a job on the queue.

Verify — The run moves to `failed` with a reason, and a new run can be started.

26.7 AI runs are refused

Problem — The button is disabled, or an internal note says the run was refused.

Causes and fixes

MESSAGE	CAUSE	FIX
"...set to AI Observer... writes nothing"	Capability mode too low	Raise to AI Operator on `/admin/ai` or the board
"Apply mode... needs AI Agent"	Apply requires the top mode	Raise to AI Agent
"Automatic runs... need AI Agent"	Unattended work requires AI Agent	Raise, or run manually
Cap reached	Daily run cap	Wait for the board's own midnight, or raise `daily_auto_run_cap`
Coding runtime unavailable	`AI_CODE_DRIVER=unavailable`	Set `AI_CODE_DRIVER=claude_code`, ensure the binary and `ANTHROPIC_API_KEY` are in the **worker's** environment
No credential	No key configured	Set one in `/admin/ai`, or `ANTHROPIC_API_KEY`

26.8 GitHub webhook returns 401

Problem — GitHub shows a red cross; no links appear on tickets.

Cause — `GITHUB_WEBHOOK_SECRET` is unset or does not match the repository's webhook. **Unset means every delivery is refused** — that is the only safe default.

Solution — Set the same secret in both places. Deliver only `application/json`.

Verify — Redeliver from GitHub's webhook settings; expect **202 Accepted**. A ping should return 200.

26.9 GitHub webhook returns 202 but nothing links

Problem — Deliveries succeed, no `github_links` rows appear.

Causes

CAUSE	FIX
The repository is not attached to any board	Add it on `/boards/{board}/ai/settings`. **This is the security boundary** — an unattached repository resolves to no boards at all
No ticket key in the branch name or commit messages	Use `NL-42` style references
The prefix does not match a board	Check the board's `ticket_prefix`
The queue worker is not running	See above
The event is not one of the five understood	Only `push`, `create`, `pull_request`, `check_suite`, `status`

Verify

```
php artisan tinker --execute="print_r(App\Models\WebhookDelivery::latest()->take(5)->get(['event','status','links_written','error'])->toArray());"
```

26.10 Slack notifications never arrive

Problem — No messages in the channel.

Causes

CHECK	WHERE
`SLACK_NOTIFICATIONS_ENABLED=true`	Environment
The board's master switch is on	`/boards/{board}/integrations`
The webhook URL is configured	Same screen — it is **write-only**, so it shows "configured", never the value
The board subscribes to that event	Same screen. `ai_run_finished` is **off** by default
A worker serves `default`	See above
`APP_KEY` has not been rotated	Rotation makes stored URLs unreadable, degrading silently to "not configured"

Verify — `SlackNotifier` never throws, so failures appear only in the log. Check the application log for Slack delivery errors.

26.11 Storage configuration failure

Problem — Production refuses to boot, with a durability error.

Cause — `FILESYSTEM_DISK` / `ATTACHMENT_DISK` is `local`, or is `volume` while the volume is **not actually mounted**.

Solution — Use `s3` (any S3-compatible provider), or mount a Railway volume and set `ATTACHMENT_VOLUME_PATH` to its path.

Verify — The container boots. Upload a file and download it back.

> **NOTE**
> This refusal is deliberate. Writing attachments into the container would lose every file at the next deploy — silently — which is the exact failure the check exists to prevent.

26.12 File uploads return 500, or silently vanish

Problem — Small forms work; uploading a file fails. In Livewire the composer simply empties with no error.

Cause — nginx buffers any request body larger than `client_body_buffer_size` to disk, and the temp paths must be writable by `www-data`. The Alpine package's default paths are owned by the `nginx` user instead.

Solution — The Dockerfile and entrypoint both create and chown `/var/cache/nginx/{client_body,fastcgi}`. If you have customised either, keep that. Also check `client_max_body_size` (32m), `upload_max_filesize` (32M) and `post_max_size` (34M) against your attachment ceiling.

Verify — Upload a file larger than 16 KB to a ticket.

26.13 Realtime never updates

Problem — Boards and the bell only refresh on reload.

Causes

CAUSE	FIX
The image was built without `VITE_REVERB_APP_KEY`	**Rebuild** — these are build arguments, not runtime variables
The Reverb service has no public domain	Browsers must reach it directly
`REVERB_HOST` / `PORT` / `SCHEME` point at the wrong place	Should be the public domain, 443, https
More than one Reverb replica	Reverb holds connections in process memory. Use one replica, or `REVERB_SCALING_ENABLED=true` plus Redis
`BROADCAST_CONNECTION=null`	Set it to `reverb`

No worker on `default`	Broadcasts are queued

Verify — The browser console shows a WebSocket connection; a change made in one tab appears in another.

26.14 Railway deployment fails the health check

Problem — The deploy never goes live.

Causes — `APP_KEY` missing (the entrypoint aborts); the database is unreachable; the storage durability check refuses; or `PORT` is not being honoured.

Solution — Read the deploy logs; the entrypoint prints each step. `/health` returns booleans for the database and cache.

Verify

```
curl -s https://<APP_URL>/health
curl -s https://<APP_URL>/up
```

26.15 Missing environment variables

Problem — A feature is quietly absent rather than broken.

Cause — This application's design is to **degrade honestly**: an unconfigured integration reports itself unconfigured rather than failing.

SYMPTOM	MISSING
AI refuses with "not configured"	`ANTHROPIC_API_KEY` / `OPENAI_API_KEY`, or a stored credential
The voice control is disabled	`OPENAI_API_KEY`
Audio attachments say transcription is not configured	`OPENAI_API_KEY`
Apply mode fails immediately	`AI_CODE_DRIVER`, `GITHUB_TOKEN`
Every webhook is refused	`GITHUB_WEBHOOK_SECRET`
Slack is silent	Per-board webhook URL
SMS records show `failed` with an explanation	`SMS_DRIVER`, `SMS_46ELKS_*`
External knowledge lookups say they are unavailable	`AI_KNOWLEDGE_DRIVER` — **no driver ships**

26.16 Configuration changes have no effect

Problem — An environment variable was changed but nothing behaves differently.

Cause — The entrypoint caches configuration, routes, views and events at boot.

Solution — Restart the service. Locally:

```
php artisan config:clear && php artisan route:clear && php artisan view:clear
```

Verify — `php artisan tinker --execute="echo config('ai.modes.default');"`

SECTION 27

HTTP Endpoints and Webhooks

IMPORTANT

Nexora exposes no general REST or JSON API for third parties. There is no `routes/api.php` and no OpenAPI specification. The one machine-facing interface is the **remote MCP server** at `POST /mcp`: AI clients such as Claude and ChatGPT call it with an OAuth token on behalf of one signed-in person, to read and to make the changes that person could make by hand, under their in-app permissions. Everything else below is a browser route, an inbound webhook, or a small number of controller endpoints the application's own front end calls.

27.1 Remote MCP server — `POST /mcp`

Defined in `routes/ai.php`, which laravel/mcp loads **outside** the `web` group: no session, no CSRF, a bearer token on every request. `MCP_SERVER_ENABLED=false` removes every route in this section.

ROUTE	PURPOSE	AUTHENTICATION	LIMIT
`POST /mcp`	MCP JSON-RPC: `initialize`, `tools/list`, `tools/call`	Passport token (`auth:api`), scope `mcp:use`, active user	`throttle:mcp` — per person; write tools also `MCP_WRITE_RATE_LIMIT` a minute
`GET` / `DELETE /mcp`	Always `405` (no server-initiated stream)	—	—
`GET /.well-known/oauth-protected-resource[/mcp]`	RFC 9728 metadata: `resource` is exactly `https://<host>/mcp`	Public	`throttle:mcp-oauth` — per IP
`GET /.well-known/oauth-authorization-server[/…]`	RFC 8414 metadata: endpoints, S256 PKCE, scope	Public	`throttle:mcp-oauth`
`POST /oauth/register`	Dynamic client registration (RFC 7591), public clients only, redirect URIs limited to `config('mcp.redirect_domains')`	Public	`throttle:mcp-oauth`
`GET /oauth/authorize`	Nexora's login if needed, then the consent screen (`resources/views/mcp/authorize.blade.php`)	Web session	—
`POST` / `DELETE /oauth/authorize`	Approve / deny	Web session + CSRF	—
`POST /oauth/token`	Code exchange (PKCE S256) and refresh-token rotation	Client	Passport's own

The handshake. An unauthenticated `POST /mcp` is always a JSON `401` with `WWW-Authenticate: Bearer realm="mcp", resource_metadata="…/.well-known/oauth-protected-resource/mcp"`, whatever

the client sends in `Accept` — `bootstrap/app.php` forces JSON rendering for these paths, because a redirect to the login page here is what makes a client report that it "couldn't reach the server".

The tools are the in-app assistant's read tools, through `AiToolRegistry::invoke()`, and the write tools in `App\Mcp\Tools\Write` (staff: tickets, checklists, comments, doc pages, delete after confirming; customers: raise a ticket, edit their own, reply), which go through `ExecuteChatAction::perform()` — see section 9 for each tool, who gets it, the confirmation policy and the display rules. Every call writes an `ai_tool_invocations` row with `channel = mcp`; a write's category is `action`. `MCP_WRITES_ENABLED=false` removes the write tools without a deploy.

27.2 Inbound webhook — GitHub

`POST /webhooks/github`

PROPERTY	VALUE
Authentication	**HMAC-SHA256** over the raw body, in `X-Hub-Signature-256`
CSRF	Exempt — the only exempt path in the application
Rate limit	`throttle:github-webhooks` — 300/min per IP, applied **before** anything else runs
Content type	`application/json`
Purpose	Link branches, commits, pull requests and CI results to tickets

Required headers

HEADER	PURPOSE
`X-Hub-Signature-256`	`sha256=<hex digest>`. **Required**
`X-GitHub-Event`	Event name
`X-GitHub-Delivery`	Delivery id, used for de-duplication

Events acted on: `push`, `create`, `pull_request`, `check_suite`, `status`. `ping` is answered 200. Anything else is acknowledged and ignored.

Responses

STATUS	BODY	MEANING
`200`	`Webhook configured.`	A `ping`
`200`	`Already received.`	A replay — the original delivery and its links stand
`202`	`Accepted.`	Verified, recorded, queued
`202`	`Event ignored.`	Not one of the five
`202`	`Malformed payload.`	Body was not valid JSON

`401`	`Invalid signature.`	**Identical** whether the secret is unset, the header is missing or the digest is wrong
`429`	—	Rate limited

> **NOTE**
>
> Everything that is not a signature failure answers **2xx**, including events we ignore and payloads we cannot parse. GitHub retries a non-2xx and shows a red cross in the repository settings, and neither helps for a delivery that will never succeed. A bad signature is the one case that genuinely is the caller's fault.
>
> The controller does **no parsing beyond reading two fields, no ticket lookup and no writing to ticket history.** That is partly latency — GitHub disables an endpoint that repeatedly takes more than ten seconds — and partly blast radius: the less this method does with unverified-shaped input, the less there is to get wrong in the request a stranger can make.

Setting it up

1. Repository → Settings → Webhooks → Add webhook.
2. Payload URL: `https://<APP_URL>/webhooks/github`
3. Content type: `application/json`
4. Secret: the value of `GITHUB_WEBHOOK_SECRET`
5. Events: pushes, branch/tag creation, pull requests, check suites, statuses.

27.3 Health endpoints

ROUTE	AUTH	RETURNS
`GET /up`	None	Framework probe. Used by the Docker `HEALTHCHECK`
`GET /health`	None	`{ database: bool, cache: bool, migrations_pending: bool, timestamp }`

Deliberately returns booleans and a timestamp only — no hostnames, no versions, no driver names, no error text.

27.4 Application endpoints

These are called by Nexora's own front end. All require an authenticated session.

`GET /attachments/{attachment}`

Authorized through `AttachmentPolicy`, which delegates to the owning record's policy. Redirects to a signed URL (S3 disks, 5-minute TTL) or streams the file (local disk). Denials are 404.

`GET /stats/export?dataset=…&board=…&range=…&from=…&to=…`

Team CSV. `role:admin,team`, re-checked in the controller. `dataset` is an **allow-list** — anything not in `KEYS` is refused rather than reflected.

`GET /stats/customer/export`

Customer CSV. Open to any authenticated user, because it can only reach `CustomerStatisticsExport` → `CustomerStatistics`, neither of which can see a team figure.

`GET /ai/messages/{message}/blocks/{block}/export.csv`

Exports one table or chart out of an assistant answer. Both parameters are constrained to digits, so a malformed URL is a router 404 rather than a cast to zero.

Four authorization filters: the route group's `role:admin,team`; `visibleTo`; `ownedBy` (**an administrator cannot export a colleague's conversation**); and a non-null session id.

`POST /ai/voice/listen`

A recording in, the transcript out. `throttle:ai-voice` (30/min per user). Allowed to **anyone eligible for the assistant panel, customers included** — transcribing reads nothing from the workspace.

`GET /ai/messages/{message}/speech`

A stored assistant turn, as audio. `throttle:ai-voice`. Resolved through `visibleTo` **and** `ownedBy`. A `GET` so an `<audio>` element can play it directly, and addressed by message id rather than by posting text — what is read aloud is what is stored.

`PUT /settings/theme`

Persists the appearance preference. Validated against `ThemePreference`. `PUT` because it replaces one value on the signed-in person's own row; there is no user id in the payload.

`POST /logout`

`POST /broadcasting/auth`

Registered by `withBroadcasting()`, behind the `web` middleware. Authorizes a channel subscription with the caller's session, decided by `routes/channels.php`. **A browser cannot join a channel merely by knowing its name.**

27.5 Browser routes

ROUTE	COMPONENT	GATE
`GET /`	redirect to `/dashboard`	—
`GET /login`	`Auth\Login`	guest

`GET /register`	`Auth\Register`	guest; **404 unless** `ALLOW_PUBLIC_REGISTRATION`
`GET /forgot-password`	`Auth\ForgotPassword`	guest
`GET /reset-password/{token}`	`Auth\ResetPassword`	guest
`GET /confirm-password`	`Auth\ConfirmPassword`	auth
`GET /dashboard`	`Dashboard\Index`	auth
`GET /boards`	`Boards\Index`	auth
`GET /boards/create`	`Boards\ManageBoard`	`role:admin` + confirm
`GET /boards/{board}`	`Boards\Show`	auth + `BoardPolicy::view`. `?tab=<slug>` opens that tab; without it, the tab this person last had open there, else the default tab
`GET /boards/{board}/edit`	`Boards\ManageBoard`	`role:admin` + confirm
`GET /boards/{board}/settings`	`Boards\Settings`	auth + `manageColumns`. `?tab=<slug>` picks the tab whose columns are edited
`GET /boards/{board}/ai`	`Ai\Chat`	`role:admin,team` + `useAiChat`
`GET /boards/{board}/ai/settings`	`Boards\AiSettings`	`role:admin,team` + `manageAiSettings`
`GET /boards/{board}/integrations`	`Boards\Integrations`	`role:admin,team` + `manageAiSettings`
`GET /boards/{board}/tickets/create`	`Tickets\Create`	auth + `TicketPolicy::create`. `?tab=<slug>` starts the form in that tab
`GET /boards/{board}/tickets/{number}`	`Tickets\Show`	auth + `TicketFinder`
`GET /boards/{board}/docs`	`Docs\Show`	auth + `DocPagePolicy::viewAny`
`GET /boards/{board}/docs/{slug}`	`Docs\Show`	auth + `DocPageFinder`
`GET /activity`	`Activity\Index`	`role:admin,team`
`GET /stats`	`Stats\Team`	`role:admin,team`
`GET /stats/customer`	`Stats\Customer`	auth
`GET /settings`	`Settings\Index`	auth
`GET /settings/profile`	`Profile\UpdateProfile`	auth
`GET /admin/ai`	`Ai\GlobalSettings`	`role:admin` + confirm
`GET /admin/users`	`Users\Index`	`role:admin` + confirm
`GET /admin/users/create`	`Users\ManageUser`	`role:admin` + confirm
`GET /admin/users/{user}/edit`	`Users\ManageUser`	`role:admin` + confirm

Route ordering is deliberate: literal segments such as `/boards/create`, `/boards/{board}/ai` and `/boards/{board}/settings` are declared **before** `/boards/{board}`, so a literal is never treated as a slug.

Board tabs add no route: switching, adding, renaming, reordering and removing tabs, and dropping a card on a tab, are Livewire actions of `Boards\Show`, each re-authorized. A `?tab=` slug is only ever resolved within the board in the URL; anything else falls back to the board's default tab.

27.6 Outbound HTTP

TARGET	CLIENT	PURPOSE	TIMEOUT
Anthropic	`anthropic-ai/sdk` via `ClaudeService`	Chat, analysis	300s
OpenAI `/chat/completions`	`Http` via `OpenAiService`	Chat	300s
OpenAI audio	`OpenAiVoiceProvider`, `AudioTranscriber`	Transcription, speech	120s / 180s
GitHub `POST /repos/{owner}/{repo}/pulls`	`PullRequestClient`	Open a **draft** pull request	60s
Slack incoming webhook	`SlackWebhookChannel`	Board notifications	8s
46elks `POST /a1/sms`	`ElksSmsProvider`	SMS alerts	15s

SECTION 28

Administrator Guide

Written for the person who runs the Nexora deployment.

28.1 Your responsibilities

AREA	WHAT ONLY YOU CAN DO
Users	Create, edit, deactivate, change roles
Boards	Create, rename, delete, manage membership
AI credentials	Hold the provider API keys
AI trust level	Set the workspace capability mode
Spending	Set token limits and run caps

Everything else — tabs, columns, labels, documentation, board AI settings, integrations — is the delivery team's own work on their own boards.

28.2 Managing users

`/admin/users` — requires re-entering your password.

TASK	NOTES
Create	Name, email, role, initial password. Production passwords need 12+ characters, mixed case, and are checked against known breaches
Change a role	A separate gate from editing, because it is the privilege-escalation path
Deactivate	Sets `deactivated_at`. **Takes effect on the person's very next request**, not at their next login
Delete	Preserves their history — tickets, comments and pages keep existing with a null creator

> **IMPORTANT**
> You **cannot change your own role** and **cannot delete your own account**. Both are explicitly forbidden, so the workspace cannot be left with no administrator.

28.3 Managing boards

`/boards/create` and `/boards/{board}/edit` — both require re-entering your password.

The slug and ticket prefix are derived automatically and are globally unique. New boards get one tab, **Main**, with the standard five columns; the team can split the board into more tabs from the "+" on its tab strip. Add members from the board edit screen; **membership is the access grant**, and there is nothing else to configure for access.

Adding a **customer** to a board lets them see that board's customer-visible tickets and published documentation, raise tickets, take part in the customer conversation, and use a read-only assistant. It grants nothing internal, ever.

28.4 Global AI settings

`/admin/ai` — `role:admin` plus password confirmation, and the component re-authorizes on mount, on every action and on every render.

SETTING	WHAT IT DOES
Enabled	Master switch for the workspace
Provider	Anthropic or OpenAI
Default model	Inherited by every board that does not override it
Capability mode	AI Observer / AI Operator / AI Agent — the trust ceiling
Session token limit	Per conversation
Daily user token limit	Per person per day
Credentials	Anthropic and OpenAI keys

Choosing a capability mode

CHOOSE	WHEN
AI Observer	You want answers and nothing else. No writes at all — not even an internal note
AI Operator	You want the AI to draft changes that a person confirms, and to run manual analyses
AI Agent	You want unattended automation: automatic runs, apply-mode pull requests, and reversible changes carried out as soon as somebody asks

Raising the mode **grants no user a capability they did not already have by hand**. It raises the ceiling on what the AI may attempt; every write is still authorized against the person who confirmed it.

> **NOTE**
>
> Credential fields are **write-only**. They start blank and are blanked again after every save. You can replace a key — which is recorded in `rotated_at` and `updated_by_id` — but nobody, including you, can read one back out through the browser.
>
> There is deliberately **no "test key" button**. It would spend money on every press and be pressable by anybody who reaches the page; the first real question already tells you whether the credential works, because the provider's own refusal is reported verbatim.

28.5 Monitoring

QUESTION	WHERE
Is the deployment healthy?	`GET /health`
What has been happening?	`/activity`
How much is the AI costing?	`/stats` — the AI section
Did a webhook arrive?	`webhook_deliveries`
Was the on-call rota reached?	`sms_messages`
What did the AI look up?	`ai_tool_invocations`
What failed?	`failed_jobs`, and the application log

28.6 Routine maintenance

TASK	FREQUENCY	HOW
Review users and deactivate leavers	Monthly	`/admin/users`
Review AI spend	Monthly	`/stats`
Check failed jobs	Weekly	`failed_jobs`
Rotate provider keys	Per your policy	`/admin/ai` — replace, never read
Verify the scheduler is running	After each deploy	Check that `workspace:prune` ran

SECTION 29

Team Member Guide

Written for the delivery team.

29.1 What you can do

You see every board you have been **added to**. Ask an administrator to add you; there is no way to join one yourself.

AREA	YOUR PERMISSIONS
Boards	Open the ones you are a member of
Columns and labels	Create, rename, reorder, delete — on your boards
Tickets	Full control: create, edit, move, assign, label, link, delete
Visibility	**You decide what the customer sees**
Conversations	Both streams — customer and internal
Documentation	Write, organise, publish
AI assistant	Full access, including write tools (mode permitting)
AI runs	Start suggest and apply runs
Board AI settings	Model, project context, auto-run policy, repositories
Integrations	Slack channel and SMS rota for your boards
Statistics and activity	Full team view
Board record	**Not yours** — name, prefix, membership and deletion are administrator-only

29.2 The one thing to get right

IMPORTANT

A ticket you raise is visible to the board's customers by default; comments and pages are not. On the New ticket form and in quick add, "Visible to the customer" starts ticked — **untick it for internal work** (the form says how many customers would see the ticket). The assistant follows the same default and makes a ticket internal when you ask it to.

A comment you write and a documentation page you add are internal until somebody deliberately publishes them. Publishing an existing ticket or page is staff-only and recorded in the timeline. A board can keep new tickets internal by default through its `new_tickets_visible_to_customers` setting.

The reverse also holds: a ticket a **customer** raises is always customer-visible, always lands in the first column of its tab, and is never pre-assigned.

29.3 Splitting a board into tabs

A board can hold several areas of work — "Admin dashboard", "Public website", "Backend API" — as **tabs**, each with its own columns and tickets, like tabs in a browser. Click a tab in the strip above the columns to switch; your filters stay on, and each tab shows how many tickets match them.

1. **Add a tab** with "+": name it, and it starts with the same columns as the tab you are on. Change its columns from the tab menu (⋯) → *Edit columns*.
2. **Rename, move left/right, delete** from the tab menu, or drag tabs along the strip. The leftmost tab is where the board opens and where tickets go when nobody picks a tab.
3. **Move a ticket to another tab** by dragging its card onto that tab, or with the **Tab** choice on the ticket page. It lands in the column with the same name — or a done column if it was done, or else the first column.
4. **Delete a tab** and its tickets move to the tab you choose — the panel shows where each column's tickets will go before anything happens. Tickets are never deleted with a tab.

A link to a board can name a tab (`…/boards/gritch?tab=backend-api`); without one, the board opens on the tab you last had open there.

29.4 Working a ticket

1. Pick it up from the board; drag it into *In Progress*.
2. Assign it to yourself.
3. Write your working notes in the **internal** stream — the customer cannot see them, cannot count them and cannot tell they exist.
4. Reply to the customer in the **customer** stream.
5. Create a branch named after the ticket key — `NL-42-fix-login`. The branch, its commits and its pull request appear on the ticket automatically.
6. Move it to *Done*; if the board subscribes, Slack is told.

29.5 Using the assistant

The panel is on every page. Choose what the conversation is about — one board, or the whole workspace — and what it is for:

MODE	USE IT FOR
Reading	"What is overdue?", "What changed on this board this week?", "What does the deployment runbook say?"
Writing	"Raise a ticket for the Safari login bug, high priority, assign it to me"

Everything	"What is missing from this board — and make tickets for it"

Tick **Outside Project** when you want general technical knowledge alongside the project. It does not widen what you can see by a single row.

You can attach files — specifications, CSVs, screenshots, recordings — and speak your question instead of typing it.

When the assistant proposes a change you get a preview and a Confirm button. Under **AI Agent**, reversible changes happen immediately because you asked for them; **deleting a ticket always requires typing its key by hand.**

> **NOTE**
> The assistant can only ever do what **you** could do by hand. If you cannot assign tickets, confirming a proposal will not assign one — and it will tell you why rather than partly succeeding.

29.6 Running the AI on a ticket

MODE	WHAT YOU GET
Suggest	An internal note with the AI's analysis of the ticket and the repository
Apply	An isolated clone, a new branch, your configured tests run, and a **draft** pull request for you to review

Apply mode **never merges, never pushes to a protected branch, and never opens a pull request for work that did not happen.** If validation fails, the run fails and nothing is pushed.

29.7 Documentation

Open a page and start typing — there is no Edit button. Where your work is saved depends on the audience:

- an **internal** page saves as you type;
- a **published** page saves to a draft and stays there until you press **Publish**, because a page a customer is reading should change when you say so.

You cannot publish a page whose parent is internal — publish the parent first, deliberately. Making a page internal hides its whole subtree.

SECTION 30

Customer Guide

Written for a customer stakeholder. No technical knowledge assumed.

30.1 What Nexora is, for you

Nexora is where you can see the work your delivery team is doing for you, raise new requests, and talk to the team about them — without waiting for a status meeting or an emailed report.

30.2 Signing in

Your delivery team creates your account and adds you to your project. Sign in with your email address and password; use **Forgot password** if you need to reset it. You can change your name, email, password and light/dark appearance under **Settings → Profile**.

30.3 Your dashboard

After signing in you see the projects you have been added to. Opening one shows its board: a set of columns, with a card for each piece of work, arranged left to right roughly in the order work flows. A large project may be split into **tabs** above the columns — one per area of work; click a tab to see its columns and the tickets in it that are shared with you.

30.4 Raising a request

1. Open your project (and the tab the request belongs in, if it has tabs).
2. Choose **New ticket**.
3. Give it a clear title, describe what you need, set a priority, and attach any screenshots. If the project has tabs, the form starts in the tab you were on; you can change it.
4. Save.

Your ticket is **immediately visible to you and to the team**, appears at the start of their workflow, and is left unassigned so the team can decide who picks it up. They are notified straight away — and for a **critical** request, the on-call engineer can be texted.

You can keep editing a ticket you raised: its title, description, priority, type and due date. Who is working on it and where it sits in the workflow are the team's decisions.

30.5 Following progress

Open any ticket to see its status, priority, labels, due date, the conversation, and its history.

30.6 Talking to the team

Each ticket has a conversation you take part in. Write a comment, attach a file, and mention a colleague with `@`. The team is notified.

> **NOTE**
>
> **You will not see everything on a ticket, and that is by design.** The delivery team keeps separate working notes — technical detail, internal discussion, engineering activity — that are not shown to you. This is not information being withheld from you dishonestly; it is the team being able to think out loud without every half-formed thought becoming a client communication. What is meant for you is in your conversation.

30.7 Documentation

Your project may include documentation the team has **published** to you: guides, processes, release notes, technical references. Pages are organised as a tree; there is a contents list for each page. Documentation is written by the team — you can read it, but not change it.

Some pages are internal and you will not see them. There is no indication that they exist.

30.8 Searching

Press **Ctrl + K** (**Cmd + K** on a Mac) anywhere to search your tickets, documentation and projects. Results only ever include things you can already open.

30.9 Your progress summary

Statistics gives you a plain summary of your own project: how many requests are open and closed, how they break down by priority, how many were raised over recent weeks, and recent activity. You can download the figures as a spreadsheet.

30.10 Asking the assistant

Nexora includes an AI assistant, available from any page. Ask it things like:

- *"What is the status of my open requests?"*
- *"Which of my tickets are highest priority?"*
- *"What does the documentation say about the release process?"*
- *"Show me how many requests I have raised each week this quarter."*

It can produce charts and tables from your own figures, and read them aloud. You can attach a file — a specification, a spreadsheet, a screenshot — and ask questions about it.

IMPORTANT

Your assistant is read-only, and it can only see what you can see.

- It **cannot change anything** — no tickets, no comments, no documentation. Not in any configuration, ever.
- It **cannot see internal work**: internal tickets, the team's private notes, internal documentation, engineering activity or repository information are all invisible to it, exactly as they are to you.
- Its numbers count **only the work you can open**, so a chart it draws for you is correct for your view.
- Your conversation is **private to you**. Nobody else — including administrators — can read it, and you cannot read anybody else's.

30.11 What you cannot do

For completeness: you cannot move tickets between columns or tabs, assign work, add or change labels, link tickets together, delete tickets, edit somebody else's ticket, write documentation, see internal notes, or see the team's engineering activity. Those are the delivery team's workflow.

SECTION 31

User Workflows

Six end-to-end walkthroughs, each describing what the implementation actually does.

31.1 Workflow 1 — Standing up a new customer project

Actor: Administrator

FIGURE

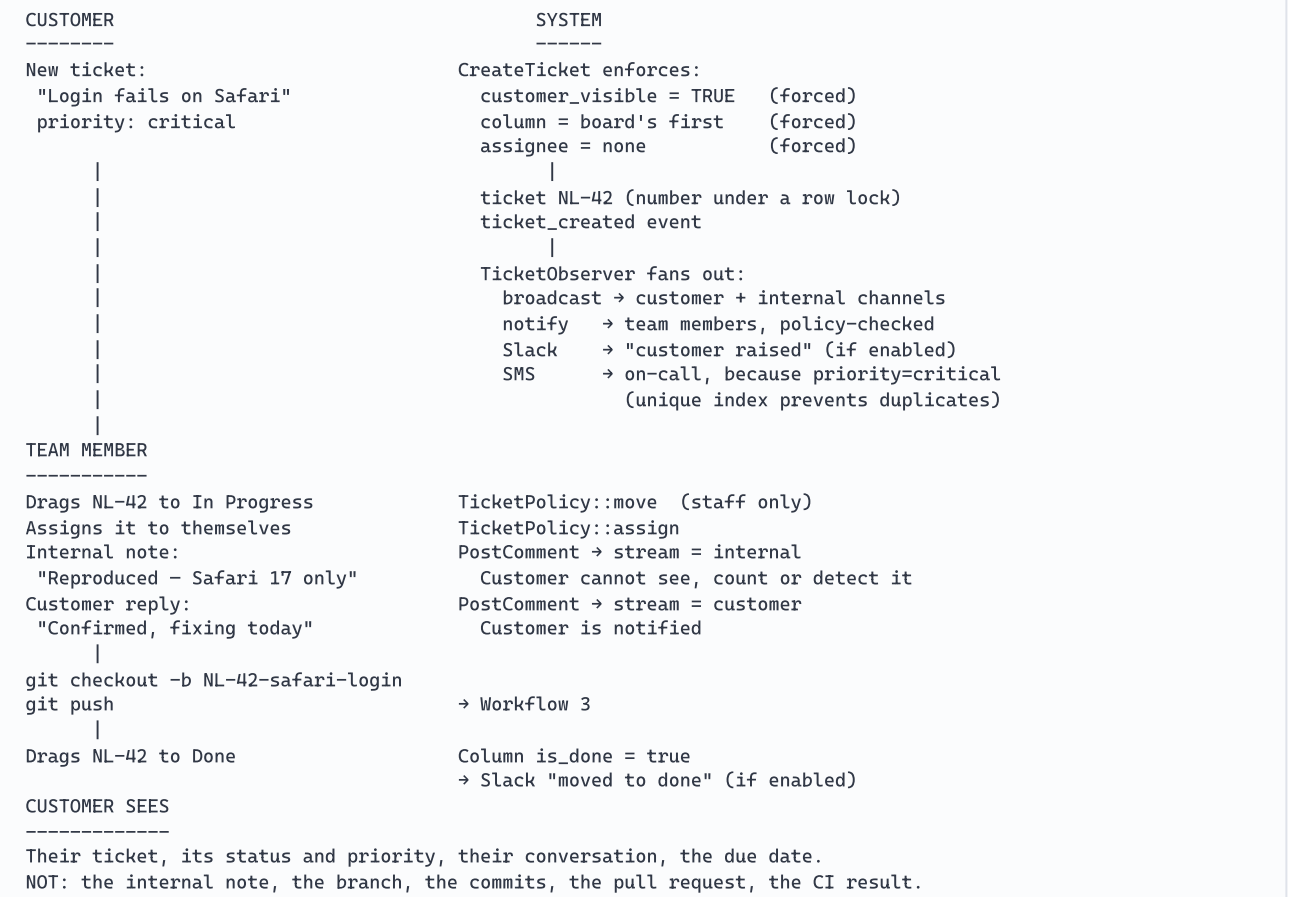
```
1. /admin/users → Create user            Customer contact, role = customer
                                         (password confirmation required)
2. /boards/create → "NutriLens"          slug nutrilens, prefix NL, auto-derived
                                         5 default columns created
3. /boards/nutrilens/edit → Members      Add the delivery team, then the customer
4. /boards/nutrilens/settings            Rename columns, create labels
5. /boards/nutrilens/ai/settings         Attach acme/nutrilens as the primary repository
                                         Write the project context
                                         Leave automatic runs OFF
6. /boards/nutrilens/integrations        Paste the Slack webhook URL (write-only)
                                         Choose events; add the on-call rota
7. GitHub → Settings → Webhooks          URL, secret, JSON, five event types
8. /boards/nutrilens/docs                Write the first pages; publish the ones
                                         the customer should read
```

What has happened underneath: a `board_members` row per person (the entire access grant); `BoardCreated` and `BoardMemberAdded` events in the activity feed; an encrypted Slack webhook URL in `boards.settings.slack`; a `board_repositories` row that is now the **security boundary** for inbound GitHub deliveries.

31.2 Workflow 2 — A customer request, from filing to delivery

Actors: Customer, then Team member

FIGURE



31.3 Workflow 3 — GitHub activity reaching a ticket

Actor: Developer

FIGURE

```
git checkout -b NL-42-safari-login
git commit -m "NL-42 fix cookie SameSite on Safari"
git push
      |
      v
GitHub sends `push` with X-Hub-Signature-256
      |
      v
POST /webhooks/github
 1. throttle:github-webhooks                          429 if exceeded
 2. HMAC-SHA256 over the RAW body, hash_equals        401 if wrong
 3. INSERT webhook_deliveries (unique = the claim)    200 if already seen
 4. dispatch ProcessGitHubWebhookJob → queue `default`
 5. 202 Accepted                                      (well under GitHub's 10s)
      |
      v  worker
WebhookProcessor
 repository "acme/nutrilens" → board_repositories → boards in scope
     └─ no attachment anywhere? → 0 links, recorded as ignored   ←── the boundary
 extract keys from the branch name AND every commit message
 "NL-42" → prefix NL + number 42, resolved ON THOSE BOARDS ONLY
 upsert github_links (ticket, repository, type, external_id)
 write a ticket event ONLY if something actually changed
      |
      v
NL-42 timeline: branch created, commit pushed
Later: pull request opened → state, CI status, author, merged_at
```

31.4 Workflow 4 — Asking the assistant, and confirming a change

Actor: Team member. Workspace at **AI Operator**.

FIGURE

```
"Which tickets on NutriLens are overdue, and raise one to chase the worst?"
      |
      v
AiSessionManager resolves the session — owner, scope, budget, chat mode
AiConfigurationResolver → provider, model, capability mode (board over workspace)
AiCapabilityGuard → Operator: proposals allowed, unattended work NOT
Chat mode = Everything → read tools AND write tools
      |
      v
Context built AS THIS PERSON (their boards, their visible tickets)
      |
      v
Round 1: model calls search_tickets {board: nutrilens, overdue: true}
           → Ticket::visibleTo(THIS USER) → 3 rows → audit row written
Round 2: model calls get_ticket {ticket: NL-31}
           → TicketFinder with THIS USER as viewer → audit row written
      |
      v
Answer: prose + a table of the three overdue tickets
        + ONE proposal: propose_create_ticket
          { title: "Chase NL-31 …", board: nutrilens, priority: high }
        (several new tickets are one proposal too: propose_create_tickets)
      |
      v
Screen shows a PREVIEW and a Confirm button          ← Operator: nothing written yet
      |
Person presses Confirm
      v
ExecuteChatAction
  • re-resolve the board through BoardAccess (the confirming person's membership)
  • AiCapabilityGuard::assertCanExecuteProposal   ← re-checked, in case the mode
                                                    was tightened since the preview
  • Gate: TicketPolicy::create, against the CONFIRMING person
  • call CreateTicket — the ordinary action, with all its own rules
      v
NL-57 exists. Visible to customers unless the board or the request says internal.
ticket_created event. Observer fans out.
```

Under **AI Agent** the preview step is skipped for this reversible action and the ticket is created immediately — **but every check above still runs, identically.** Under **AI Observer** no write tool is sent at all, so no proposal could exist.

A **customer** asking the same question gets the three overdue tickets **they can see** and no proposal, because `allowsProposals()` refuses a customer in every mode.

31.5 Workflow 5 — An AI coding session

Actor: Team member. Workspace at **AI Agent**, `AI_CODE_DRIVER=claude_code`.

FIGURE

```
Ticket NL-42 → "Run AI" → Apply mode
      |
AiRunPolicy::create (staff, board, apply → staff)
AiCapabilityGuard::allowsTicketRun (apply needs Agent)
AiRunCap (manual cap; admins may exceed if configured)
      |
      v
ai_runs row: status=queued, mode=apply, uuid
ticket event: ai_run_queued
ExecuteAiRunJob → queue `ai`, worker service
      |
      v  claim(): queued → running, conditional UPDATE, affected rows checked
preparing   coding runtime available?       no → FAIL, note says exactly what is missing
preparing   clone into storage/app/ai-runs/<uuid>, depth 1
preparing   base branch protected?          yes → FAIL
preparing   create branch ai/NL-42-<uuid>
coding      Claude Code CLI edits the checkout
coding      re-derive the diff from `git status`  ← not from what it claimed
            empty diff?                     yes → FAIL, no PR opened
testing     run AI_VALIDATION_COMMANDS
            non-zero exit?                  yes → FAIL, nothing pushed
pull_request commit as "Workspace AI"
pull_request push refs/heads/ai/NL-42-<uuid>   ← the NEW branch, never the base
pull_request POST /repos/acme/nutrilens/pulls  draft: true
finished    internal note with the PR link
            ai_usage_records row: tokens, cost
            ticket event: ai_run_completed
            Slack if the board subscribes (off by default)
      |
finally     the isolated directory is deleted, however the run ended
```

The customer sees none of this. Not the run, not its status, not its cost, not its failure — `AiRunPolicy` denies as 404.

31.6 Workflow 6 — Publishing documentation to a customer

Actor: Team member

FIGURE

```
/boards/nutrilens/docs → New page "Release process"
      |
CreatePage → customer_visible = FALSE     ← always internal at creation
      |
Author types. Page is internal, so autosave writes body_md directly
through UpdatePage. No draft, no Publish step, no audience to protect.
      |
Ready to share → "Publish"
      |
SetPageVisibility
  • is any ancestor internal?   yes → REFUSED
  |   (exposing the parent is the author's decision, not a side effect)
  • customer_visible = TRUE
  • recorded in the activity feed
      |
      v
Now the page is customer-visible, so the editor's behaviour CHANGES:
autosave writes draft_md via SaveDraft and stays there.
Nothing a customer is reading changes between keystrokes.
      |
"Publish" again → draft_md moves into body_md, edit recorded,
                  other clients told, draft discarded
      |
Later, retracting it:
SetPageVisibility → customer_visible = FALSE
  • cascades DOWN the whole subtree
  • retracting always fails closed: more is hidden, never less
```

SECTION 32

Implementation Status

Every capability discussed in this document, with its honest state. This is the section to read before making a commitment to anybody about what Nexora does.

32.1 Fully implemented

CAPABILITY	EVIDENCE
Three-role model with an absolute customer boundary	`UserRole`, `BoardAccess`, 28 security tests
Board membership as the access-control list	`board_members`, `BoardPolicy`
Kanban boards, columns, drag and drop	`Boards\Show`, `x-sort`
Board tabs: own columns per tab, the tab strip, moving tickets between tabs, AI and MCP aware	`BoardTab`, `App\Actions\Tabs\*`, `MoveTicketToTab`, `TabColumnMapping`
Tickets: type, priority, assignee, estimate, due date, position	`tickets` table, `TicketPolicy`
Labels, checklists, ticket links, attachments	Four dedicated tables
Two-stream conversation, enforced in the action	`CommentStream`, `PostComment`
Append-only ticket history with 20 event types	`ticket_events`, `TicketEventType`
Nested documentation with the ancestor visibility rule	`DocPageFinder`, `SetPageVisibility`
WYSIWYG editing over Markdown, with audience-aware autosave	TipTap, `EditorHtml`, `SaveDraft`
Workspace activity feed	`spatie/laravel-activitylog`, `ActivityLogger`
Command palette across four categories	`GlobalSearch`
In-app notifications, policy-checked per recipient	`NotificationDispatcher`
Realtime with audience-split private channels	Reverb, `BoardBroadcaster`, `channels.php`
Light / dark / system theme, persisted per user	`ThemePreference`, `theme.js`
Team and customer statistics as separate services	`TeamStatistics`, `CustomerStatistics`
Flow metrics from history, not current state	`FlowMetrics`
CSV export (team and customer), PNG chart export	`StatisticsExport`, `chart-export.js`
Workspace AI assistant, panel and full page	`Assistant`, `Chat`
Three-way AI settings model	`AiCapabilityMode`, `AiChatMode`, `AiKnowledgeScope`
Ten retrieval tools, bounded loop, full audit	`AiToolRegistry`, `ai_tool_invocations`
Seven propose-and-confirm write actions	`AiActionType`, `ExecuteChatAction`

AI runs: suggest mode, end to end	`TicketAnalysisService`, `ExecuteAiRunJob`
AI charts, tables and KPI rows, server-rendered from validated specs	`RichResponse\*`
Voice: speech-to-text and text-to-speech	`AiVoiceController`, `OpenAiVoiceProvider`
AI attachments across seven kinds	`Attachments\*`, `ProcessAiAttachment`
Token accounting and cost calculation	`ai_usage_records`, `CostCalculationService`
Two providers with independent field inheritance	`AiConfigurationResolver`, `AiCredentialVault`
Daily run caps with no bypass for automatic runs	`AiRunCap`
GitHub inbound: five event types, signed, de-duplicated	`WebhookProcessor`, `WebhookSignature`
GitHub outbound: push a branch, open a draft pull request	`PullRequestClient`
Slack: per-board, per-event, encrypted webhook URL	`SlackNotifier`, `BoardSlackSettings`
SMS alerts for critical tickets, with retention	`AlertCriticalTicket`, `sms_messages`
Three-queue architecture with a disk-aware worker split	`supervisord.conf`, `railway*.json`
Scheduled housekeeping	`routes/console.php`
Signed-URL downloads with policy delegation	`AttachmentController`, `AttachmentPolicy`
Production storage-durability enforcement	`AppServiceProvider`
Docker multi-stage build, three Railway services	`Dockerfile`, `entrypoint.sh`

32.2 Partially implemented

CAPABILITY	WHAT EXISTS	WHAT IS MISSING
Apply-mode code generation	The whole pipeline: clone, branch, validate, commit, push, draft PR, with every refusal point	`AI_CODE_DRIVER` **defaults to** `unavailable`, in which case an apply run fails immediately with an internal note naming what is missing. Requires `claude_code` plus the binary and a key in the worker's environment
Repository cloning	`GitClient`, `RepositoryCheckout`, context summarisation	`AI_REPOSITORY_CLONE_ENABLED` **defaults to** `false`. Suggest mode then reasons from metadata and the ticket alone, and **records that the working tree was unavailable** rather than pretending
External knowledge lookup	`ExternalKnowledgeProviderInterface`, a tool, config, and a refusing default	**No driver ships.** `AI_KNOWLEDGE_DRIVER` is empty and resolves to `UnavailableExternalKnowledge`. Outside Project mode still works — what is absent is live third-party lookup. Adding one is a class plus a `match` arm

OpenAI model catalogue	A working adapter and provider selection	Context windows and pricing are **not stated** for OpenAI models, so they render as blank and report no cost. Vision is opt-in per id
SMS delivery receipts	`SMS_46ELKS_DLR_URL` may be configured	**No receiving endpoint is exposed.** Setting it would produce retries at the provider
Email	Password reset works	No other transactional mail. No email notification channel — notifications are in-app, Slack or SMS
Board archiving	`archived_at` , `notArchived()` scope	No UI to archive or restore a board
Ticket type	Stored, filterable, in AI schemas	No workflow behaviour differs by type

32.3 Planned / Not Yet Implemented

CAPABILITY	STATUS
AI Standup Generator	**Not implemented.** Searched the whole repository: the only match is a test fixture filename (`standup.mp3`). There is no standup service, command, component, route, view or job. The building blocks exist — `get_activity` , `get_code_activity` , `ActivityReader` , `GithubLinkReader` — so it is buildable, but **nothing generates a standup today**
General REST / JSON API for third parties	No `routes/api.php` , no OpenAPI spec. The only machine-facing interface is the remote MCP server (`/mcp` , OAuth), which reads and makes the changes the signed-in person could make by hand
Webhooks **out** of Nexora	Only Slack and SMS leave the application
Email notifications	Password reset only
Two-factor authentication	Not implemented
SSO / SAML	Not implemented (OAuth exists only as the authorization server for MCP connectors)
Email verification	Column exists, no flow
Documentation version history	Only the current body and one draft
Documentation page comments	Not implemented
Collaborative editing (multiple cursors)	Autosave is single-author
Export documentation to PDF/Word	Not implemented
Per-ticket CSV export	Deliberately absent — aggregates only
Time tracking	`estimate` is stored; nothing logs actual time
Sprints / iterations / epics	Not implemented

Recurring tickets, templates	Not implemented
Multi-tenant organisations	**No workspace entity.** One deployment is one workspace; `BoardIdentifiers` notes the two closures that would need to become organisation-aware
Custom fields	Not implemented
Ticket import / export	Not implemented
Browser / end-to-end tests	None
JavaScript unit tests	None
CI pipeline configuration	Not present in the repository
Content-Security-Policy, HSTS headers	Not set

32.4 Documentation gaps and inconsistencies found

Issues discovered while writing this document. None is a functional defect in itself, but each is worth an owner.

#	FINDING	IMPACT	SUGGESTED ACTION
1	`README.md` **is empty (0 bytes).**	A developer cloning the repository gets no orientation at all. The `supervisord.conf` comments and `config/ai.php` both refer readers to "the deployment notes in README.md", which do not exist	Populate it, or point it at this document
2	**Product naming is inconsistent.** `WORKSPACE_NAME` defaults to `Aqueduct Workspace`, the package is `aqueduct/workspace`, but the AI system prompt, comments and one aria-label say *Nexora*	A deployment that has not set `WORKSPACE_NAME` presents as Aqueduct while its assistant calls itself Nexora	Decide the product name; set `WORKSPACE_NAME` and `WORKSPACE_SHORT_NAME` accordingly
3	**No version identifier anywhere.** No git tag, no `version` in `composer.json`, no application constant	Nothing can state which build is deployed; support and rollback have no vocabulary	Adopt tags, and surface a version in `/health`
4	`AiVisualizationSecurityTest` **has a date-sensitive assertion.** It forbids the digit `7` anywhere in the customer's answer, but the report header carries today's date	The suite fails on roughly a third of calendar days, for no real reason — and it fails in the *security* suite, which is the one people trust most	Narrow the assertion to `(total 7)` rather than `7`

5	**Two pre-existing `ThemePreferenceTest` failures.**	Two red tests that are not a regression	Fix or quarantine, so "green" means green
6	**`DemoUserSeeder` contains passwords in source.** Documented in the class as a deliberate demo trade-off	Anybody who can read the repository can sign in to a demo deployment	Change the accounts before anything real lands, as the class itself instructs
7	**Default capability mode is `agent` in config but `observer` in the `ai_settings` column default.** Both are deliberate and documented, but they disagree	A deployment's effective posture depends on whether a settings row exists	Decide one, or document the interaction in the admin screen
8	**No Content-Security-Policy or HSTS header.**	Two cheap defence-in-depth controls are absent	Add both at nginx
9	**`SESSION_SECURE_COOKIE=false` in `.env.example`.** Correct for local HTTP; wrong for production	An easy-to-miss production hardening step	Note it in a deployment checklist
10	**No CI configuration in the repository.** A 139-file suite with a dedicated fast security gate exists but nothing runs it automatically	Regressions rely on somebody remembering	Add a workflow running `composer test:security` on every push and the full suite on merge
11	**Ticket `type` has no behaviour.** Stored and filterable, but nothing differs between a bug and a feature	A field that looks meaningful but is decorative	Either give it behaviour or document it as a label
12	**Board archiving has no UI.** The column and scope exist	Dead capability	Add the control, or remove the column

SECTION 33

Glossary

TERM	MEANING IN NEXORA
Administrator	`admin`. Sees every board without membership; the only role that may create boards, manage users, hold AI credentials and set the workspace capability mode
AI Agent	The most permissive capability mode. Adds unattended work: automatic runs, apply-mode pull requests, and reversible changes carried out without a confirmation step
AI Observer	The most restrictive capability mode. Reads and answers; writes nothing, **not even an internal note**, so no ticket run may start
AI Operator	The middle capability mode. Proposes changes a person confirms, and permits manual suggest runs
Apply mode	An AI run that changes code: isolated clone, new branch, validation, commit, push, **draft** pull request. Never merges, never pushes to a protected branch
Assignee	The staff member doing the work. Customers are never assignable
Attachment	A file on a ticket, comment, documentation page or AI conversation. Has no visibility of its own — it inherits its owner's
Board	The unit of work, access and configuration. Everything hangs off one. **There is no entity above it**
Board member	An explicit `board_members` row. **This is the access grant**; administrators bypass it
Capability mode	The administrator's ceiling on what the AI may ever attempt. See AI Observer / Operator / Agent
Chat mode	The asker's choice per conversation: Reading, Writing or Everything
Customer	`customer`. The security boundary of the product. May never observe internal content, on any board, including boards they belong to
Customer stream	The ticket conversation the customer takes part in
Customer-visible	The positive visibility flag on a ticket or page. **Defaults to false**, so anything created by code that forgets to set it is internal
Digest	The shortened form of an AI attachment sent on turns after the first — name, kind, shape, outline, leading excerpt
Draft (documentation)	The autosave buffer for a **published** page. Nothing renders it to a reader

Draft PR	A pull request opened by apply mode, marked draft so it arrives visibly unfinished rather than looking like finished work
Everything mode	The chat mode offering both read and write tools
Internal	Not visible to customers. The default for tickets, comments and documentation pages
Internal note	A comment in the internal stream. Customers cannot read it, count it, or detect that it exists
Knowledge scope	Where an answer may be sourced from: **Project only** (default) or **Outside Project** (project **and** general knowledge)
Label	A board-scoped tag. Staff-only to change
Outside Project	The additive knowledge scope. Never replaces the project — it adds general knowledge alongside it, and widens data access by nothing
Primary repository	The repository apply mode targets when a run does not name one
Proposal	A change the AI has *described*. A preview until a person confirms it — at which point it is re-resolved, re-authorized and executed through the ordinary action
Queue	Background work. Three: `ai` (runs), `default` (broadcasts, notifications, webhooks, Slack, SMS), `attachments` (AI file processing, in the web container)
Reading mode	The default chat mode. Lookup tools, no write tools
Run	One AI working session on one ticket. Suggest or apply
Session (AI)	One conversation. Belongs to **one person**, including against administrators
Slug	The URL identifier for a board, a board tab or a documentation page. Primary keys are never exposed
Staff	Administrators and team members together. The set that may see internal content
Stream	Which conversation a comment belongs to: `customer` or `internal`
Suggest mode	An AI run that analyses and posts an internal note. Changes no code
Tab	One area of work inside a board ("Admin dashboard", "Public website"), with its own columns. A ticket's tab is its column's tab. The leftmost tab is the board's default
Team member	`team`. Full working access to boards they are a member of; not workspace administration
Ticket	A unit of work. Addressed by `PREFIX-NUMBER`, e.g. `NL-42`
Ticket key	The human identifier — the board's prefix, a hyphen, the per-board number. What GitHub branches and commits reference

Ticket prefix	A board's 2–6 character code, globally unique
Tool	A read-only lookup the AI may call. Runs **with the asking person as the viewer**, and is audited whatever the outcome. Over MCP an AI app also has **write tools**, which make a change as the person, under their permissions
Webhook (inbound)	GitHub telling Nexora about code activity. Authenticated by HMAC-SHA256 over the raw body
Webhook (outbound)	A Slack incoming-webhook URL. A bearer credential, so it is encrypted at rest and write-only in the UI
Workspace	The whole deployment. **Not an entity** — there is no workspace table. Used in this document to mean "everything in this installation"
Workspace AI	The assistant. Two surfaces: the panel (everyone, read-only for customers) and the full-page board chat (staff only)

SECTION 34

Appendix — Document Provenance

This document was produced by reading the Nexora repository at commit `30bfc83`. The following were examined in full or in substantial part:

AREA	FILES
Configuration	`composer.json`, `package.json`, `.env.example` (183 variables), `phpunit.xml`, `vite.config.js`, all 19 files in `config/`
Routing	`routes/web.php`, `routes/channels.php`, `routes/console.php`, `bootstrap/app.php`, `bootstrap/providers.php`
Container and deployment	`Dockerfile`, `docker/entrypoint.sh`, `docker/nginx.conf`, `docker/php.ini`, `docker/php-fpm.conf`, `docker/supervisord.conf`, `railway.json`, `railway.worker.json`, `railway.reverb.json`
Domain	All 25 models, all 42 migrations, all 8 policies, all 27 enums, key actions and services across AI, GitHub, Slack, SMS, statistics, search and attachments
Presentation	All 36 Livewire components (class documentation), the 107-file `resources/` tree, the 16 JavaScript modules
Quality	The test-file inventory (139 files) and a full test run

Verification performed while writing:

- The full test suite was executed: **1,718 passed, 3 failed, 6,436 assertions, 311 seconds.**
- The three failures were isolated, reproduced individually and diagnosed. All three are test defects; none is an application defect. See section 25.
- Every environment variable named in this document was checked against `.env.example` and the configuration file that reads it.
- Every permission claim was derived from middleware, policies, actions, query scopes or tool `availableTo()` methods — never assumed.
- The claim that no AI Standup Generator exists was verified by a repository-wide search.
- One claim in an early draft — that login was not rate limited — was found to be wrong on verification and corrected; login is throttled at 5 attempts, keyed on email and IP.

No application code was modified in the production of this document. The repository changes are limited to two things: the new `docs/` directory, containing this document, its Markdown source and its build script; and four lines appended to `.gitignore`, so that the build tooling's `node_modules` and intermediate render artefacts are not committed.